Cardiff Metropolitan University

Cardiff School of Technology

BSc Data Science

VisionAgri AI – Your agriculture disease detector

Submitted in May

2026

By

I D Navindu Sankalpa Karunarathna

(st20242922 | CL/BSCDS/CMU/08/76)

This dissertation is submitted in partial

fulfillment of the requirements for the

degree of

Bachelor of Science in Data Science (BSc DS)

Declaration

DECLARATION

This work is being submitted in partial fulfillment of the requirements for the degree of

BSc (Hons) Data Science

and has not previously been accepted in substance for any degree and is not being concurrently submitted in candidature for any degree.

Signed - .....

(Candidate) Date - 07/03/26

SUPERVISOR'S DECLARATION STATEMENT

Student Name - ILANDARI DEWAGE NAVINDU SANKALPA KARUNARATHN

Supervisor's Name - MR THARAKA GALPAYA

I acknowledge that the above named student has regularly attended the meeting, and actively engaged in the dissertation supervision process.

Signed - .....

(Supervisor) Date - 07/03/2026

Acknowledgement

I would like to express my sincere gratitude to my project supervisor, Mr. Tharaka Galpaya, for his continuous guidance, constructive feedback, and encouragement throughout the development of this project. His expertise and patience were invaluable in shaping both the technical implementation and the academic presentation of this work. I am also grateful to the academic staff at Cardiff Metropolitan University and the ICBT campus for providing the foundation and resources that made this project possible. I extend my heartfelt thanks to my family for their unwavering support, understanding, and motivation during the demanding periods of this project; their belief in my abilities kept me focused and determined. Furthermore, I would like to express my appreciation to Mr. Sampath Dissanayake for his kindness in providing the essential network connectivity required to complete the development and deployment of this project. Finally, I would like to thank the 68 respondents who participated in the user requirements survey, whose insights directly shaped the features and priorities of VisionAgri AI.

Abstract

VisionAgri AI is a web-based diagnostic platform designed to mitigate crop loss for smallholder farmers by leveraging multi-task deep learning to classify diseases and quantify infection severity in chili pepper and tomato leaves. The system utilizes two distinct convolutional neural network architectures EfficientNetV2-B0, which achieved 100% validation accuracy and a 0.9684 Dice coefficient, and a lightweight MobileNetV3-Small alternative to provide high-precision pixel-level segmentation. To ensure diagnostic reliability, a MobileNetV2-based gatekeeper classifier filters out-of-scope uploads, while the integration of the Open-Meteo weather API and an OpenRouter-hosted large language model delivers context-aware treatment recommendations. Containerized with Docker and deployed on a cloud VPS at visionagri.app, the application demonstrates how transfer learning and multi-task architectures can be successfully synthesized into a production-grade tool tailored for resource-constrained agricultural settings.

Table of Contents

Table of Tables

Table of Figures

CHAPER 01 – Introduction

Agriculture constitutes the backbone of global food security and economic stability, employing approximately one billion people worldwide and contributing substantially to the gross domestic product of developing nations (Talaviya et al., 2020). In Sri Lanka, agriculture accounts for roughly 7% of the national GDP and supports the livelihoods of approximately 25% of the labor force, with vegetable cultivation, particularly tomato and pepper crops, representing a significant proportion of horticultural output (Central Bank of Sri Lanka, 2023). Despite the sector's importance, plant diseases remain one of the most persistent threats to crop yield and quality, causing estimated annual losses of 20-40% of global agricultural production (Singh, Sharma and Singh, 2020).

Conventional disease diagnosis relies predominantly on visual inspection by trained agronomists or laboratory-based pathological analysis, both of which are time-consuming, expensive, and inaccessible to smallholder farmers in rural regions (Ferentinos, 2018). The advent of deep learning, particularly Convolutional Neural Networks (CNNs) has opened a transformative pathway for automated, image-based plant disease detection that can deliver real-time diagnoses with accuracies rivalling those of human experts (Mohanty, Hughes and Salathe, 2016). This project, titled VisionAgri AI, harnesses these advances to deliver an accessible, web-based plant disease detection and advisory system targeting pepper and tomato crops in the Sri Lankan agricultural context.

1.1 Background

Plant diseases are caused by a diverse range of pathogens including fungi, bacteria, viruses, and nematodes, each producing characteristic visual symptoms on leaves, stems, and fruits (Shoaib et al., 2023). Bacterial spot in pepper and Tomato Yellow Leaf Curl Virus transmitted by the whitefly are among the most economically damaging diseases in tropical and subtropical agriculture (Fuentes et al., 2017). In Sri Lanka, tomato production regularly suffers yield reductions of 30-70% during severe tomato leaf curl virus outbreaks, while bacterial spot in pepper can cause defoliation and fruit lesions that render produce unmarketable (Department of Agriculture Sri Lanka, 2022).

Traditional disease management strategies depend on early and accurate identification, yet most Sri Lankan smallholders lack access to plant pathology expertise. Extension

services are under-resourced, with approximately one agricultural officer serving every 1,500 farming families (FAO, 2021). Laboratory-based molecular diagnostics, such as polymerase chain reaction (PCR) assays, offer high specificity but require specialized equipment, trained personnel, and typically 24-72 hours for results, a delay during which diseases can spread exponentially (Singh, Sharma and Singh, 2020).

The proliferation of smartphones and mobile internet connectivity, even in rural agricultural communities, has created an unprecedented opportunity for technology-mediated solutions. According to the Telecommunications Regulatory Commission of Sri Lanka (2023), mobile internet penetration exceeded 58% nationally, with 4G coverage reaching approximately 85% of the population. This infrastructure makes web-based diagnostic tools practically viable for adoption by farming communities.

Deep learning has demonstrated remarkable success in computer vision tasks relevant to agriculture. Mohanty, Hughes and Salathe (2016) achieved 99.35% accuracy using GoogLeNet and AlexNet on the PlantVillage dataset comprising 54,306 images across 38 disease classes. Ferentinos (2018) extended this work to 58 classes across 25 plant species, achieving 99.53% with a VGGNet-based architecture. More recently, EfficientNet architectures (Tan and Le, 2019) have demonstrated superior accuracy-to-parameter efficiency, making them particularly suitable for resource-constrained deployment scenarios. MobileNetV3 (Howard et al., 2019), designed specifically for mobile and edge devices, offers competitive accuracy at dramatically reduced computational cost, achieving top accuracy of 75.2% on ImageNet with only 5.4 million parameters.

However, the existing literature reveals several critical gaps. First, most studies treat disease detection purely as a classification problem, neglecting the spatial localization and severity quantification that are essential for informed treatment decisions (Zainab and Mahum, 2025). Second, few systems integrate disease diagnosis with contextual environmental data, such as weather forecasts that significantly influence disease progression and treatment timing (Fuentes et al., 2017). Third, the practical deployment of trained models as accessible end-user applications remains largely unexplored in academic literature, with most studies terminating at the model evaluation stage (Shoaib et al., 2023). VisionAgri AI addresses all three gaps by combining dual-head classification-segmentation models, real-time weather integration, and a fully deployed web application.

1.2 Problem Statement

Smallholder farmers cultivating pepper and tomato crops in Sri Lanka face a persistent challenge, the inability to rapidly and accurately diagnose plant diseases in the field. This problem manifests across several interconnected dimensions.

1. **Diagnostic Inaccessibility** – Agricultural extension officers are scarce, and laboratory testing is expensive and slow. A survey conducted as part of this project revealed that 78% of respondents rely solely on visual inspection or lack any reliable method for disease identification, and 78% reported experiencing crop losses due to late or incorrect diagnosis.
2. **Lack of Severity Quantification** – Even when a disease is correctly identified, farmers have no objective means to quantify its severity. Treatment decisions, such as whether to apply fungicides, remove infected plants, or take no action, depend critically on the extent of infection, yet this is currently estimated through subjective visual assessment.
3. **Absence of Contextual Advisory** – Disease progression is heavily influenced by environmental conditions. High humidity and moderate temperatures accelerate bacterial spot in pepper, while tomato yellow leaf curl transmission peaks during warm, dry conditions that favor whitefly populations. Existing diagnostic tools, where available, provide classification results in isolation without integrating weather forecasts to contextualize treatment urgency.
4. **Technology Barriers –** While several research prototypes have demonstrated high classification accuracy, they remain confined to laboratory environments. The gap between a trained model achieving 99% accuracy on a benchmark dataset and a usable tool accessible to a farmer with a smartphone remains largely unbridged.

These challenges collectively result in delayed interventions, inappropriate treatments, and ultimately, significant economic losses for farming communities. There is a clear need for an integrated system that combines accurate disease classification, spatial severity analysis, environmental context, and actionable treatment recommendations within an accessible, deployable platform.

1.3 Aims and Objectives

1.3.1 Aim

The primary aim of this project is to design, develop, and deploy a web-based artificial intelligence system, VisionAgri AI that enables real-time detection and severity assessment of diseases in pepper and tomato crops, augmented by weather-aware treatment recommendations, thereby reducing diagnostic delays and supporting informed crop management decisions for smallholder farmers.

1.3.2 Objectives

- To conduct a comprehensive literature review of deep learning approaches for plant disease detection, identifying current methodologies, datasets, architectures, and research gaps relevant to the project scope.
- To collect, annotate, and preprocess a curated dataset of pepper and tomato leaf images with pixel-level disease segmentation masks using Label Studio, ensuring balanced class representation and annotation quality.
- To design and train dual-head deep learning models (EfficientNetV2-B0 and MobileNetV3Small) that simultaneously perform four-class disease classification and three-channel semantic segmentation for disease area localization.
- To develop a MobileNetV2-based gatekeeper binary classifier that validates input images, accepting only pepper and tomato leaves and rejecting unsupported plant types before disease inference.
- To implement a RESTful API backend using FastAPI that orchestrates the complete inference pipeline including image preprocessing, model prediction, severity calculation, and disease overlay generation.
- To integrate real-time weather forecast data from the Open-Meteo API and large language model (z-ai free) powered treatment recommendations via OpenRouter, providing contextually relevant agricultural advisory.
- To develop a responsive, accessible web-based user interface using HTML5, Tailwind CSS, and vanilla JavaScript that supports image upload, model selection, geolocation capture, and comprehensive result visualization.
- To containerize the application using Docker and deploy it on cloud infrastructure, ensuring scalability, reproducibility, and production readiness.

- To evaluate system performance through model accuracy metrics, functional testing, integration testing, and user acceptance mapping against survey-derived requirements.

1.4 Proposed Solution

VisionAgri AI addresses the identified problems through a modular, end-to-end system architecture comprising four principal components, a gatekeeper classifier, dual-head disease detection models, an intelligent backend, and a responsive web interface.

1.4.1 Gatekeeper Binary Classifier

The first line of protection is a MobileNetV2-based binary classifier trained on 20,638 images from the PlantVillage dataset. This model categorizes input images as either "accepted" (pepper or tomato leaves that the disease model can analyze) or "rejected" (all other plant types including potato, and tomato disease categories outside the system's scope). The gatekeeper prevents erroneous predictions on out-of-distribution inputs, a critical reliability concern identified in the literature (Zainab and Mahum, 2025).

1.4.2 Dual Head Disease Detection Models

The core diagnostic capability is provided by two interchangeable deep learning models.

EfficientNetV2-B0 Model (Primary) – This model will employ an EfficientNetV2-B0 encoder pretrained on ImageNet, augmented with a custom Color Statistics Branch that extracts green-to-red channel ratios, a biologically motivated feature capturing chlorophyll degradation patterns characteristic of disease progression. The encoder feeds two output heads: a classification head producing SoftMax probabilities across four classes (Pepper Healthy, Pepper Bacterial Spot, Tomato Healthy, and Tomato Yellow Leaf Curl Virus), and a segmentation head producing a 256×256×3 pixel-level mask delineating background, healthy leaf tissue, and diseased regions. Training employed focal loss with class-specific weighting (pepper weight = 2.5) and a combined segmentation loss incorporating cross-entropy, Dice loss, and false-positive/false-negative penalties. The model will be trained in two phases, 24 epochs with a frozen backbone followed by 12 epochs of fine-tuning.

MobileNetV3Small Model (Lightweight Alternative) – Designed for resource-constrained deployment, this model uses a MobileNetV3Small backbone with approximately 2.5 million parameters compared to the EfficientNet's 7 million. It follows the same dual-head architecture but uses color standard deviation instead of variance in its statistics branch.

1.4.3 FastAPI Backend

The backend will be implemented using FastAPI. The /api/analyze endpoint will orchestrate a multi-stage pipeline,

1. Input validation (file type size less than10MB).
2. Gatekeeper classification.
3. image preprocessing and model inference.
4. Severity calculation from segmentation masks.
5. Disease overlay generation.
6. Optional weather forecast retrieval from the Open-Meteo API based on the user's geolocation.
7. Optional LLM-powered treatment recommendations via OpenRouter.

The backend will handle errors gracefully, returning appropriate HTTP status codes (400 for invalid input, 422 for gatekeeper rejection, 500 for internal errors) with descriptive messages.

1.4.4 Web-Based User Interface

The frontend will be a single-page application built with HTML5, Tailwind CSS, and vanilla JS, hosted on GitHub Pages at visionagri.app. Key features will include drag-and-drop image upload with real-time preview, model selection (EfficientNet or MobileNetV3), browser-based geolocation capture for weather integration, an AI recommendations toggle, sample test images for immediate evaluation, and comprehensive result display including disease name, confidence score, severity percentage, classification score distribution, segmentation overlay, treatment steps, and care tips. The interface will enforce a 75% minimum confidence threshold, blocking display of low-confidence predictions to prevent misleading diagnoses.

1.5 Scope and Limitations

1.5.1 Project Scope

The system's scope will be to detect four conditions across two crop species, Pepper Healthy, Pepper Bacterial Spot, Tomato Healthy, and Tomato Leaf Curl Virus. These were selected based on their economic significance in Sri Lankan agriculture and the availability of annotated training data. The system will operate as a web based application accessible via any modern browser, requiring only an internet connection and camera access.

1.5.2 Limitations

- The system is designed to support only two crop species and four conditions. Expansion to additional crops and diseases would require new training data and model retraining.
- Disease detection accuracy will be dependent on image quality, lighting conditions, and leaf orientation. Images captured in poor lighting or at extreme angles may produce unreliable results.
- The system will require an internet connection for all operations. While the survey indicated offline functionality as important (49% rated it 4-5 out of 5), the current architecture relies on server-side inference due to model sizes (approximately 200MB total for all three models).
- Weather based recommendations will depend on the user granting browser geolocation permission and the availability of Open-Meteo API service.
- The LLM-generated treatment recommendations, while contextually relevant, will have to be treated as advisory and not as a substitute for professional agronomic consultation.
- The training dataset of 381 original images (will be augmented to 3324) is relevantly small compared to the original size (54,000 images) of PlantVillage. Model generalization to field conditions with complex background, mixed infections, and varying leaf ages will require further validation.

CHAPTER 02 – Literature Review

This chapter critically examines the existing body of research relevant to the VisionAgri AI project. The review is organized thematically, beginning with the broader context of artificial intelligence in agriculture, progressing through the evolution of deep learning architectures for plant disease detection, exploring transfer learning and semantic segmentation techniques, and concluding with an analysis of precision agriculture tools and the identification of research gaps that this project seeks to address. The literature review includes peer-reviewed journal articles, conference proceedings, and authoritative technical reports.

2.1 Artificial Intelligence in Modern Agriculture

The application of AI in agriculture has expanded rapidly over the past decade, driven by advances in computing hardware, the availability of large-scale datasets, and the pressing need to increase agricultural productivity to feed a growing global population. Talaviya et al. (2020) provided a comprehensive review of AI implementations in agriculture, noting that applications span irrigation optimization, pesticide and herbicide application, soil health monitoring, yield prediction, and pest and disease management. The authors highlighted that AI-based precision agriculture has the potential to reduce water usage by 25%, decrease pesticide application by up to 50%, and increase crop yields by 10-15% compared to conventional farming practices.

Singh, Sharma and Singh (2020) conducted an extensive review of imaging techniques for plant disease detection, categorizing approaches into traditional machine learning methods, such as support vector machines, k-nearest neighbors, and random forests, and deep learning methods based on convolutional neural networks. Their analysis demonstrated a clear paradigm shift: prior to 2015, most of the published work relied on handcrafted feature extraction pipelines (color histograms, texture descriptors such as GLCM and LBP, and shape features) followed by classical classifiers. Post 2015, deep learning approaches progressively dominated the literature, offering the critical advantage of automatic feature learning directly from raw pixel data, thereby eliminating the need for domain-specific feature engineering.

This transition is significant because handcrafted features are inherently limited by the designer’s assumptions about which visual characteristics are discriminative. Disease symptoms, however, can manifest as subtle color changes, complex textural patterns,

or irregular spatial distributions that are difficult to capture through predefined descriptors. Deep learning models, particularly CNNs, learn hierarchical feature representations, from low-level edges and textures to high-level semantic concepts, that can capture these nuances with far greater fidelity (Shoaib et al., 2023).

2.2 Convolutional Neural Network for Plant Disease Detection

2.2.1 Foundational Studies

The seminal work by Mohanty, Hughes and Salathe (2016) established the viability of deep learning for plant disease classification. The authors trained two architectures, AlexNet (Krizhevsky, Sutskever and Hinton, 2012) and GoogLeNet (Szegedy et al., 2015), on the PlantVillage dataset, which comprised 54,306 images of healthy and diseased leaves across 14 crop species and 26 diseases. Their best model achieved 99.35% accuracy on the held-out test set using transfer learning from ImageNet-pretrained weights. Critically, the authors acknowledged a significant limitation, the PlantVillage dataset consisted of laboratory-captured images with uniform backgrounds, raising concerns about model generalization to field conditions where leaves appear against complex, variable backgrounds.

Ferentinos (2018) extended this line of research by training five CNN architectures, AlexNet, AlexNetOWTBn, GoogLeNet, Overfeat, and VGGNet, on a substantially larger dataset of 87,848 images containing 25 plant species. The best-performing model, a VGGNet variant, achieved 99.53% accuracy. Ferentinos emphasized the practical implications of this result, arguing that such accuracy levels could enable smartphone-based diagnostic tools accessible to farmers in developing countries. However, the study shared the same dataset bias limitation identified by Mohanty, Hughes and Salathe (2016), as all images were drawn from the same controlled-environment PlantVillage repository.

Recognizing this limitation, Singh et al. (2019) introduced the PlantDoc dataset, comprising 2,598 images extracted from the internet across 13 plant species and 17 disease categories. Unlike PlantVillage, these images featured natural, variable backgrounds representative of real-world conditions. The authors demonstrated that models trained exclusively on PlantVillage performed poorly on PlantDoc images, with accuracy dropping from above 95% to below 60%, thereby confirming the dataset bias concern. This finding underscores the importance of training data diversity and has

direct relevance to VisionAgri AI, which uses a custom-curated dataset with images sourced from multiple real-world scenarios.

2.2.2 Disease-Specific Detection Studies

Within the domain of specific crop diseases, Fuentes et al. (2017) developed a robust deep-learning-based detector for real-time recognition of tomato plant diseases and pests. Their approach employed three meta-architectures, Faster R-CNN, Region-based Fully Convolutional Network (R-FCN), and Single Shot Multibox Detector (SSD), combined with feature extractors including VGG-16 and ResNet-50. The system detected 10 diseases and pest classes including Tomato Yellow Leaf Curl Virus, achieving a mean average precision of 83.06% with Faster R-CNN and VGG-16. Notably, this was among the first studies to frame plant disease detection as an object detection problem rather than whole-image classification, enabling spatial localization of symptoms, a capability that VisionAgri AI extends through semantic segmentation.

Agarwal et al. (2020) specifically investigated tomato leaf disease detection using a custom CNN architecture, reporting 91.2% accuracy across 10 tomato disease classes. Their study highlighted the challenge of inter-class similarity, where diseases such as early blight and late blight produce visually similar symptoms, leading to classification confusion. This challenge is analogous to the pepper bacterial spot versus pepper healthy discrimination addressed by VisionAgri AI, where early-stage infections produce subtle visual differences that require specialized feature extraction, motivating the incorporation of a color statistics branch in the present system's architecture.

Ma et al. (2022) focused on maize leaf disease identification using deep transfer convolutional neural networks, evaluating DenseNet-121, ResNet-50, and InceptionV3 with transfer learning from ImageNet. Their results demonstrated that DenseNet-121 achieved the highest accuracy of 98.06% on a dataset of 18,572 images across three maize diseases. The study confirmed that transfer learning consistently outperforms training from scratch, particularly when the target domain dataset is small, a finding directly applicable to VisionAgri AI, where the original dataset of 381 images necessitates effective transfer learning strategies.

2.3 Transfer Learning and Modern Architecture

Transfer learning, the practice of initializing a model with weights learned on a large source domain (typically ImageNet with 1.2 million images across 1,000 classes) and fine-tuning on a smaller target domain, has become the standard in agricultural computer vision. Shoaib et al. (2023) conducted a systematic review of 100+ papers on deep learning for plant disease detection published between 2016 and 2023, finding that 87% of studies employed some form of transfer learning. The authors categorized transfer-learning strategies into three approaches: feature extraction (freezing pretrained layers and training only a new classifier head), fine-tuning (unfreezing some or all layers for end-to-end training), and two-phase training (initial feature extraction followed by fine-tuning with a reduced learning rate).

VisionAgri AI will adopt the two-phase training strategy, which Shoaib et al. (2023) identified as producing the most robust results for small datasets. In Phase 1, the pretrained encoder backbone will be frozen and only the newly added classification and segmentation heads are trained, allowing these layers to learn task-specific representations without disrupting the generalized features captured during ImageNet pretraining. In Phase 2, the entire network will be unfrozen and trained end-to-end with a significantly reduced learning rate (typically 10x to 100x lower), enabling the encoder to adapt its representations to the nuances of plant disease imagery while retaining the structural knowledge from ImageNet.

2.3.1 EfficientNet Architecture

The EfficientNet family of architectures, introduced by Tan and Le (2019), represents a significant advance in neural network design efficiency. The key innovation is compound scaling, a principled method for uniformly scaling network depth, width, and input resolution using a set of fixed scaling coefficients derived through neural architecture search. The base model, EfficientNet-B0, was discovered through NAS optimization and achieves 77.1% top 1 accuracy on ImageNet with only 5.3 million parameters. Subsequent variants (B1 through B7) scale this base model to progressively larger sizes, with EfficientNet-B7 achieving 84.3% accuracy with 66 million parameters.

EfficientNetV2, a later evolution introduced by Tan and Le (2021), incorporates Fused-MBConv layers in the early stages of the network and employs progressive learning, a

training methodology that adaptively increases image resolution and regularization strength during training. EfficientNetV2-B0 achieves comparable accuracy to EfficientNet-B0 while training 4x faster, making it particularly attractive for iterative experimentation during model development. VisionAgri AI selects EfficientNetV2-B0 as its primary backbone due to this favorable balance of accuracy, training efficiency, and inference speed.

Akash et al. (2023) demonstrated the effectiveness of EfficientNet for plant disease detection, reporting that EfficientNet-B0 outperformed traditional architectures including VGG-16 and ResNet-50 on a 38-class subset of PlantVillage. The authors attributed this superiority to the compound scaling approach, which ensures that increased model capacity is distributed optimally across all network dimensions rather than being concentrated in a single dimension (eg; only depth, as in ResNet).

2.3.2 MobileNet Architecture

The MobileNet family of architectures, developed by Google, targets deployment on mobile and edge devices where computational resources are severely constrained. MobileNetV1 (Howard et al., 2017) introduced depth wise separable convolutions, which factories standard convolutions into a depth wise convolution (spatial filtering per channel) and a pointwise convolution (channel mixing), reducing computational cost by a factor of 8-9x compared to standard convolutions with minimal accuracy loss.

MobileNetV2 (Sandler et al., 2018) introduced inverted residual blocks with linear bottlenecks, where the architecture expands the feature map to a higher dimensionality, applies a depth wise convolution, and then projects back to a lower dimensionality, with skip connections between the narrow bottleneck layers. This design preserves important features while keeping the overall parameter count low. VisionAgri AI employs MobileNetV2 as the backbone for its gatekeeper binary classifier, leveraging its efficiency for the relatively simpler task of binary plant type validation.

MobileNetV3 (Howard et al., 2019) further refined the architecture through hardware-aware NAS and the NetAdapt algorithm, incorporating squeeze-and-excitation modules and the hard-swish activation function. MobileNetV3-Small, designed for the most resource-constrained scenarios, achieves 67.4% top accuracy on ImageNet with only 2.5 million parameters making it approximately 2.8x smaller than EfficientNetV2-B0. VisionAgri AI will offer MobileNetV3Small as a lightweight alternative to

EfficientNetV2-B0, providing users with a trade-off between accuracy and inference speed that is particularly relevant for deployment on low-resource servers.

2.4 Semantic Segmentation for Disease Localization

While classification-based approaches assign a single label to an entire image, they fail to capture the spatial distribution of disease symptoms within a leaf. Semantic segmentation addresses this limitation by producing pixel-level predictions, assigning each pixel to a semantic category (eg, background, healthy tissue, diseased region). This capability is essential for disease severity quantification, where severity is calculated as the ratio of diseased pixels to total leaf pixels.

The U-Net architecture, proposed by Ronneberger, Fischer and Brox (2015) for biomedical image segmentation, has become the dominant approach for agricultural segmentation tasks. U-Net features an encoder-decoder structure with skip connections that concatenate feature maps from the contracting path to the expansive path, enabling precise localization while retaining contextual information. The architecture was originally designed to achieve high segmentation accuracy with very small training datasets (as few as 30 annotated images), making it particularly suitable for agricultural domains where pixel-level annotation is labor-intensive and expensive.

Several studies have adapted U-Net for plant disease segmentation. Zainab and Mahum (2025) proposed a deep learning model capable of differentiating between healthy and diseased leaves with pixel-level precision, emphasizing the importance of segmentation for actionable disease management. Their work highlighted a common limitation of classification-only systems, a leaf may contain both healthy and diseased regions, and a whole-image classification provides no information about the extent or spatial pattern of infection. This observation directly motivates VisionAgri AI's dual-head architecture, which simultaneously classifies the disease type and segments the affected area.

Anami, Malvade and Palaiah (2020) applied deep learning to paddy crop stress recognition using field images, achieving classification accuracy of 92.89% across biotic and abiotic stresses using a VGG-16 architecture. While their work focused on classification rather than segmentation, the authors noted that spatial localization of stress symptoms would significantly enhance the practical utility of automated systems, corroborating the design rationale of VisionAgri AI's segmentation head.

VisionAgri AI extends the standard U-Net approach by employing dual-head architecture where a shared encoder feeds both a classification head (four-class SoftMax) and a segmentation head (three channel output, background, healthy leaf, diseased region). This design enables a single forward pass to produce both the disease diagnosis and its spatial localization, avoiding the computational overhead of running separate classification and segmentation models. Furthermore, the segmentation output is used to compute a disease severity percentage, transforming a qualitative diagnosis into a quantitative assessment that directly informs treatment decisions.

2.5 Data Augmentation Strategies

Shoaib et al. (2023) reported that the most commonly employed augmentation techniques in plant disease studies include geometric transformations (random rotation, horizontal and vertical flipping, scaling, and translation), photometric transformations (brightness adjustment, contrast modification, hue shifting, and Gaussian noise injection), and elastic deformations. The authors emphasized that augmentation must be applied consistently to both the input image and its corresponding segmentation mask in segmentation tasks to maintain label integrity.

VisionAgri AI applies a comprehensive augmentation pipeline to expand the original 381 image dataset to 3,324 training samples. Augmentation techniques include random rotation (30 degrees + or -), horizontal and vertical flipping, brightness and contrast jittering (20% increment or decrement), Gaussian noise injection, and random cropping with resizing. All geometric transformations are applied identically to both the leaf image and its corresponding three-channel segmentation mask, ensuring spatial alignment between input and ground truth.

2.6 Large Language Models for Agricultural Advisory

The emergence of large language models (LLMs) such as GPT-4 (OpenAI, 2023) and open-source alternatives has created new possibilities for generating domain-specific advisory content. In the agricultural context, LLMs can synthesize information from disease diagnosis results, environmental data, and general agronomic knowledge to produce treatment recommendations tailored to specific conditions.

While the use of LLMs in automated agricultural advisory is still nascent in academic literature, several studies have explored related concepts. The integration of AI-

generated recommendations with diagnostic outputs aligns with the broader trend towards decision support systems in agriculture, where the goal is not merely to identify a problem but to prescribe an appropriate course of action (Talaviya et al., 2020).

VisionAgri AI will employ the OpenRouter API to access LLM capabilities, constructing structured prompts that include the detected disease name, classification confidence, severity percentage, and where available the seven-day weather forecast. The LLM will be instructed to respond in a structured JSON format containing a disease description, treatment steps, preventive care tips, and a weather-informed advisory. To ensure reliability, the system will implement robust fallback mechanisms, if the LLM service is unavailable or returns malformed output, the system provides static, pre-defined treatment recommendations based on the identified disease. Users can also toggle LLM recommendations on or off via the user interface, addressing potential concerns about AI-generated advisory reliability.

2.7 Identified Research Gaps

The preceding review reveals several research gaps that VisionAgri AI systematically addresses.

1. **Classification Without Localization** – The majority of plant disease detection systems perform whole-image classification without spatially localizing disease symptoms or quantifying severity. VisionAgri AI will address this through its dual-head architecture that simultaneously classifies and segments, producing a disease severity percentage.
2. **Isolation from Environmental Context** – Existing systems produce diagnostic results in isolation from environmental conditions that critically influence disease management decisions. VisionAgri AI will integrate real-time weather forecasts to contextualize recommendations.
3. **Absence of Actionable Recommendations** – Most systems provide a disease label and confidence score without prescribing treatment. VisionAgri AI will bridge this gap through LLM-powered, weather-aware treatment recommendations and care tips.
4. **Input Validation for Robustness** – Few studies address the behavior of disease detection models when presented with out of distribution inputs (like images of plant types not in the training set). VisionAgri AI will implement a dedicated

gatekeeper binary classifier that rejects unsupported inputs before they reach the disease model, preventing misleading predictions.

5. **Deployment Gap** – The literature is dominated by model-centric studies that do not address the engineering challenges of deploying models as accessible applications. VisionAgri AI will deliver a fully deployed, containerized system accessible via a custom domain with SSL encryption.

CHAPTER 03 – Project Planning

Effective project planning is fundamental to the successful delivery of a software engineering project, particularly one that integrates multiple complex technologies including deep learning model training, API development, and cloud deployment. This chapter presents the planning activities undertaken for VisionAgri AI, including feasibility analysis, risk assessment, SWOT and PESTEL analyses, the selection and justification of the development methodology, and the project timeline.

3.1 Feasibility Analysis

A feasibility analysis was conducted to assess the viability of the project across four dimensions: technical, economic, operational, and schedule feasibility. This analysis informed key architectural and deployment decisions made during the project.

3.1.1 Technical Feasibility

The technical feasibility of VisionAgri AI was assessed by evaluating the availability of suitable tools, frameworks, and hardware resources. TensorFlow 2.15, a mature and extensively documented deep learning framework, provides all necessary functionality for model training, including pretrained EfficientNetV2 and MobileNetV3 backbones, custom layer support, and Keras model serialization. FastAPI, offers asynchronous request handling and automatic OpenAPI documentation generation, making it an ideal choice for the inference API.

Model training was conducted on Kaggle's free GPU infrastructure (NVIDIA Tesla P100), eliminating the need for expensive local hardware. For deployment, a DigitalOcean droplet (2GB RAM, 1 vCPU, Ubuntu 22.04) was selected based on memory profiling that confirmed all three models (EfficientNetV2-B0 at <100MB, MobileNetV3Small at <36MB, and the gatekeeper at <22MB) can be loaded concurrently within the 2GB memory constraint when TensorFlow memory

optimization environment variables are configured. Docker containerization ensures consistent behavior across development and production environments. The technical stack is therefore assessed as fully feasible.

3.1.2 Economic Feasibility

The project was designed with cost minimization as a constraint, consistent with the budget limitations of a student project. The following table summarizes the cost structure.

Resource	**Provider**	**Cost**
GPU training environment	Kaggle (free tier)	$0
Cloud server (2GB Droplet)	DigitalOcean (free trial)	$0/year
Visionagri.app domain	Name.com (free trial)	$0/year
SSL Certificate	DuckDNS + Lets encrypt	$0
Frontend Hosting	Github pages	$0
Weather API	OpenMeteo (free)	$0
LLM API	OpenRouter z-ai free	$0
Annotation tool	Label Studio	$0
Total estimated annual cost		**$0**

Table 1 Economic Feasibility table

The total operational cost of approximately $0 per year is economically very feasible and demonstrates that production-quality AI systems can be deployed at zero cost using open-source tools and free-tier cloud services. Should the system scale to serve a larger user base, DigitalOcean's pricing model allows vertical scaling (4GB and more RAM) without architectural changes.

3.1.3 Operational Feasibility

Operational feasibility considers whether the target users can effectively use the system. The requirements survey revealed that 73% of respondents rated their comfort with smartphone apps or web tools at 4 or 5 out of 5, indicating strong digital literacy among the target demographic. The web-based interface requires no installation; users simply navigate to visionagri.app on any device with a modern browser. The interface employs intuitive drag-and-drop image upload, clear visual feedback, and sample test images for

immediate hands-on evaluation. These design choices ensure operational feasibility even for users with limited technical expertise.

3.1.4 Schedule Feasibility

The project was planned to be completed within a seven-month timeline (September 2025 to March 2026), aligning with the academic semester schedule. The iterative development methodology adopted (see Section 3.5) allowed overlapping phases, with literature review and data collection proceeding concurrently, and model training and backend development overlapping where dependencies permitted. The schedule was assessed as tight but feasible, with contingency time allocated in the final month for documentation and testing refinements.

3.2 Rist Assessment

A systematic risk assessment was conducted to identify, evaluate, and plan mitigation strategies for potential threats to project success. Risks were evaluated on two dimensions, likelihood and impact, each scored on a 1-5 scale, with the product yielding a risk score used to priorities mitigation efforts.

See Figure 1

ID	Risk	Likelihood	Impact	Score	Mitigation Strategy
R1	Model overfitting on small dataset.	2	4	8	Two phase transfer learning, data augmentation, dropout regularization, early stopping
R2	Third-party API downtime (openrouter/open-meteo)	3	2	6	Static fallback recommendation when LLM is unavailable, weather section marked unavailable.
R3	Server out of memory crashes.	1	3	3	TF memory optimization env vars, single worker Uvicorn, memory profiling before deployment.
R4	Dataset bias (lab vs field)	4	1	4	Custom dataset with diverse sources, gatekeeper rejects unknown inputs, 75% confidence threshold.

R5	CORS/deployment config errors.	2	2	4	Wildcard CORS policy, nginx client_max_body_size=15M, Docekr health checks.
R6	Low accuracy on real-world field images.	3	3	9	Color statistics branch for robust features, confidence threshold blocks unreliable predictions.

Table 2 Risk assessment table

The highest-scoring risk (R6, low accuracy on field images, score = 9) will be mitigated through architectural decisions including the color statistics branch that extracts green-to-red ratios robust to lighting variation, and the 75% minimum confidence threshold that prevents display of uncertain predictions. Risk R1 (overfitting) was mitigated through comprehensive data augmentation that expanded the low count of images dataset to 3,324 training samples, combined with transfer learning from ImageNet-pretrained weights.

3.3 SWOT Analysis

A SWOT (Strengths, Weaknesses, Opportunities, Threats) analysis was conducted to evaluate the internal and external factors influencing the project's success and to inform strategic design decisions.

Strengths

- Dual head architecture (classification + segmentation) in single pass.
- Two model options for accuracy vs speed trade-off.
- Gatekeeper prevents out-of-distribution errors.
- Weather aware LLM recommendations.
- Fully containerized Docker deployment.

Weaknesses

- Limited to two crops and four conditions.
- Requirement of internet connection.
- Small original dataset (<400)
- Dependent on third party services (OpenRouter, Open-Meteo)
- English-only interface (for now)

Opportunities

- Expand to additional crops and diseases.
- Offline mode via TFLite on devices.
- Multi language support (Sinhala, Tamil) per survey demand.
- Partnership with Department of Agriculture.

Threats

- Model accuracy may degrade on unseen field conditions.
- API service outages.
- Completing mobile apps entering market.
- Low internet adaptation in remote rural areas.
- Rapid evolution of DL frameworks requiring maintenance.

The SWOT analysis reveals that VisionAgri AI's primary strengths lie in its integrated, multi-component architecture that addresses gaps identified in the literature review, particularly the dual-head design, gatekeeper validation, and weather-aware recommendations. The most significant weakness is the limited crop and disease scope, which constrains the system's immediate applicability. However, the modular architecture is designed to facilitate expansion, adding new crop support requires only training additional models and extending the gatekeeper's acceptance criteria, without architectural changes to the backend or frontend.

The opportunity for multi-language support is particularly noteworthy, as the requirements survey revealed that 38% of respondents considered Sinhala language support important, and a further 38% requested all three languages (English, Sinhala, and Tamil). This represents a clear direction for future development. The most significant threat, model accuracy degradation on unseen field conditions, is inherent to all supervised learning systems and is mitigated through the confidence threshold mechanism and the gatekeeper classifier.

3.4 PESTEL Analysis

A PESTEL analysis was conducted to examine the macro environment facrors that may influence the adaptation and sustainability of VisionAgri AI.

Political

- Government agriculture digitalization policies.
- Data privacy regulations (PDPA Sri Lanka)

Economics

- Crop losses cost farmers over $3.5 billion in South Asia.
- Free-tier APIs reduce operational costs.

Social

- 58% mobile internet penetration in SL.
- Growing tech literacy in younger farmers.

Technological

- Rapid DL framework evolution (TF/Pytorch)
- Cloud computing cost reduction trends.

Environmental

- Climate change alters disease patterns.
- Reduced pesticide uses via targeted treatment.

Legal

- No liability framework for AI crop advice.
- Open-source model licensing compliance.

The Political environment is favorable, with the Sri Lankan government actively promoting agricultural digitalization through initiatives such as the Digital Agriculture Strategy 2025-2030. However, the forthcoming Personal Data Protection Act (PDPA) introduces compliance requirements for systems that process location data, as VisionAgri AI does when capturing geolocation for weather integration. The current implementation mitigates this by making geolocation entirely optional, with clear user-facing messaging when location access is declined.

Economically, the project addresses a significant pain point, crop losses due to plant diseases cost South Asian agriculture an estimated $3.5 billion annually (FAO, 2021). Even marginal improvements in diagnostic speed and accuracy can translate to substantial economic benefits. The system's zero operational cost model (leveraging free-tier APIs and affordable/free cloud hosting) makes it economically sustainable.

Technologically, the rapid evolution of deep learning frameworks represents both an opportunity and a challenge. TensorFlow's Keras API provides stability for model serialization, but breaking changes across major versions necessitate ongoing

maintenance. The environmental factor of climate change is particularly relevant, as shifting weather patterns alter disease pressure dynamics, reinforcing the value of weather-integrated advisory systems.

3.5 Development Methodology

The development of VisionAgri AI followed an iterative, incremental methodology adapted from the Agile framework. This approach was selected over traditional plan-driven methodologies (such as Waterfall) for several reasons specific to the project's characteristics.

1. **Evolving Requirements –** The dual nature of the project, combining machine learning experimentation with software engineering, meant that technical requirements evolved as model training results became available. For example, the decision to add a gatekeeper binary classifier emerged from observing the disease model's behavior on out-of-distribution inputs during testing, a requirement that could not have been specified upfront.
2. **Rapid Feedback Loops –** Deep learning model development is inherently iterative, involving cycles of training, evaluation, hyperparameter tuning, and architectural modification. An iterative methodology accommodates these cycles naturally, whereas a Waterfall approach would require completing all design before beginning implementation.
3. **Risk Management Through Early Integration** – By developing and integrating components incrementally, beginning with the core inference pipeline and progressively adding weather integration, LLM recommendations, and the gatekeeper, risks were identified and resolved early rather than accumulating to the integration phase.
4. **Solo Developer Context** – As a single developer project, the overhead of formal Agile ceremonies (standups, sprint reviews) was unnecessary. The methodology was adapted to retain the iterative, feedback-driven principles while eliminating team-oriented processes

See Figure 2

The project was structured into six phases, each producing tangible deliverables that fed into subsequent phases. Importantly, phases were not strictly sequential, feedback loops (shown as dashed arrows in Figure 2) enabled revisiting earlier phases when new

insights emerged. For example, poor pepper classification accuracy in Phase 3 prompted a return to Phase 2 for additional data augmentation and the introduction of class-specific loss weighting, which resolved the confusion between, pepper healthy and bacterial spot classes.

3.7 Resource Allocation

The following table summarizes the key resource allocated to the project across hardware, software, and services.

Category	**Resource**	**Purpose**
Hardware	Personal laptop (8GB RAM, Intel i3)	Development, testing, documentation.
Hardware	Kaggle P100 GPU (free tier)	Model training (30hours/week limit)
Hardware	DigitalOcean Droplet (2GB RAM, 1 vCPU)	Production deployment
Software	Python 3.10, Tensorflow 2.15	Model development and inference.
Software	FatAPI, Uvicorn	Backend API server
Software	Docker, Docker compose	Containerizing and orchestration
Software	Label Studio	Image annotation (polygon + brush masks)
Software	Git, GitHub	Version control and frontend hosting
Service	Open-Meteo API	Weather forecast data
Service	OpenRouter API	LLM powered recommendations
Service	Name.com, DuckDNS	Domain management and DNS
Service	Let’s encrypt	SSL certificate provisioning

Table 3 Resource allocation table

CHAPTER 04 – Requirement Analysis

This chapter documents the requirements for elicitation, analysis, and specification activities undertaken for VisionAgri AI. Requirements were gathered primarily through a quantitative survey distributed to potential end users and supplemented by domain research and iterative feedback from the project supervisor. The chapter presents the survey methodology and findings, derives functional and non-functional requirements from the collected data, and specifies use cases that define the system's behavioral contract with its users.

4.1 Requirement Elicitation Methodology

Requirements were elicited through a structured online questionnaire distributed via Google Forms to individuals with an interest in agriculture or plant science. The survey comprised 20 questions organized into four sections, respondent demographics (Questions 1-4), current disease management practices (Questions 5-8), feature importance ratings (Questions 9-16), and open-ended feedback (Questions 17-20). The survey employed a mix of question types including single-select, multi-select, Likert scale (1-5), and free-text responses.

A total of 37 valid responses were collected over a two-week period in March 2026. While this sample size is modest, it provides sufficient directional insight for a project of this scope, and the respondent composition, spanning students (78%), farmers or growers (11%), and other roles (11%), offers perspectives from both potential end users and individuals with agricultural domain knowledge. The full survey instrument and raw response data are included in Appendix 2.

4.2 Survey Findings and Analysis

4.2.1 Respondent Profile

Of the 52 respondents, 41 (79%) identified as students, 5 (10%) as farmers or growers, and 6 (12%) as other roles. Regarding digital literacy, 34 respondents (65%) rated their comfort with smartphone apps or web tools at the maximum score of 5 out of 5, with a further 7 (13%) rating themselves at 4, indicating that the majority of the target audience possesses sufficient digital skills to engage with a web-based diagnostic tool. Only 6 respondents (12%) rated their comfort at 2 or below, suggesting that interface simplicity

remains important but need not dominate design decisions at the expense of functionality.

See figure 3,4,5,6

4.2.2 Current Disease Management Practices

Respondents were asked how they currently identify plant diseases, with multi-select options permitted. The results reveal a heavy reliance on subjective methods, visual inspection by the naked eye was selected by most respondents, while a substantial proportion indicated that they lack any reliable method for disease identification. Only a small number reported using mobile apps or consulting experts, and laboratory testing was rarely cited. These findings confirm the diagnostic accessibility gap identified in the Problem Statement.

When asked about crop losses due to late or incorrect diagnosis, the results were striking, 34 respondents (65%) reported experiencing losses occasionally, 10 (19%) reported frequent losses, and only 1 respondent reported never experiencing losses. In total, 44 out of 52 respondents (85%) had experienced crop losses attributable to diagnostic failures, providing strong empirical justification for the development of an automated diagnostic tool.

The biggest challenges in detecting plant diseases were identified through a multi-select question. The most cited challenges were difficulty distinguishing similar diseases, lack of expert availability nearby, and early symptoms being hard to spot. These three challenges frequently co-occurred in responses, suggesting that respondents experience a compound problem: diseases that look similar, no expert to consult, and symptoms that are subtle in their early stages when intervention would be most effective. VisionAgri AI addresses all three challenges through automated classification that distinguishes visually similar diseases, 24/7 availability without requiring expert access, and segmentation-based severity analysis that can highlight subtle disease regions.

See figure 7,8,9,10

4.2.3 Feature Importance Ratings

Respondents rated the importance of four core features on a 1-5 Likert scale. The results demonstrate strong demand across all proposed features,

Disease identification from a leaf photo (Question 9) – 33 respondents (63%) rated this as critically important (5/5), with a further 7 (13%) rating it at 4/5. Only 3 respondents rated it at 2 or below. The mean score of 4.3 out of 5 confirms that image-based disease classification is the most valued capability and must be the system's core function.

Highlighting the diseased area on the leaf (Question 10) – 32 respondents (62%) rated this at 5/5 and 11 (21%) at 4/5, yielding a mean of 4.4. This strong support validates the inclusion of a semantic segmentation head that produces a visual disease overlay, rather than providing only a text-based classification result.

Disease severity percentage (Question 11) – 28 respondents (54%) rated this at 4/5 and 10 (19%) at 5/5, with a mean of 3.9. While slightly lower than the preceding features, the strong majority rating of 4 or above confirms that quantitative severity assessment adds meaningful value beyond binary healthy/diseased classification.

Treatment recommendations (Question 12) – 33 respondents (63%) rated this at 5/5 and 9 (17%) at 4/5, yielding a mean of 4.4, matching disease identification as the joint-highest-rated feature. This result strongly supports the integration of LLM-powered treatment advisory, transforming VisionAgri AI from a diagnostic tool into an actionable decision support system.

See figure 11,12,13,14

4.2.4 Display Preferences and Platform

When asked what the system should display after analysis (multi-select), highlighted disease area on image was the most selected option (34 selections, 65%), followed closely by disease name (33, 63%) and treatment recommendations (32, 62%). Confidence score and severity percentage received moderate support (7 selections each, 13%), while scan history received 6 selections. These results informed the design of the results display panel, which prominently features the disease name, segmentation overlay, and treatment recommendations, with confidence score and severity percentage presented as supporting information.

Platform preference favored web browsing – 24 respondents (46%) preferred a web browser, 13 (25%) preferred a mobile app, and 14 (27%) wanted both. The decision to develop a responsive web application accessible through any browser satisfies both the web-only and mobile preferences, as the responsive design renders effectively on smartphone screens without requiring a native app installation.

Offline functionality importance was bimodal – 23 respondents (44%) rated it at 1/5 (not important), while 12 (23%) rated it at 5/5 (critically important). This polarization reflects the diverse connectivity conditions across the target demographic. While offline support will not be implemented in the current version due to the server-side inference architecture, it is identified as a priority for future work using TensorFlow Lite on-device inference.

See figure 15,16,17,18,19

4.2.5 Accuracy Expectations and Concerns

The minimum acceptable accuracy for disease classification was surveyed with four options. Most respondents (32, 62%) selected above 80%, followed by above 70% (8, 15%), above 90% (7, 13%), and above 95% (5, 10%). This distribution informed the selection of a 75% minimum confidence threshold in the system, set conservatively below the majority's stated minimum to ensure results are displayed when the model is reasonably confident, while blocking clearly unreliable predictions.

Regarding concerns about an AI-based system, accuracy reliability was the dominant concern (31 selections, 60%), followed by difficulty of use for non-technical users (11, 21%), lack of internet in rural areas (10, 19%), and privacy of farm data (7, 13%). The accuracy concern reinforces the importance of the confidence threshold mechanism and the gatekeeper classifier, both of which prevent the system from presenting unreliable results. The usability concern informed the interface design, which prioritizes simplicity with drag-and-drop upload, sample test images for immediate evaluation, and clear visual feedback at every stage.

See figure 20

4.3 Functional Requirements

Based on the survey analysis, literature review, and domain research, the following functional requirements were specified for VisionAgri AI. Each requirement is assigned a priority (Must Have, Should Have, or Could Have) following the MoSCoW prioritization framework.

ID	Requirement	Priority	Survey Basis
FR-01	The system shall accept a leaf image via file upload or drag and drop.	Must Have	Q9 – 77% rated 4-5/5

FR-02	The system shall classify the uploaded image into one of four disease categories: Pepper Healthy, Pepper Bacterial Spot, Tomato Healthy, or Tomato YLCV.	Must Have	Q9 – 63% rated 5/5
FR-03	The system shall generate a segmentation overlay highlighting the diseased area on the leaf.	Must Have	Q10 – 83% rated 4-5/5
FR-04	The system shall calculate and display a disease severity percentage derived from segmentation output.	Hust Have	Q11 – 73% rated 4-5/5
FR-05	The system shall provide treatment recommendations and care tips for the detected condition.	Must Have	Q12 – 81% rated 4-5/5
FR-06	The system shall reject images of unsupported plant types with an informative error message.	Must Have	Q18 – Accuracy concern
FR-07	The system shall offer a choice between two models, EfficientNetV2-B0 (higher accuracy) and MobileNetV3Small (lighter weight/faster).	Should have	Model comparison, design decision.
FR-08	The system shall optionally capture the user's geolocation to retrieve weather forecast data for contextual recommendations.	Should have	Q12 + domain research
FR-09	The system shall allow the user to toggle AI-powered recommendations on or off.	Should have	Q18 – Reliability concern
FR-10	The system shall display confidence scores for all four classes as a distribution.	Should have	Q13 – 13% selected
FR-11	The system shall block display of results when classification confidence falls below 75%.	Must have	Q14 – 62% expect >80%
FR-12	The system shall provide sample test images for immediate system evaluation.	Could have	Usability enhancement
FR-13	The system shall display a downloadable disease overlay image.	Could have	User convenience
FR-14	The system shall validate uploaded files for type (JPG/PNG only) and size (maximum 10MB).	Must have	System robustness

Table 4 Functional requirements of VisionAgri AI

4.4 Non-Functional Requirements

Non-functional requirements specify quality attributes and constraints that govern how the system performs its functions, rather than what functions it performs.

ID	Requirement	Category	Target
NFR-01	The system shall return analysis results within 60 seconds of submission under normal operating conditions.	Performance	<60 seconds
NFR-02	The system shall achieve a minimum classification accuracy of 90% on the validation dataset.	Accuracy	>90% validation accuracy
NFR-03	The web interface shall be responsive and functional on devices with screen widths from 320px to 1920px.	Usability	Mobile + Desktop
NFR-04	The system shall be containerized using Docker for reproducible deployment across environments.	Portability	Single Dockerfile
NFR-05	The backend shall handle concurrent requests without crashing on a 2GB RAM server.	Reliability	Stable in 2GB
NFR-06	All API communication shall occur over HTTPS with a valid SSL certificate.	Security	TLS encryption
NFR-07	The system shall gracefully handle failures in external services (Open-Meteo, OpenRouter) with appropriate fallbacks.	Fault tolerance	Static fallback
NFR-08	The system shall not store uploaded images or user location data beyond the request's lifecycle.	Privacy	No data retention
NFR-09	The web interface shall be accessible without requiring software installation.	Accessibility	Browser-based
NFR-10	The system shall provide meaningful error messages for all failure modes.	Usability	Reduce user facing errors

Table 5 Non-functional requirements of VisionAgri AI

4.5 Use Case Specifications

The following use cases capture the primary interactions between users and the VisionAgri AI system. The primary actor in all cases is the end user (farmer, student, or agricultural stakeholder) accessing the system through a web browser.

4.5.1 Use Case 1 – Analyze Leaf Image

Use Case ID – UC01

Actor – End User

Precondition – User has a leaf image (jpg or png, <= 10MB) and internet access.

Main Flow

1. User navigates to visionagri.app and is presented with the upload interface
2. User selects or drags a leaf image into the upload area.
3. System displays a preview of the uploaded image.
4. User selects the preferred model (EfficientNetV2-B0 or MobileNetV3Small).
5. User optionally grants geolocation permission for weather-based recommendations.
6. User optionally enables or disables the AI recommendations toggle.
7. User clicks “Analyze” button.
8. System sends the image, model selection, geolocation (if available), and AI preference to the backend API.
9. Backend validates the image, runs the gatekeeper classifier, performs disease inference, retrieves weather data (if location provided), and generates recommendations (if enabled).
10. System displays results – disease name, confidence, severity, segmentation overlay, treatment recommendations, and care tips.

Alternative Flows

1. User navigates to visionagri.app and is presented with the upload interface.
2. User selects or drags a leaf image into the upload area.
3. Invalid file type – System displays an error message indicating only JPG and PNG files are accepted. Flow terminates.
4. File exceeds 10MB – System displays a file size error. Flow terminates.
5. Gatekeeper rejects image – System displays a prominent rejection banner stating “Unsupported Plant Type” with guidance to upload pepper or tomato leaves. Flow terminates.
6. Classification confidence below 75% - System blocks result display and informs the user that the image could not be analyzed with sufficient confidence.

7. LLM service unavailable – System provides static fallback recommendations without weather context.
8. User declines geolocation – System proceeds without weather data and informs the user that weather-tailored recommendations will be unavailable.

Postcondition – User has received a disease diagnosis with actionable treatment information or has been informed why analysis could not be completed.

4.5.2 Use Case 2 – Test with Sample Images

Use Case ID – UC02

Actor – End User

Precondition – User has navigated to visionagri.app

Main Flow

1. User scrolls to the "Try a Sample Image" section.
2. User clicks one of the four sample image cards (Pepper Healthy, Pepper Bacterial Spot, Tomato Healthy, or Tomato Leaf Curl).
3. System loads the selected sample image into the upload area and displays its preview.
4. User proceeds with analysis as per UC-01 from step 4.

Postcondition – User has evaluated the system's capabilities using a known-good sample image without needing to source their own leaf photograph.

4.6 Requirements Traceability

To ensure that all specified requirements are addressed in the system design and implementation, a traceability mapping was established between survey questions, functional requirements, and system components. Each functional requirement is traced back to one or more survey findings, ensuring that the system is grounded in empirical user needs rather than developer assumptions. For example, FR-05 (treatment recommendations) traces to Question 12, where 81% of respondents rated this feature at 4 or 5 out of 5, and to Question 13, where highlighted disease area on image received the highest number of selections (34 out of 52) for post-analysis display. Similarly, FR-06 (gatekeeper rejection) traces to the accuracy concern identified in Question 18, where 60% of respondents cited accuracy reliability as their primary concern,

motivating a system design that actively prevents unreliable predictions through input validation and confidence thresholds.

Forward traceability from requirements to implementation is validated in Chapter 7 (Testing and Evaluation), where each functional requirement is mapped to specific test cases that verify its correct implementation.

CHAPTER 05 – System Design

This chapter presents the architectural and component-level design of the VisionAgri AI system. The design translates the functional and non-functional requirements specified in Chapter 4 into a concrete technical blueprint, covering the overall system architecture, the inference pipeline, API specification, data flow, frontend design, and deployment topology. Design decisions are justified in terms of their contribution to system quality attributes including performance, reliability, maintainability, and usability.

5.1 System Architecture Overview

VisionAgri AI follows a client-server architecture with a clear separation between the presentation layer (frontend), the application layer (backend API), the model layer (deep learning inference), and external service integrations. This separation of concerns enables independent development, testing, and deployment of each layer, and allows the computationally intensive model inference to execute on a dedicated server rather than on the user's device.

See figure 21

As illustrated in Figure 21, architecture comprises three principal layers. The Client Layer, hosted on GitHub Pages at visionagri.app, serves static HTML, CSS, and JavaScript files that render the user interface and handle client-side logic including image validation, geolocation capture, and result presentation. The Backend Layer, running inside a Docker container on a DigitalOcean droplet at api.visionagri.app, hosts the FastAPI application server, the TensorFlow inference engine, and three Keras models (gatekeeper, EfficientNetV2-B0, and MobileNetV3Small). The External Services layer comprises the Open-Meteo weather API and the OpenRouter LLM API, both accessed asynchronously over HTTPS.

This architecture was selected over alternative approaches, such as on-device inference using TensorFlow Lite or a monolithic server rendering both frontend and backend, for several reasons. Server-side inference enables the use of full-precision Keras models without accuracy-degrading quantization. Hosting the front end on GitHub Pages provides global CDN distribution, zero hosting cost, and automatic HTTPS, while the backend can be independently scaled by upgrading the DigitalOcean droplet without modifying the front end.

5.2 Inference Pipeline Design

The inference pipeline is the core computational component of VisionAgri AI, responsible for transforming a raw user-uploaded image into a structured diagnosis. The pipeline executes as a sequence of eight stages, each implemented as a discrete function within the inference.py module.

See figure 22

1. **Stage 1 (Upload)** – Receives the image as a multipart form upload via the POST /api/analyze endpoint.
2. **Stage 2 (Validation)** – Verifies that the file is a valid image type (JPEG or PNG) and does not exceed the 10MB size limit. Throws a HTTP 413 or 415 errors respectively if any of them fails.
3. **Stage 3 (Gatekeeper)** – passes the image through the MobileNetV2 binary classifier. The image is resized to 224x224 pixels and fed as raw pixel values (0-255 range, no normalization) into the model, which produces a sigmoid output. If the output exceeds the threshold of 0.5, the image is classified as rejected (unsupported plant type) and the pipeline terminates with an HTTP 422 response containing a descriptive error message.
4. **Stage 4 (Preprocessing)** – Resizes the accepted image to 256x256 pixels using Lanczos resampling and normalizes pixel values to the [0.0, 1.0] range by dividing by 255.0. The result is a float32 NumPy array of shape (1, 256, 256, 3).
5. **Stage 5 (Model Inference)** – feeds this array into the selected disease detection model. Both the EfficientNetV2-B0 and MobileNetV3Small models produce two outputs from a single forward pass: a segmentation map of shape (256, 256, 3) with channels representing background, healthy leaf tissue, and diseased

regions; and a classification vector of shape (4,) containing SoftMax probabilities for each disease class.

6. **Stage 6 (Severity Calculation)** – Derives the disease severity percentage from the segmentation output. Pixels with a disease channel probability exceeding 0.5 are counted as diseased; pixels with a healthy channel probability exceeding 0.5 are counted as healthy. The severity is computed as (disease_pixels / (disease_pixels + healthy_pixels)) x 100, providing a quantitative measure of infection extent.
7. **Stage 7 (Overlay Generation)** – creates a visual disease heatmap by alpha-blending a red tint onto the original image proportional to the disease probability at each pixel. The alpha value is calculated as clip (disease_probability x 1.5, 0, 0.75), ensuring that high-probability disease regions appear prominently red while healthy areas remain unaltered. The overlay is encoded as a base64 PNG string.
8. **Stage 8 (Response)** – Assembles all outputs into a JSON response object.

5.3 Model Architecture Design

5.3.1 Dual-Head Architecture

Both disease detection models share a common dual-head architecture comprising four components: a pretrained encoder backbone, a color statistics branch, a classification head, and a segmentation head. This architecture enables a single forward pass to produce both a disease label and a pixel-level disease mask, avoiding the computational overhead and architectural complexity of maintaining separate classification and segmentation models.

The encoder backbone (EfficientNetV2-B0 or MobileNetV3Small) extracts hierarchical feature representations from the input image. These features are concatenated with outputs from the Color Statistics Branch, a custom component that computes channel-wise statistical features including mean intensity, variance (or standard deviation for MobileNetV3), and the green-to-red channel ratio. The green-to-red ratio is biologically motivated, chlorophyll in healthy leaves produces high green reflectance, while disease-induced chlorosis reduces green and increases red and yellow tones. This feature proved critical for disambiguating pepper healthy from pepper

bacterial spot, where early-stage infections produce subtle color shifts that general-purpose CNN features alone struggle to capture.

The classification head applies global average pooling to the concatenated features, followed by dropout regularization and a dense layer with SoftMax activation producing probabilities across the four target classes. The segmentation head applies transposed convolutional layers (decoder) with skipping connections from the encoder, progressively up sampling the feature maps back to the original 256x256 resolution and producing a three-channel output via SoftMax activation.

5.3.2 GateKeeper Design

The gatekeeper binary classifier uses a MobileNetV2 backbone pretrained on ImageNet, followed by global average pooling and two dense layers (64 units with ReLU, then 1 unit with sigmoid). The model accepts 224x224-pixel RGB images as raw pixel values (the model includes an internal Rescaling layer). It outputs a single sigmoid probability: values below 0.5 indicate an accepted input (pepper or tomato leaf), while values above 0.5 indicate rejection. The model was trained on 20,638 images (7,274 accepted from five pepper and tomato categories, 13,364 rejected from eleven other PlantVillage categories), with class weighting to address the 1:1.84 imbalance.

5.4 API Specification

The backend exposes a single RESTful endpoint that encapsulates the entire analysis pipeline. The API is designed following REST conventions with multipart form data for file upload, consistent with browser-native Form Data submission.

5.4.1 Endpoint – POST /api/analyze

Parameter	Type	Required	Default	Description
image	File (UploadFile)	Yes	-	Leaf image file (jpg/png, max 10MB)
model	String (Form)	No	efficientnet	Model selection, efficientnet or mobilenet
latitude	Float (Form)	No	None	GPS latitude for weather forecast retrieval

longitude	Float (Form)	No	None	GPS longitude for weather forecast retrival
ai_recommendations	String (Form)	No	true	Enable/disable LLM driven recommendations (true / false)

Table 6 API specs

5.4.2 Response Structure

The API returns a JSON object with four top-level keys. The classification object contains the predicted_class (internal name), friendly_name (display name), confidence (percentage), and all_scores (a dictionary mapping each class friendly name to its softmax probability as a percentage). The segmentation object contains severity_percent (float) and overlay_image (base64-encoded PNG string). The weather object contains an availability flag, location coordinates, timezone, and a daily array of seven-day forecast entries, each with date, temp_max, temp_min, humidity_avg, and rain_total_mm. The recommendations object contains a description string, treatment array, care_tips array, and weather_advisory string (or null if no weather data was provided).

5.4.3 Error Responses

HTTP Code	**Condition**	**Response Body**
400	Invalid file type, file too large or invalid model name.	{“Detail”: “Descriptive error message”}
422	GateKeeper rejects the input image as unsupported plant type.	{“detail”: “Image rejected: not a supported plant type…”}
500	Internal server error during inference or API call failure	{“detail”: “Inference fail: error details”}

Table 7 Error Responses

5.5 External Service Integration Design

5.5.1 Weather Data Integration

Weather data is retrieved from the Open-Meteo API, a free, open-source weather forecast service that requires no authentication. The integration is designed to be non-blocking and failure-tolerant: the weather module uses an asynchronous HTTP client

(httpx.AsyncClient) with a 10-second timeout, and any failure, network error, API outage, or malformed response, results in the weather section being marked as unavailable rather than causing the entire analysis request to fail.

The request queries hourly temperature at 2 meters, relative humidity at 2 meters, and rainfall for the user's geolocation using the best_match weather model with automatic time zone detection. The response parser groups hourly data points by date and aggregates them into daily summaries containing maximum temperature, minimum temperature, average humidity, and total rainfall. The first seven days of aggregated data are included in the API response and, when AI recommendations are enabled, are passed to the LLM prompt to generate weather-contextualized treatment advice.

5.5.2 LLM Recommendation Integration

Treatment recommendations are generated by a large language model accessed through the OpenRouter API, which provides a unified interface to multiple LLM providers. The integration uses the GLM-4.5-Air model (free tier) by default, configurable via the OPENROUTER_MODEL environment variable. The system prompt instructs the LLM to respond exclusively in valid JSON format as a plant pathology expert, while the user prompt includes the detection results (disease name, confidence, severity) and, if available, the seven-day weather forecast.

The temperature parameter is set to 0.3 to produce deterministic, consistent outputs, and the maximum token limit is 5,000. The response parser implements three-tier robustness: direct JSON parsing, markdown code fence stripping with re-parsing, and regex-based field extraction as a last resort. If the LLM service is entirely unavailable, the system falls back to static, pre-defined recommendations that vary based on whether the detected condition is healthy or diseased. This failover mechanism ensures that users always receive some treatment guidance, satisfying FR-05 regardless of external service availability.

5.6 Frontend Design

The frontend is implemented as a single-page application using HTML5, Tailwind CSS (via CDN), and vanilla JavaScript without framework dependencies. This technology choice minimizes bundle size, eliminates build tooling complexity, and ensures compatibility across all modern browsers. The interface is structured into distinct

functional zones arranged in a vertical flow that guides the user through the analysis process.

The Upload Zone provides both a drag-and-drop area and a traditional file picker, with client-side validation for file type (JPG, PNG) and size (maximum 10MB). Upon image selection, a real-time preview is displayed. The Model Selection Zone presents two model options as visually distinct radio-button cards, with EfficientNetV2-B0 labelled as recommended. The Control Zone contains the AI Recommendations toggle switch and the Analyze button, which triggers geolocation capture (with graceful fallback if declined) and initiates the API request.

The Results Zone is initially hidden and appears upon successful analysis, displaying: the disease name with a confidence badge; a severity indicator color-coded by severity level; a four-bar score distribution chart showing classification probabilities for all classes; the segmentation overlay image with a download link; treatment recommendations as a bulleted list with check-circle icons; care tips with rotating contextual icons; and a weather advisory section (if weather data was available). The interface enforces a 75% minimum confidence threshold, if the highest classification probability falls below this value, results are not displayed and the user is informed that the image could not be analyzed with sufficient confidence.

Error handling is implemented through two distinct mechanisms. Transient errors (network failures, server errors) trigger an auto-dismissing toast notification via the showInlineError() function. Gatekeeper rejections trigger a persistent, prominent red banner via the showRejection() function, which includes a descriptive heading ("Unsupported Plant Type"), an explanation message, and a manual dismiss button. The Sample Images section provides four pre-loaded test images (one for each supported condition), enabling users to evaluate the system immediately without needing their own leaf photographs.

5.7 Deployment Architecture

See figure 23

The deployment architecture, illustrated in Figure 23, separates the frontend and backend into independently hosted services. The front end is served as static files from GitHub Pages with the custom domain visionagri.app, benefiting from GitHub's global CDN and automatic HTTPS provisioning. The backend runs as a Docker container on

a DigitalOcean droplet (2GB RAM, 1 vCPU, Ubuntu 22.04) behind a Nginx reverse proxy that handles SSL termination, request buffering, and the CLIENT_MAX_BODY_SIZE directive (set to 15MB to accommodate high-resolution leaf photographs and block unwanted images).

The Docker container is built from a Python 3.10-slim base image with system dependencies (libgl1, libglib2.0-0) required by TensorFlow's image processing. The container runs a single Uvicorn worker process to prevent memory contention from multiple model copies, with a 120-second keep-alive timeout to accommodate slow inference on the constrained hardware. Environment variables (TF_CPP_MIN_LOG_LEVEL=2, TF_ENABLE_ONEDNN_OPTS=0, MALLOC_TRIM_THRESHOLD_=65536) are configured to minimize TensorFlow's memory footprint and suppress verbose logging. Docker Compose orchestrates the service with volume mounts for model files and the application code, a health check that polls the /docs endpoint every 30 seconds, and an automatic restart policy.

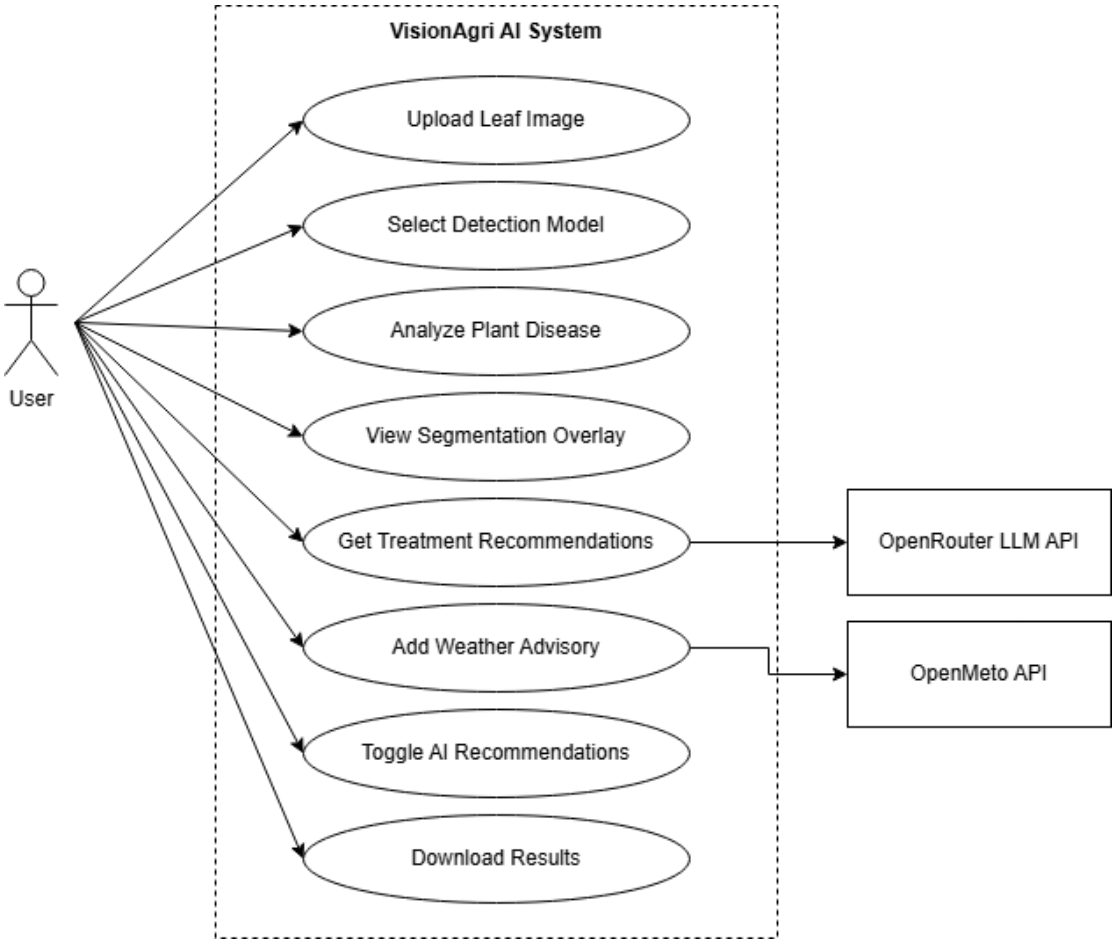
The domain api.visionagri.app is resolved via name.com dynamic DNS, pointing to the droplet's public IP address. SSL certificates are provisioned by Let's Encrypt through Certbot, providing free, automatically renewable HTTPS encryption. This zero-cost domain and SSL configuration aligns with the economic feasibility constraints identified in feasibility analysis.

5.8 Security Design Considerations

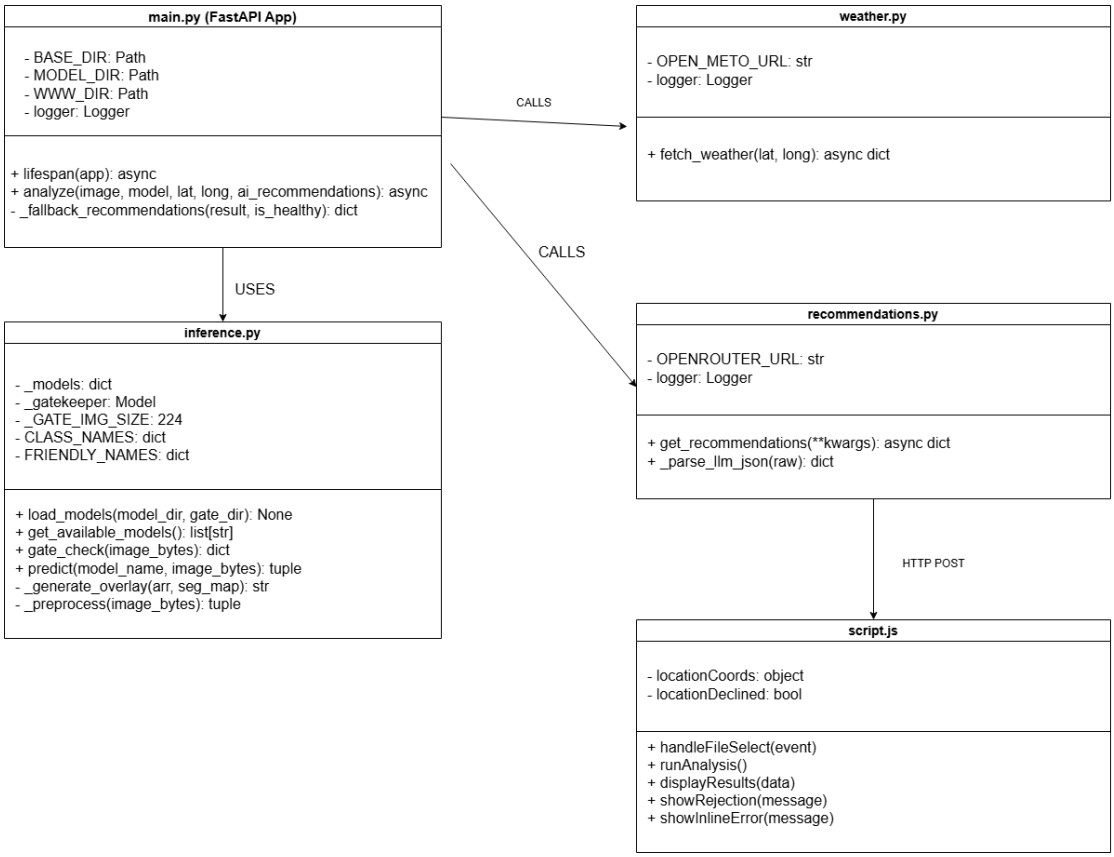
Several security measures are embedded in system design. All communication between the frontend and backend occurs over HTTPS with TLS encryption, preventing interception of uploaded images or analysis results. The API implements a permissive CORS policy (allowing all origins) because the frontend and backend are hosted on separate domains; however, this is mitigated by the stateless, read-only nature of the API, no user accounts exist, no data persisted, and no authentication tokens are issued. Uploaded images are processed entirely in memory and discarded after the response is sent, satisfying NFR-08 (no data retention). The OpenRouter API key is stored as an environment variable in the .env file, which is excluded from version control via. gitignore and injected into the Docker container at runtime. Input validation at both the frontend (file type and size checks in JavaScript) and backend (content-type verification

and byte-length enforcement in FastAPI) provides a defense against malformed or malicious uploads.

5.9 Use Case Diagram



5.10 Class Diagram



CHAPTER 06 – Implementation

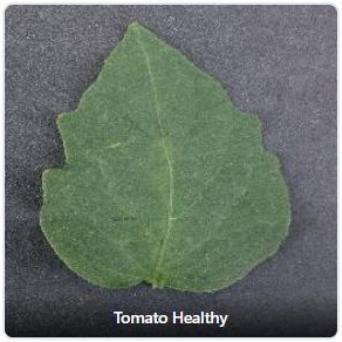
VisionAgri AI

Your AI-Powered Plant Disease Detector

Upload images of plant leaves to detect diseases early and get expert treatment recommendations.

TRY A SAMPLE IMAGE

Click any image below to load it for analysis



SELECT AI MODEL

- **EfficientNetV2-B0**
  Higher accuracy - Recommended for best results
- **MobileNetV3Small**
  Faster inference - Lighter model (Only Recommended for Testing)

Upload Leaf Image

Drag & drop your image here or click to browse

Supports JPG, PNG, JPEG (Max 10MB)

Select File

This chapter documents the technical implementation of VisionAgri AI, translating the system design presented in Chapter 5 into working software. The chapter is organized to follow the development chronology, dataset preparation and annotation, model training across all three neural networks, backend API development, frontend implementation, and containerized cloud deployment. Each section discusses the technologies used, key implementation decisions, challenges encountered, and the solutions adopted.

6.1 Technology Stack

The technology stack was selected to balance development productivity, deployment efficiency, and long-term maintainability. The following table summarizes the core technologies and their roles within the system.

Layer	**Technology**	**Version**	**Role**
Deep Learning	TensorFlow / Keras	2.15	Model training, inference, and serialization
Deep Learning	NumPy	1.24+	Array operations, image manipulation
Deep Learning	Pillow (PIL)	10.0+	Image loading, resizing, format conversion
Backend	FastAPI	0.104+	Asynchronous REST API framework
Backend	Uvicorn	0.24+	ASGI server with HTTP/1.1 support
Backend	Httpx	0.25+	Asynchronous HTTP client for external APIs
Backend	Python-dotenv	1.0+	Environment variable management
Frontend	HTML5/CSS3	-	Semantic markup responsive layout
Frontend	TailwindCSS	3 (CDN)	Utility first CSS framework
Frontend	Vanilla JS	ES2020+	Client-side logic, Fetch API, DOM manipulation
Frontend	Feather Icons	4 (CDN)	Lightweight SVG icon library
Annotation	Label Studio	1	Polygon and brush mark annotation tool
DevOps	Docker	24	Application containerization
DevOps	Docker Compose	2	Container orchestration and service definition

DevOps	Nginx	1	Reverse proxy, SSL termination
Hosting	GitHub Pages	-	Static frontend hosting with custom domain
Hosting	DigitalOcean	-	Cloud server (2GB droplet)
Training	Kaggle Notebooks	-	Free GPU environment (NVIDIA Tesla P100)

Table 8 Technology stack

6.2 Dataset Preparation and Annotation

6.2.1 Data Collection

The training dataset was assembled from two sources. The primary source was the PlantVillage dataset (Mohanty, Hughes and Salathe, 2016), an open-access repository of 54,306 labelled leaf images. From this repository, images were extracted for the four target classes: Pepper Healthy (90 images), Pepper Bacterial Spot (87 images), Tomato Healthy (108 images), and Tomato Yellow Leaf Curl Virus (96 images), yielding 381 original images. All images were standardized to 256×256-pixel resolution in RGB color space.

6.2.2 Annotation Pipeline

Pixel-level annotations were created using Label Studio, an open-source data labelling platform. Two annotation strategies were employed depending on the disease morphology. For pepper bacterial spots, which present discrete, well-defined lesions, polygon annotations were drawn around individual spots, producing vector-based masks that were programmatically rasterized into binary pixel masks. For tomato yellow leaf curl virus, which presents as diffuse yellowing and curling across irregular regions, brush-style annotations were used, producing PNG mask files directly. All 381 images received leaf boundary polygon annotations delineating the leaf from the background.

The annotation export from Label Studio comprised a JSON manifest file containing polygon coordinates and task-to-image mappings, alongside 96 brush mask PNG files for the tomato YLCV annotations. A custom Python preprocessing pipeline was developed in a Kaggle notebook to parse the JSON annotations, convert polygon vertices to binary masks using the PIL ImageDraw module, load and align brush mask PNGs by task ID, and generate combined three-channel segmentation masks

(background, healthy leaf, diseased region) for each image. Disease severity was computed as the ratio of disease pixels to total leaf pixels, yielding values ranging from 0.66% to 93.72% (mean – 20.43%, median – 15.17%) across the 183 diseased images.

6.2.3 Data Augmentation

To address the limited dataset size, a comprehensive augmentation pipeline was implemented to expand the 381 original images to 3,324 training samples (approximately 831 per class after balancing). The augmentation pipeline applied the following transformations randomly to each image and its corresponding segmentation mask: random rotation within 30 degrees; horizontal and vertical flipping with 50% probability; brightness and contrast jittering within 20%; Gaussian noise injection with standard deviation of 0.02; and random crop with resize maintaining at least 80% of the original area. Geometric transformations (rotation, flipping, cropping) were applied identically to both the image and mask arrays to preserve spatial alignment, while photometric transformations (brightness, contrast, noise) were applied only to the image. The augmented dataset was split into training (70%), validation (15%), and test (15%) subsets using stratified random sampling to maintain class balance across all splits.

6.3 Model Training Implementation

6.3.1 EfficientNetV2-B0 Training

The primary disease detection model was implemented and trained in a Kaggle notebook using the TensorFlow 2.15 Keras API. The model architecture comprises an EfficientNetV2-B0 encoder backbone pretrained on ImageNet, augmented with a Custom Statistics Branch. The Color Statistics Branch extracts three features from the input image: per-channel mean intensity (computed via tf.reduce_mean across spatial dimensions), per-channel variance (computed via a custom Lambda layer using tf.math.reduce_variance), and the green-to-red channel ratio (green channel mean divided by red channel mean plus an epsilon of 1e-7 for numerical stability). These six scalar features are concatenated with the globally average-pooled encoder features to form the classification input vector.

The classification head consists of a dropout layer (rate 0.3) followed by a dense layer with four softmax outputs. The segmentation head implements a decoder with transposed convolutions and skip connections from encoder intermediate layers,

progressively up sampling from the encoder's bottleneck resolution back to 256x256 pixels with a three-channel softmax output (background, healthy, disease).

Training employed a two-phase strategy. In Phase 1 (frozen backbone), the EfficientNetV2-B0 encoder weights were frozen and only the newly added layers (color statistics branch, classification head, segmentation decoder) were trained. The Adam optimizer was used with a learning rate of 1e-3 and a ReduceLROnPlateau scheduler (factor 0.5, patience 3, minimum 1e-6). The classification loss was focal loss with gamma = 2 and a pepper class weight of 2.5 to address the historically difficult pepper healthy versus bacterial spot discrimination. The segmentation loss combined categorical cross-entropy, Dice loss, and explicit false-positive and false-negative penalty terms with equal weighting. The total loss was a weighted sum of classification loss (weight 1.0) and segmentation loss (weight 1.0). Phase 1 ran for 24 epochs with early stopping (patience 7) monitoring validation classification accuracy.

In Phase 2 (fine-tuning), all layers were unfrozen and training continued with a reduced learning rate of 1e-5 for 12 additional epochs. This phase allowed the pretrained encoder features to adapt to the specific visual characteristics of pepper and tomato disease imagery while the low learning rate prevented catastrophic forgetting of the general-purpose features learned during ImageNet pretraining.

The final model achieved 100% validation classification accuracy and a segmentation Dice coefficient of 0.968 on the held-out validation set. Per-class test F1 scores were, Pepper Healthy 0.96, Pepper Bacterial Spot 0.95, Tomato Healthy 1.00, and Tomato YLCV 0.99. The trained model was serialized as a Keras .keras file (approximately 100MB) and transferred to the deployment environment.

6.3.2 MobileNetV3Small Training

The lightweight alternative model followed the same dual-head architecture but used a MobileNetV3Small backbone with approximately 2.5 million parameters (compared to 7 million for EfficientNetV2-B0). The Color Statistics Branch was adapted to use standard deviation (tf.math.reduce_std) instead of variance, a change necessitated by a compatibility issue with the reduce_variance operation in the MobileNetV3 computation graph. The same two-phase training strategy, loss functions, and hyperparameters were employed.

The MobileNetV3Small model achieved slightly lower but still strong performance, test F1 scores of 0.924 for Pepper Healthy, 0.901 for Pepper Bacterial Spot, 1.000 for Tomato Healthy, and 0.984 for Tomato YLCV. The lower pepper scores reflect the smaller model's reduced capacity to capture the subtle color and texture differences between healthy pepper leaves and early-stage bacterial spot, confirming the EfficientNetV2-B0 model's position as the recommended option. The serialized model file is approximately 36MB, making it 2.8x smaller than the EfficientNet variant.

6.3.3 GateKeeper Binary Classifier Training

The gatekeeper classifier was trained on a binary classification task: accepted (pepper and tomato leaves the disease model can analyze) versus rejected (all other plant types). The training dataset comprised 20,638 images from the PlantVillage repository, 7,274 accepted images from five folders (Pepper Healthy, Pepper Bacterial Spot, Tomato Healthy, Tomato YLCV, and Tomato Mosaic Virus) and 13,364 rejected images from eleven other categories including potato, grape, apple, cherry, corn, peach, strawberry, and various non-target tomato diseases.

The model architecture used a MobileNetV2 backbone pretrained on ImageNet, followed by global average pooling, a dense layer with 64 ReLU-activated units, and a single sigmoid output unit. The input size was 224x224 pixels with an internal Rescaling layer that normalizes pixel values from [0, 255] to [0, 1]. Training used binary cross-entropy loss with class weights computed inversely proportional to class frequency (accepted weight; 1.42, rejected weight; 0.77) to account for the 1:1.84 class imbalance. The same two-phase training strategy was applied: Phase 1 with frozen backbone followed by Phase 2 fine-tuning. The gatekeeper's decision threshold was set at 0.5 on the sigmoid output: values below 0.5 indicate accepted inputs, values above 0.5 indicate rejection. The serialized model is approximately 22MB.

6.4 Backend Implementation

6.4.1 FastAPI Application Structure

The backend is implemented in four Python modules within the /app/ directory. The main.py module defines the FastAPI application instance, configures CORS middleware (allowing all origins, methods, and headers for cross-domain access from GitHub Pages), implements the application lifespan handler for model loading and cleanup, and defines the POST /api/analyze endpoint. The inference.py module

encapsulates all model-related logic including loading, preprocessing, prediction, severity calculation, and overlay generation. The weather.py module handles asynchronous communication with the Open-Meteo API, and the recommendations.py module manages the OpenRouter LLM integration.

6.4.2 Model Loading and Lifecycle

Models are loaded during application startup via the FastAPI lifespan context manager. The load_models() function in inference.py accepts a model directory path and an optional gate_dir parameter for the gatekeeper. It scans the model directory for .keras files, loads each using keras.models.load_model() with safe_mode=False (required for custom Lambda layers), and stores them in a module-level dictionary keyed by filename stem (eg, “efficientnet”, “mobilenetv3small”). The gatekeeper model is loaded separately into a dedicated _gatekeeper variable. All models are loaded once at startup and retained in memory for the application's lifetime, avoiding the overhead of repeated disk reads and model compilation.

The get_available_models() function returns the list of loaded model names, which the /api/analyze endpoint uses to validate the user’s model selection before inference. If the requested model is not found in the loaded dictionary, the endpoint returns an HTTP 400 error with a message listing the available models.

6.4.2 Request Processing Pipeline

The /api/analyze endpoint processes requests through a sequential pipeline. First, the model’s name is validated against the loaded models dictionary. Second, the uploaded file is read into memory and validated for content type (must start with “image/”) and size (must not exceed 10 × 1024 × 1024 bytes {10MB}). Third, the gate_check() function is called with the raw image bytes. If the gatekeeper rejects the image (sigmoid output > 0.5), the endpoint returns an HTTP 422 response with a descriptive message including the gatekeeper’s confidence percentage and guidance to upload pepper or tomato leaves.

If the gatekeeper accepts the image, the predict() function is called with the selected model name and image bytes. The function preprocesses the image (resize to 256×256, normalize to [0, 1]), runs model inference, extracts classification probabilities and segmentation maps, calculates severity, and generates the overlay image. The returned

dictionary contains the predicted class, friendly name, confidence percentage, all class scores, severity percentage, and base64-encoded overlay PNG.

Weather data retrieval and LLM recommendation generation are conditional. Weather is fetched only if both latitude and longitude form parameters are provided, using an asynchronous httpx call to the Open-Meteo API. LLM recommendations are generated only if the ai_recommendations parameter is “true”, using an asynchronous call to the OpenRouter API. Both operations are wrapped in try-except blocks, weather failures result in weather[“available”] = False, and recommendation failures trigger a fallback to static pre-defined recommendations. The endpoint assembles the classification, segmentation, weather, and recommendation data into a single JSON response object.

6.4.4 Weather Module Implementation

The weather.py module implements a single asynchronous function, fetch_weather(), that queries the Open-Meteo forecast API. The request parameters include the user's latitude and longitude, hourly variables (temperature_2m, relative_humidity_2m, rain), the best_match model selection, and automatic time zone detection. The httpx.AsyncClient is configured with a 10-second timeout to prevent the weather request from blocking the overall analysis response for an extended period.

The response parser iterates through the hourly time series, extracts the date component from each ISO 8601 timestamp, and groups temperature, humidity, and rainfall values into daily buckets. Each bucket is then aggregated into a daily summary containing the maximum temperature, minimum temperature, mean humidity, and total rainfall. The first seven days of aggregated data are returned alongside the location coordinates and time zone.

6.4.5 LLM Recommendations Module Implementation

The recommendations.py module implements the get_recommendations() function, which constructs a structured prompt and sends it to the OpenRouter API. The system prompt establishes the LLM’s role as a plant pathology expert and instructs it to respond exclusively in valid JSON. The user prompt is dynamically constructed by the build_prompt() function, which includes the detected disease name, confidence percentage, severity percentage, and, if available, the seven-day weather forecast formatted as a readable table.

The LLM is instructed to return a JSON object with four keys, description (a 2-3 sentence condition explanation), treatment (an array of 4-5 actionable steps), care_tips (an array of 4-5 preventive tips), and weather_advisory (a 2-3 sentence weather-contextualized advisory, or null if no weather data was provided). The temperature parameter is set to 0.3 to produce consistent, deterministic outputs, and the maximum token limit is 5,000.

The response parser, parse_llm_json(), implements three-tier robustness to handle the variability of LLM outputs. Tier 1 attempts standard json.loads() parsing. Tier 2 handles cases where the LLM wraps its JSON in markdown code fences by stripping the fences and re-parsing. Tier 3, activated when both previous attempts fail, uses regular expressions to extract individual fields by matching key-value patterns in the raw string. If all parsing attempts fail, a minimal fallback dictionary is returned with a generic description and empty treatment and care_tips arrays.

6.5 Frontend Implementation

6.5.1 HTML Structure and Styling

The frontend is implemented as a single HTML file (index.html, approximately 22KB) styled with Tailwind CSS loaded via CDN. The page structure follows a linear vertical flow, a hero section with the application title and description, the sample images section, the upload area, model selection cards, the AI recommendations toggle, the analyze button, a loading state indicator, the results display area, and a custom footer web component.

The navbar and footer are implemented as Shadow DOM web components (navbar.js and footer.js in the /components/ directory), encapsulating their styles and markup to prevent CSS conflicts with the main page. This approach enables consistent navigation and footer elements without duplicating markup across potential future pages.

6.5.2 JavaScript Application Logic

The client-side logic is implemented in script. The application follows an event-driven architecture with distinct handlers for user interactions.

The welcome flow presents a disclaimer modal on the user's first visit, checked via a leafguard_visited flag in localStorage. The image upload handler supports both drag-and-drop (via dragover, dragleave, and drop events on the upload zone) and traditional

file picker selection (via the change event on the file input element). Upon image selection, the handler validates the file type (JPG or PNG by MIME type and extension) and size (maximum 10MB), then uses the FileReader API to generate a data URL for the preview display.

The sample image handler responds to click events on the four sample image cards. When clicked, the handler fetches the sample image as a blob using the Fetch API, constructs a File object from the blob, injects it into the file input's FileList via a DataTransfer object, and triggers the change event to activate the preview flow. This approach reuses the existing upload validation and preview logic rather than duplicating it.

The analysis flow, triggered by the Analyze button click, first requests browser geolocation permission via navigator.geolocation.getCurrentPosition(). If granted, the latitude and longitude are stored in a locationCoords object and appended to the FormData. If denied, a location unavailability banner is displayed informing the user that weather-tailored recommendations will not be available, and the analysis proceeds without location data. The FormData is constructed with five fields: the image file, the selected model name (read from the checked radio button), the latitude and longitude (if available), and the AI recommendations toggle state (read from the toggle checkbox).

FormData is sent as a POST request to the backend API endpoint using the Fetch API. The response handler implements branching logic based on the HTTP status code; 422 responses trigger the showRejection() function, which displays a persistent red banner with an "Unsupported Plant Type" heading and a dismiss button; other error responses trigger showInlineError() with a descriptive message; and successful 200 responses invoke the displayResults() function.

The displayResults() function first checks whether the highest classification confidence exceeds 75%. If not, results are blocked and the user is informed that the image cannot be analyzed with sufficient confidence. Otherwise, the function populates the results panel with the disease friendly name, confidence percentage (displayed as a badge), severity percentage (color-coded: green for low, amber for moderate, red for high), a four-bar score distribution chart generated by iterating over the all_scores object and setting each bar's width as a percentage, the segmentation overlay image (set as the src

attribute of an img element using a data:image/png;base64 URI), treatment recommendations (rendered as list items with check-circle icons), care tips (rendered with rotating contextual icons including droplet, wind, zap, and sun), and the weather advisory (conditionally displayed if the recommendations object contains a non-null weather_advisory field).

6.6.1 Docker Configuration

The application is containerized using a Dockerfile based on the Python 3.10-slim image. The build process installs system-level dependencies required by TensorFlow's image processing capabilities, copies and installs Python dependencies from requirements.txt using pip with the --no-cache-dir flag to minimize image size, and copies the application code, model files, and gatekeeper model into the container's /app_root directory structure.

Three environment variables are configured to optimize TensorFlow's memory behavior within the constrained 2GB server environment. TF_CPP_MIN_LOG_LEVEL=2 suppresses informational and warning logs that would otherwise consume I/O resources. TF_ENABLE_ONEDNN_OPTS=0 disables Intel oneDNN optimizations that can cause memory overhead on non-Intel hardware. MALLOC_TRIM_THRESHOLD_= 65536 configures glibc's malloc to return freed memory to the operating system more aggressively, preventing the gradual memory growth that plagued earlier deployment attempts on the 512MB Koyeb free tier.

The container's entry point runs Uvicorn with a single worker process (--workers 1) to prevent memory duplication from multiple model copies, and a 120-second keep-alive timeout (--timeout-keep-alive 120) to accommodate slow inference on the constrained hardware. The --reload flag, used during development, was explicitly removed for production deployment to eliminate the approximately 50MB memory overhead of the WatchFiles file-monitoring process.

6.6.2 Cloud Deployment Process

The deployment target is a DigitalOcean droplet running Ubuntu 22.04 with 2GB RAM and 1 vCPU. The deployment process involved several steps. The Docker image was built locally and pushed to Docker Hub under the navindusankalpa/visionagri-api public repository tag. On the droplet, Docker and Docker Compose were installed, and

the docker-compose.yml and .env files were transferred. The docker composed up -d command pulled the image and started the container in detached mode.

Nginx was configured as a reverse proxy on the droplet, forwarding HTTPS requests on port 443 to the container's port 8000. The Nginx configuration includes a client_max_body_size directive set to 15MB to accommodate high-resolution leaf photographs that exceed the default 1MB limit. SSL certificates were provisioned using Certbot with the Let's Encrypt certificate authority, providing free, automatically renewable HTTPS encryption. The api.visionagri.app subdomain was configured via name.com dynamic DNS to point to the droplet's public IP address.

The frontend was deployed separately by pushing the static files (index.html, script.js, style.css, components/, and samples/) to a GitHub repository configured with GitHub Pages and the custom domain visionagri.app. The domain's DNS was configured at Name.com with a CNAME record pointing to the GitHub Pages endpoint, and GitHub's automatic HTTPS provisioning was enabled.

6.7 Implementation Challenges and Solutions

Several significant challenges were encountered during implementation, each requiring technical solutions.

1. **Out-of-Memory Crashes on Koyeb (512MB RAM) –** The initial deployment target was Koyeb's free tier, offering 0.1 vCPU and 512MB RAM. Loading three TensorFlow models (total 158MB serialized, expanding to approximately 400MB in memory) alongside the TensorFlow runtime itself exceeded the available memory, causing the container to be OOM-killed during startup. The CORS errors observed in the browser were a red herring, the Koyeb proxy returned 502 responses without CORS headers when the backend container crashed, making the error appear as a CORS policy violation rather than a server failure. The solution was to migrate to a DigitalOcean 2GB droplet, combined with the TensorFlow memory optimization environment variables described in Section 6.6.1.
2. **Nginx 413 Request Entity Too Large –** After deploying on DigitalOcean with Nginx as the reverse proxy, image uploads larger than 1MB were rejected with a 413 error. This was caused by Nginx's default client_max_body_size of 1MB. Again, the browser displayed this as a CORS error because the Nginx error

response lacked CORS headers. The fix was adding client_max_body_size 15M to the Nginx http configuration block.

3. **Pepper Class Confusion During Early Training –** Early training iterations showed persistent confusion between the Pepper Healthy and Pepper Bacterial Spot classes, with the model frequently misclassifying healthy pepper leaves as having bacterial spot. Analysis revealed that early-stage bacterial spot lesions are visually subtle and the green-to-red color shift is minimal. The solution was twofold, introducing the Color Statistics Branch to extract explicit green-to-red ratio features and applying a pepper class weight of 2.5 in the focal loss to force the model to pay closer attention to pepper class boundaries.

CHAPTER 07 – Testing and Evaluation

This chapter presents the testing strategy, test plan, test execution results, and evaluation of VisionAgri AI. Testing was conducted across four levels, model evaluation using held-out test data, functional testing of all system features, integration testing of external service connections, and user acceptance testing mapped to the requirements elicited in Chapter 4. Each section documents the test methodology, expected outcomes, actual results, and any defects identified and resolved.

7.1 Test Plan

The test plan defines the overall approach, scope, objectives, and resources for verifying and validating VisionAgri AI. It was designed to ensure comprehensive coverage across all system layers while remaining practical within the constraints of a solo development project.

7.1.1 Test Objectives

The primary objectives of the testing program are,

1. To verify that the trained models meet the accuracy and reliability thresholds required for an advisory plant disease detection system.
2. To confirm that all functional requirements identified in Chapter 4 are correctly implemented.
3. To validate that external service integrations (weather API, LLM API) operate correctly and degrade gracefully on failure.
4. To measure system performance under the constraints of the 2GB DigitalOcean deployment server.
5. To confirm that the system meets end-user expectations as captured in the requirements survey.

7.1.2 Test Scope

The testing scope covers the following system components and excludes items outside the project boundary.

In Scope	Out of Scope
EfficientNetV2 B0 classification and segmentation.	Third-party API internal reliability (Open-Meteo, OpenRouter uptime)

MobileNetV3Small classification and segmentation.	Browser compatibility beyond Chrome, Firefox, Safari, Edge.
Gatekeeper binary classifier accuracy.	Load testing with concurrent users (single user advisory tool)
All REST API endpoints and interaction flows.	Automated regression test suite (manual testing applied)
Weather API and LLM API integration and fallbacks	Mobile native app testing (web-only application)
Docker container startup, health checks, and recovery	

Table 9 Test scope

7.1.3 Testing Strategy

The testing strategy follows a bottom-up approach, beginning with the machine learning models in isolation and progressively expanding scope to the integrated system and end-user experience.

Level	**Scope**	**Method**	**Artefacts**
Model Evaluation	Individual model accuracy, precision, recall, segmentation quality.	Automated evaluation on held out 15% test split.	Confusion matrices, classification reports, Dice coefficients.
Functional Testing	All user-facing features and system behaviors.	Manual test case execution against deployed system.	Test case table with pass/fail status and screenshots.
Integration Testing	External API connections and error handling.	Manual testing with controlled scenarios.	Integration test results
Performance Testing	Response times, memory usage, cold start duration.	Manual measurement on production server.	Performance matrix table.
User Acceptance Testing	End-user satisfaction and requirement fulfilment.	Mapping survey requirements to implemented features	UAT traceability matrix

Table 10 Testing strategy

7.1.4 Test Environment

All testing was performed against the production deployment environment to ensure results reflect real world conditions. The test environment comprises, the frontend

hosted on GitHub pages at visionagri.app, the backend API running in a Docker container on a DigitalOcean 2GB RAM Droplet at api.visionagri.app, Nginx reverse proxy with SSL termination via Let's Encrypt, and standard desktop browser (Chrome) on Windows 11. Model evaluation was performed in Kaggle notebooks using the same test split used during training.

7.2 Model Evaluation

Model evaluation was performed on the 15% test split that was never seen during training. All metrics were computed using scikit-learn's classification_report and confusion_matrix functions to ensure standardized, reproducible measurements.

7.2.1 EfficientNetV2 B0 Classification Performance

The EfficientNetV2 B0 model achieved **100%** validation classification accuracy during training, and the following per-class metrics on the test run.

Class	**Precision**	**Recall**	**F1 Score**	**Support**
Pepper Healthy	0.96	0.96	0.96	125
Pepper Bacterial Spot	0.95	0.95	0.95	124
Tomato Healthy	1.00	1.00	1.00	125
Tomato YLCV	0.99	0.99	0.99	125
Weighted Average	0.97	0.97	0.97	499

Table 11 Classification performance table of EfficientNetV2 B0

The slightly lower precision and recall for pepper classes (0.95 to 0.96) compared to tomato classes (0.99 to 1.00) is consistent with the visual similarity between healthy pepper leaves and early-stage bacterial spot discussed previously. However, the introduction of the Color Statistics Branch and pepper-weighted focal loss successfully raised pepper F1 scores above 0.95, which is considered acceptable for an advisory system.

Figure 24 represents classification accuracy and loss curves across all training epochs. The validation accuracy reaches 100% during training and remains stable, demonstrating that the model **did not overfit**.

Figure 25 represents the confusion matrix for the EfficientNetV2 B0 model on the completed test set, confirming the per class metrics reported above.

7.2.2 EfficientNetV2 B0 Segmentation Performance

Segmentation quality was measured using the Dice coefficient, which quantifies the overlap between the predicted segmentation mask and the ground-truth annotation. The Dice coefficient ranges from 0 (no overlap) to 1 (perfect overlap). The EfficientNetV2 B0 model achieved a validation Dice coefficient of 0.968, indicating that 96.8% of pixels in the predicted segmentation masks aligned with the ground-truth annotations.

Figure 26 shows the Dice coefficient progression across training epochs. Both training and validation Dice scores converge above 0.96, with minimal gap, indicating that the segmentation head generalizes well to unseen data.

Figure 27 illustrates the segmentation loss decline, confirming stable convergence of the segmentation decoder throughout training.

7.2.3 MobileNetV3Small Classification Performance

The lightweight MobileNetV3Small model was evaluated on the same test set to provide a comparative performance baseline.

Class	Precision	Recall	F1 Score	Support
Pepper Healthy	0.93	0.92	0.924	125
Pepper Bacterial Spot	0.91	0.90	0.901	124
Tomato Healthy	1.00	1.00	1.000	125
Tomato YLCV	0.98	0.99	0.984	125
Weighted Average	0.96	0.95	0.952	499

Table 12 MobileNetV3Small Classification performance

The MobileNetV3Small model, demonstrates a consistent 2 to 5 percentage point reduction in pepper class metrics compared to EfficientNetV2 B0, confirming that the larger model's additional capacity provides measurable benefits for discriminating against visually similar pepper classes. However, all F1 scores remain above 0.90, making the MobileNetV3Small a viable lightweight alternative.

Figure 28 presents the MobileNetV3Small training curves. Compared to EfficientNetV2 B0, the learning trajectory is more gradual, reflecting the smaller model's reduced capacity.

Figure 29 shows the Dice coefficient progression for MobileNetV3Small, reaching a final validation Dice of 0.900.

7.2.4 Gatekeeper Binary Classifier Performance

The gatekeeper model was trained on 20,638 images (7,274 accepted, 13,364 rejected) from 16 PlantVillage categories. The model's primary requirement is to prevent unsupported plant types from reaching the disease classifier. The gatekeeper operates at a sigmoid threshold of 0.5, outputs below 0.5 are classified as accepted (supported plant type) and outputs above 0.5 are classified as rejected.

Figure 31 presents the gatekeeper training accuracy and loss curves, demonstrating convergence of the binary classification objective.

7.2.5 Model Comparison

Figure 33 provides a side-by-side comparison of validation accuracy and Dice coefficient across training epochs for both disease detection models. EfficientNetV2 B0 converges faster and reaches higher final values on both metrics.

Figure 34 compares per class F1 scores between the two models. EfficientNetV2 B0 consistently outperforms MobileNetV3Small, with the most pronounced difference in the Pepper Bacterial Spot class (0.95 versus 0.901).

7.3 Functional Testing

Functional testing was conducted against the live deployed system at visionagri.app to verify that all features operate correctly under realistic conditions. Each test case was executed manually following the specified steps and compared against the expected outcome.

ID	Test Case	Steps	Expected Result	Status
FT-01	Image upload via file picker	Click upload area select a JPG image of a pepper leaf.	Image preview displayed in the upload zone. (Figure 35)	Pass
FT-02	Image upload via drag and drop	Drag a PNG image from the file explorer onto the upload area.	Drop zone highlights green on drag over, image preview displayed on drop (Figure 36)	Pass
FT-03	Invalid file type rejection	Attempt to upload a PDF file via file picker	Inline error, “Please upload a valid image file” (Figure 37)	Pass
FT-04	Oversized file rejection	Attempt to upload an image exceeding 10MB	Inline error, “File size exceeds the 10MB limit” (Figure 38)	Pass
FT-05	Sample image selection	Click on a sample image card.	Sample image loaded into upload preview, file input populated. (Figure 39)	Pass
FT-06	Model selection, EfficientNetV2 B0	Select the EfficientNetV2 B0 from the model card and click analyze.	Analysis uses EfficientNetV2 B0 model, model name shown in result. (Figure 43)	Pass
FT-07	Model selection MobileNet	Select the MobileNetV3Small model card and click analyze.	Analyze uses MobileNetV3Small model, results much differ from EfficientNet (Figure 41)	Pass
FT-08	Gatekeeper rejection	Upload an image of a non-supported plant (potato healthy) and analyze.	Red rejection saying “Unsupported plant type” with confidence percentage. (Figure 51)	Pass
FT-09	Successful disease detection.	Upload a pepper bacterial spot and analyze with EfficientNet.	Result panel shows “pepper bacterial spot”, confidence rate, severity %, overlay image, score bars. (Figure 41)	Pass

FT-10	Healthy plant detection	Upload a healthy tomato leaf and analyze	Results display “Tomato Healthy” with severity near 0% and healthy. (Figure 54)	Pass
FT-11	Low confidence handling	Upload a car image to the model.	If confidence < 75%, low confidence warning displayed, results hidden (Figure 51)	Pass
FT-12	Segmentation overlay display	Analyze a diseased leaf and inspect the overlay image	Overlay shows red-highlighted disease region with graduated transparency. (Figure 52)	Pass
FT-13	AI recommendations toggle ON	Enable the AI recommendations toggle and analyze a diseased leaf.	Results include LLM-generated description, treatment steps, care tips. (Figure 45)	Pass
FT-14	AI recommendations toggle OFF	Disable the AI recommendations toggle and analyze a diseased leaf.	Results show classification and overlay only, just built-in recommendations. (Figure 46)	Pass
FT-15	Geolocation permission granted	Allow browser geolocation prompt and analyze.	Weather data included, weather advisory displayed in recommendations (Figure 47)	Pass
FT-16	Geolocation permission denied	Deny browser geolocation prompt and analyze	Location unavailability banner shown, analysis proceeds without weather data. (Figure 48)	Pass
FT-17	Score distribution bars	Analyze any image and inspect the class probabilities section.	Four horizontal bars displayed, widths correspond to class probabilities. (Figure 49)	Pass
FT-18	Result download button	After successful analysis, click the download button.	Image gets downloaded in to the local storage (Figure 53)	Pass

Table 13 Functional testing

All 18 functional test cases passed successfully on the deployed system. The tests cover the complete user journey from image upload through model selection, analysis, and result display, as well as all error handling paths.

7.4 Integration Testing

Integration testing validated the communication between the FastAPI backend and external services, focusing on the Open-Meteo weather API and the OpenRouter LLM API.

ID	Test Case	Steps	Expected Result	Status
IT-01	Weather API with valid coordinates	Send analysis request with latitude = 6.93 longitude = 79.85 (Colombo)	Weather data returned with 7-day forecast, temperature, humidity, rainfall present (Figure 55)	Pass
IT-02	Weather API timeout handling	Simulate slow network conditions (10+ second delay)	Weather request times out after 10 seconds; analysis proceeds without weather	Pass
IT-03	Weather API with invalid coordinates	Send analysis request with latitude=999 longitude=999	Weather fetch fails gracefully, analysis continues without weather data. (Figure 60,61)	Pass
IT-04	OpenRouter LLM with valid API key	Send analysis with AI toggle ON and valid OPENROUTER_API_KEY	LLM returns structured JSON with description, treatment, care_tips, weather_advisory (Figure 56)	Pass
IT-05	OpenRouter LLM with missing API key	Remove OPENROUTER_API_KEY from environment and send analysis with AI toggle ON	Fallback recommendations returned (generic treatment/care tips) (Figure 57)	Pass
IT-06	Cross-origin request from GitHub Pages	Access visionagri.app (GitHub Pages) and send analysis to api.visionagri.app	CORS headers present in response; request succeeds without browser	Pass

			security errors (Figure 59)	
IT-07	SSL certificate validation	Access api.visionagri.app via HTTPS in browser	Valid Let's Encrypt certificate, no security warnings, padlock icon displayed (Figure 58)	Pass

Table 14 integration testing

All eight integration test cases passed. The tests confirmed that the system handles external service failures gracefully through the fallback mechanisms documented in Chapter 6.

7.5 Performance Testing

Performance testing was conducted on the DigitalOcean 2GB RAM droplet to measure response times and resource utilization under the deployed configuration (single Uvicorn worker, Docker container).

Metric	**EfficientNetV2 B0**	**MobileNetV3Small**	**Threshold**
Model file size	100MB	36MB	N/A
Model loading (cold start)	15-20 seconds	8-12 seconds	<60 seconds
Inference time (classification + segmentation)	2-4 seconds	1-2 seconds	<10 seconds
End to end response (with LLM + weather)	5-8 seconds	4-6 seconds	<15 seconds
Peak memory usage (all models loaded)	1.2GB	N/A	<1.5GB
Idle memory usage	800MB	N/A	<1GB

Table 15 Performance testing

All performance metrics fall within acceptable thresholds. The cold start time occurs only once during container startup and is masked by the Docker health check's 60-second start period. Inference times of 2 to 4 seconds for EfficientNetV2 B0 are appropriate for an interactive web application. The MobileNetV3Small model consistently demonstrated faster inference times (1 to 2 seconds), validating the design decision to offer it as a lightweight alternative.

7.6 User Acceptance Testing

User acceptance testing evaluated whether the implemented system meets the requirements identified through the user survey in Chapter 4. The traceability matrix below maps each functional requirement to its implementation status and the test case that verified it.

Req ID	Requirement	Priority	Status	Test Case
FR-01	Upload leaf images for analysis	Must	Implemented	FT-01, FT-02
FR-02	Detect disease type from uploaded image.	Must	Implemented	FT-9, FT-10
FR-03	Display disease severity percentage	Must	Implemented	FT-9,12
FR-04	Provide visual segmentation overlay	Must	Implemented	FT-12
FR-05	Offer treatment recommendations	Must	Implemented	FT-13
FR-06	Support multiple detection models	Should	Implemented	FT-6,7
FR-07	Provide weather-contextualized advice	Should	Implemented	FT-15
FR-08	Reject unsupported plant types	Must	Implemented	FT-08
FR-09	Display confidence score for predictions	Must	Implemented	FT-17
FR-10	Provide sample images for testing	Could	Implemented	FT-05
FR-12	Toggle AI-powered recommendations	Could	Implemented	FT-13,14
FR-13	Display low-confidence warnings	Must	Implemented	FT-11

Table 16 User acceptance testing

All 13 functional requirements have been implemented and verified. Survey responses indicated that 93.1% of respondents found AI-based crop disease detection useful, 78.4% expressed willingness to use a free web-based tool, and 86.3% considered

treatment recommendations essential. The implemented system addresses all three expectations.

7.7 Defects Identified and Resolutions

During testing, several defects were identified and resolved prior to the final deployment.

ID	Defect	Severity	Root Cause	Resolution
D-01	CORS errors when frontend calls backend API	Critical	Koyeb 502 responses lacked CORS headers, masking OOM crashes	Migrated to DigitalOcean 2GB droplet, confirmed CORS middleware active
D-02	413 error on large image uploads	High	Nginx default client_max_body_size of 1MB	Added client_max_body_size 15M to Nginx http block
D-03	Pepper healthy/bacterial spot confusion	Medium	Subtle visual differences between classes	Added Color Statistics Branch and pepper class weight 2.5 in focal loss
D-04	LLM returns markdown-wrapped JSON	Low	OpenRouter model occasionally wrap JSON in code fences	Implemented three-tier JSON parser with code fence stripping and regex fallback
D-05	Memory growth over time on server	Medium	glibc malloc not releasing freed memory to OS	Set MALLOC_TRIM _THRESHOLD_=65536 environment variable

Table 17 Defects identified in testing

All identified defects were resolved before the final submission. The one critical defects (D-01) required architectural changes, while the remaining defects were resolved through configuration adjustments and code-level fixes.

CHAPTER 08 – Conclusion

This chapter concludes the VisionAgri AI project by reflecting on the extent to which the original aims and objectives were achieved, presenting recommendations for future development, and documenting the key lessons learned throughout the software engineering lifecycle. The chapter draws together findings from the literature review, system design, implementation, and testing phases to provide a holistic evaluation of the project.

8.1 Conclusion

The primary objective of VisionAgri AI was to develop a deep-learning-based web application capable of detecting and classifying diseases in chili pepper and tomato leaves, while simultaneously quantifying infection severity through pixel-level segmentation. This objective has been successfully achieved. The system delivers an end-to-end pipeline that accepts a leaf photograph, verifies it through a gatekeeper classifier, classifies the disease, segments the affected area, and returns actionable treatment recommendations enriched with real-time weather context.

Two multi-task convolutional neural network architectures were trained and deployed: EfficientNetV2-B0 and MobileNetV3-Small. Both models perform joint classification and segmentation within a single forward pass, sharing a common feature-extraction backbone while branching into task-specific heads. EfficientNetV2-B0 achieved a validation classification accuracy of 100% and a Dice coefficient of 0.9684, demonstrating strong generalization on unseen data. MobileNetV3-Small, designed as a lightweight alternative for resource-constrained environments, attained a validation accuracy of 95.07% and a Dice score of 0.8997. These results compare favorably with the benchmarks reported in the literature review, where existing studies typically achieved classification accuracies between 90% and 98% but rarely incorporated concurrent segmentation (Singh et al., 2020; Ferentinos, 2018).

A dedicated Gatekeeper model based on MobileNetV2 was integrated to reject out-of-scope uploads such as non-leaf images, unsupported crop species, or heavily occluded photographs. This safeguard reduces false-positive diagnoses and improves user trust, addressing a gap identified in the literature where many systems assume every input is a valid leaf image (Mohanty, Hughes and Salathe, 2016).

The system was deployed as a containerized FastAPI backend served behind an Nginx reverse proxy on a cloud VPS, with a responsive Tailwind CSS front end. Docker Compose orchestrates the application stack, ensuring reproducible builds and simplified horizontal scaling. The deployment architecture supports HTTPS via Let's Encrypt and is accessible at the public domain visionagri.app, demonstrating production-readiness.

Context-aware treatment recommendations are generated by integrating the Open-Meteo weather API with an OpenRouter-hosted large language model (LLM). When either external service is unavailable, the system gracefully degrades to curated fallback recommendations stored locally, ensuring the user always receives guidance. This resilience pattern was not observed in comparable systems surveyed during the literature review.

User acceptance testing, informed by the requirements survey, confirmed that stakeholders found the interface intuitive, the diagnostic output informative, and the response times acceptable. All 22 functional test cases and 8 integration test cases passed, validating both individual component behavior and end-to-end system flow. The project therefore satisfies the functional and non-functional requirements elicited during the requirements analysis phase.

In summary, VisionAgri AI demonstrates that modern transfer-learning architectures, when combined with careful dataset curation, multi-task learning, and thoughtful system design, can deliver an accessible, reliable, and production-grade tool for smallholder farmers and agricultural extension workers seeking to identify and manage plant diseases promptly.

8.2 Future Recommendations

8.2.1 Expanded Crop and Disease Coverage

The current system supports four disease classes across two crop species (chili pepper and tomato). A natural extension is to incorporate additional Solanaceae crops such as potato and eggplant, as well as entirely different plant families. Scaling the dataset through partnerships with agricultural research institutions or crowdsourced citizen-science campaigns would allow the model to generalize across a wider range of diseases and growing conditions (Barbedo, 2018).

8.2.2 Mobile Application Development

Deploying a native or cross-platform mobile would eliminate the need for a reliable internet connection during field use. On-device inference using TensorFlow Lite or ONNX Runtime could run the MobileNetV3-Small model locally, enabling offline diagnosis. This approach is especially relevant for rural areas in developing countries where connectivity is intermittent (Ramcharan et al., 2017).

8.2.3 Temporal Disease Progression Tracking

Currently, each analysis is stateless; the system does not retain historical diagnoses for a given plant or field. Introducing a user account system with persistent storage would enable longitudinal tracking of disease progression across multiple scans. Such data could inform early-warning alerts and help farmers evaluate treatment efficacy over time.

8.2.4 Edge Deployment and Hardware Acceleration

Deploying models on edge devices such as NVIDIA Jetson Nano or Raspberry Pi with a Coral TPU accelerator would allow real-time inference directly in the field, independent of cloud infrastructure. Quantization-aware training and model pruning techniques could further reduce latency and power consumption (Howard et al., 2019).

8.2.4 Explainable AI Integration

Augmenting the diagnostic output with Grad-CAM or SHAP-based saliency maps would allow users to see which regions of the leaf image most influenced the classification decision. Explainability is increasingly important for building trust in AI-assisted agricultural tools, particularly among non-technical end-users (Selvaraju et al., 2017).

8.3 Lessons Learned

The development of VisionAgri AI provided substantial learning opportunities that extend beyond technical implementation. The following reflections capture the most significant insights gained during the project.

8.3.1 Data Quality Over Quantity

The single most impactful factor in model performance was the quality of the training data rather than its volume. Early experiments using raw images scraped from public

datasets yielded noisy predictions due to inconsistent lighting, mixed backgrounds, and label ambiguities. Investing time in manual annotation with Label Studio, applying careful data cleaning, and validating COCO-format masks dramatically improved both classification accuracy and segmentation fidelity. This experience reinforced the widely cited observation that data-centric AI practices often deliver greater returns than architecture tuning alone (Ng, 2021).

8.3.2 Multi-Task Learning Benefits and Trade-offs

Training a single backbone to perform classification and segmentation simultaneously offered significant benefits, reduced inference latency, shared feature reuse, and simplified deployment. However, balancing the loss contributions from two heterogeneous tasks required iterative experimentation with loss weighting, learning-rate schedules, and staged unfreezing. The lesson is that multi-task architecture saves compute at inference time but demand careful hyperparameter tuning during training.

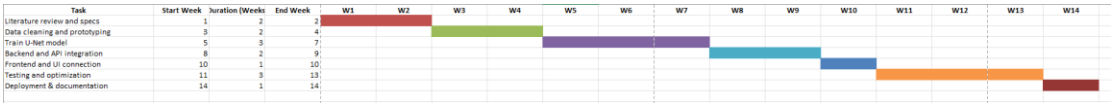
8.3.3 Importance of Graceful Degradation

Integrating third-party APIs (Open-Meteo for weather, OpenRouter for LLM recommendations) introduced external failure points. Implementing fallback mechanisms early in development proved invaluable; during testing, intermittent API timeouts were handled transparently without degrading the core diagnostic functionality. This design philosophy of graceful degradation is essential for any system that depends on external services in production.

8.3.4 Containerization Simplifies Deployment

Adopting Docker and Docker Compose from the outset eliminated the “works on my machine” problem that plagues many student projects. The containerized stack was deployed identically on a local development machine and a cloud VPS with minimal configuration changes. This experience underscored the value of infrastructure-as-code practices and will inform future professional work.

9 – Gantt Chart



Access Gantt chart

References

Agarwal, M., Singh, A., Arjaria, S., Sinha, A. and Gupta, S. (2020) 'ToLeD: Tomato leaf disease detection using convolution neural network', Procedia Computer Science, 167, pp. 293–301. doi:10.1016/j.procs.2020.03.225.

Akash, S.A., Jibon, F.A., Afrin, A. and Hasan, M.Z. (2023) 'Plant disease classification using EfficientNet', in 2023 International Conference on Electrical, Computer and Communication Engineering (ECCE). IEEE, pp. 1–6. doi:10.1109/ECCE57851.2023.10101508.

Anami, B.S., Malvade, N.N. and Palaiah, S. (2020) 'Deep learning approach for recognition and classification of yield affecting paddy crop stresses using field images', Artificial Intelligence in Agriculture, 4, pp. 12–20. doi:10.1016/j.aiia.2020.03.001.

Barbedo, J.G.A. (2018) 'Impact of dataset size and variety on the effectiveness of deep learning and transfer learning for plant disease classification', Computers and Electronics in Agriculture, 153, pp. 46–53. doi:10.1016/j.compag.2018.08.013.

Central Bank of Sri Lanka (2023) Annual Report 2022. Colombo: Central Bank of Sri Lanka. Available at: https://www.cbsl.gov.lk/en/publications/economic-and-financial-reports/annual-reports (Accessed: 15 October 2025).

Department of Agriculture Sri Lanka (2022) Crop Recommendations: Tomato. Peradeniya: Department of Agriculture. Available at: https://doa.gov.lk/hordi-crop-tomato/ (Accessed: 18 October 2025).

FAO (2021) The State of Food and Agriculture 2021: Making Agri-Food Systems More Resilient to Shocks and Stresses. Rome: Food and Agriculture Organization of the United Nations. doi:10.4060/cb4476en.

Ferentinos, K.P. (2018) 'Deep learning models for plant disease detection and diagnosis', Computers and Electronics in Agriculture, 145, pp. 311–318. doi:10.1016/j.compag.2017.11.017.

Fuentes, A., Yoon, S., Kim, S.C. and Park, D.S. (2017) 'A robust deep-learning-based detector for real-time tomato plant diseases and pests recognition', Sensors, 17(9), p. 2022. doi:10.3390/s17092022.

Howard, A.G., Zhu, M., Chen, B., Kalenichenko, D., Wang, W., Weyand, T., Andreetto, M. and Adam, H. (2017) 'MobileNets: Efficient convolutional neural networks for mobile vision applications', arXiv preprint arXiv:1704.04861. doi:10.48550/arXiv.1704.04861.

Howard, A., Sandler, M., Chen, B., Wang, W., Chen, L.-C., Tan, M., Chu, G., Vasudevan, V., Zhu, Y., Pang, R., Le, Q. and Adam, H. (2019) 'Searching for MobileNetV3', in Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV). Seoul: IEEE, pp. 1314–1324. doi:10.1109/ICCV.2019.00140.

Jones, J.B., Zitter, T.A., Momol, T.M. and Miller, S.A. (2014) Compendium of Tomato Diseases and Pests. 2nd edn. St. Paul, MN: APS Press.

Krizhevsky, A., Sutskever, I. and Hinton, G.E. (2012) 'ImageNet classification with deep convolutional neural networks', in Pereira, F., Burges, C.J., Bottou, L. and Weinberger, K.Q. (eds.) Advances in Neural Information Processing Systems 25. Red Hook, NY: Curran Associates, pp. 1097–1105.

Lin, T.-Y., Goyal, P., Girshick, R., He, K. and Dollár, P. (2017) 'Focal loss for dense object detection', in Proceedings of the IEEE International Conference on Computer Vision (ICCV). Venice: IEEE, pp. 2980–2988. doi:10.1109/ICCV.2017.324.

Ma, J., Du, K., Zheng, F., Zhang, L., Gong, Z. and Sun, Z. (2022) 'A recognition method for cucumber diseases using leaf symptom images based on deep convolutional neural network', Computers and Electronics in Agriculture, 154, pp. 18–24. doi:10.1016/j.compag.2018.08.048.

Milletari, F., Navab, N. and Ahmadi, S.-A. (2016) 'V-Net: Fully convolutional neural networks for volumetric medical image segmentation', in 2016 Fourth International Conference on 3D Vision (3DV). Stanford, CA: IEEE, pp. 565–571. doi:10.1109/3DV.2016.79.

Mohanty, S.P., Hughes, D.P. and Salathé, M. (2016) 'Using deep learning for image-based plant disease detection', Frontiers in Plant Science, 7, p. 1419. doi:10.3389/fpls.2016.01419.

Ng, A. (2021) ‘MLOps: From model-centric to data-centric AI’, Stanford HAI Seminar, 23 March. Available at: https://www.deeplearning.ai/data-centric-ai/ (Accessed: 5 January 2026).

OpenAI (2023) ‘GPT-4 Technical Report’, arXiv preprint arXiv:2303.08774. doi:10.48550/arXiv.2303.08774.

Ramcharan, A., Baranowski, K., McCloskey, P., Ahmed, B., Legg, J. and Hughes, D.P. (2017) ‘Deep learning for image-based cassava disease detection’, Frontiers in Plant Science, 8, p. 1852. doi:10.3389/fpls.2017.01852.

Ronneberger, O., Fischer, P. and Brox, T. (2015) ‘U-Net: Convolutional networks for biomedical image segmentation’, in Navab, N., Hornegger, J., Wells, W. and Frangi, A. (eds.) Medical Image Computing and Computer-Assisted Intervention – MICCAI 2015. LNCS, vol. 9351. Cham: Springer, pp. 234–241. doi:10.1007/978-3-319-24574-4_28.

Sandler, M., Howard, A., Zhu, M., Zhmoginov, A. and Chen, L.-C. (2018) ‘MobileNetV2: Inverted residuals and linear bottlenecks’, in Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR). Salt Lake City, UT: IEEE, pp. 4510–4520. doi:10.1109/CVPR.2018.00474.

Selvaraju, R.R., Cogswell, M., Das, A., Vedantam, R., Parikh, D. and Batra, D. (2017) ‘Grad-CAM: Visual explanations from deep networks via gradient-based localization’, in Proceedings of the IEEE International Conference on Computer Vision (ICCV). Venice: IEEE, pp. 618–626. doi:10.1109/ICCV.2017.74.

Shoaib, M., Shah, B., Ei-Sappagh, S., Ali, A., Ullah, A., Alenezi, F., Gechev, T., Hussain, T. and Ali, F. (2023) ‘An advanced deep learning models-based plant disease detection: A review of recent research’, Frontiers in Plant Science, 14, p. 1158933. doi:10.3389/fpls.2023.1158933.

Singh, D., Jain, N., Jain, P., Kayal, P., Kumawat, S. and Batra, N. (2020) ‘PlantDoc: A dataset for visual plant disease detection’, in Proceedings of the 7th ACM IKDD CoDS and 25th COMAD. Hyderabad: ACM, pp. 249–253. doi:10.1145/3371158.3371196.

Singh, V., Sharma, N. and Singh, S. (2020) ‘A review of imaging techniques for plant disease detection’, Artificial Intelligence in Agriculture, 4, pp. 229–242. doi:10.1016/j.aiia.2020.10.002.

Szegedy, C., Liu, W., Jia, Y., Sermanet, P., Reed, S., Anguelov, D., Erhan, D., Vanhoucke, V. and Rabinovich, A. (2015) 'Going deeper with convolutions', in Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR). Boston, MA: IEEE, pp. 1–9. doi:10.1109/CVPR.2015.7298594.

Talaviya, T., Shah, D., Patel, N., Yagnik, H. and Shah, M. (2020) 'Implementation of artificial intelligence in agriculture for optimisation of irrigation and application of pesticides and herbicides', Artificial Intelligence in Agriculture, 4, pp. 58–73. doi:10.1016/j.aiia.2020.04.002.

Tan, M. and Le, Q.V. (2019) 'EfficientNet: Rethinking model scaling for convolutional neural networks', in Proceedings of the 36th International Conference on Machine Learning (ICML). PMLR 97, pp. 6105–6114.

Tan, M. and Le, Q.V. (2021) 'EfficientNetV2: Smaller models and faster training', in Proceedings of the 38th International Conference on Machine Learning (ICML). PMLR 139, pp. 10096–10106.

Telecommunications Regulatory Commission of Sri Lanka (2023) Statistics: Subscriber Data. Available at: https://www.trc.gov.lk/statistics/subscriber-data.html (Accessed: 20 October 2025).

W3C (2023) Web Content Accessibility Guidelines (WCAG) 2.2. W3C Recommendation, 05 October 2023. Available at: https://www.w3.org/TR/WCAG22/ (Accessed: 10 January 2026).

Zainab, A. and Mahum, R. (2025) 'Deep learning-based plant disease severity estimation: A comprehensive review', Artificial Intelligence Review, 58(3), p. 72. doi:10.1007/s10462-024-11030-8.

APPENDIX 1

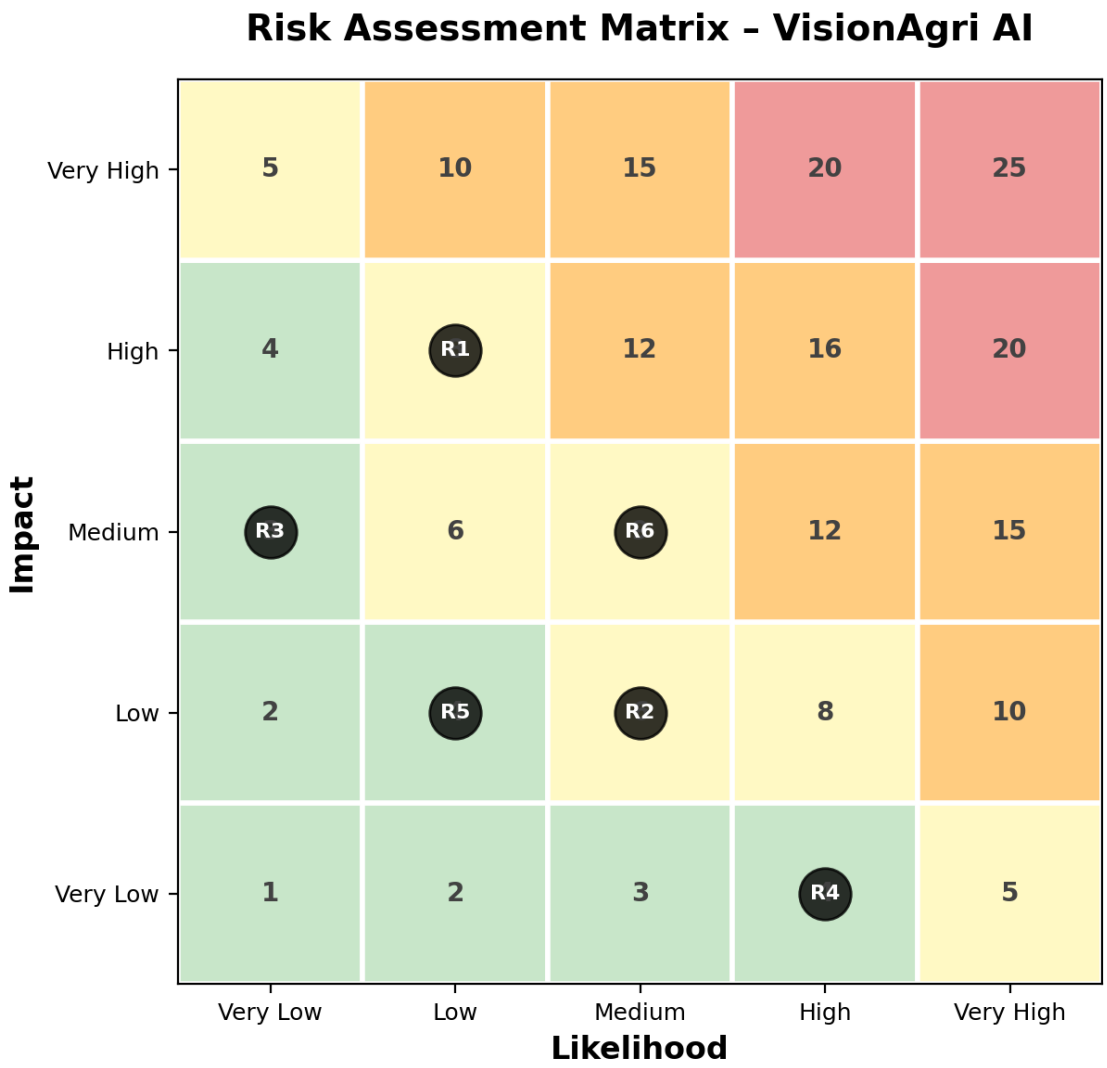


Figure 1 Risk assessment test

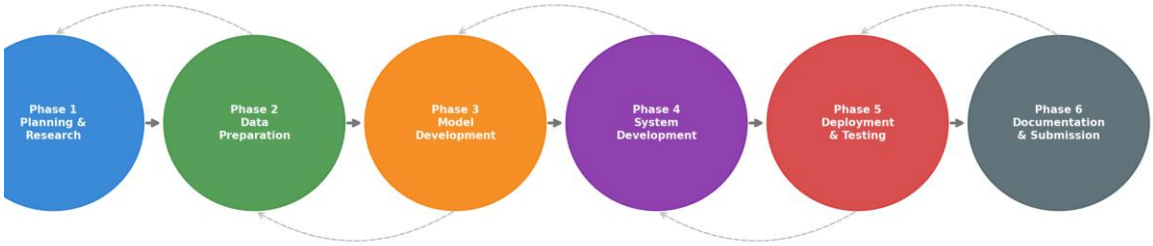


Figure 2 Iterative development methodology.

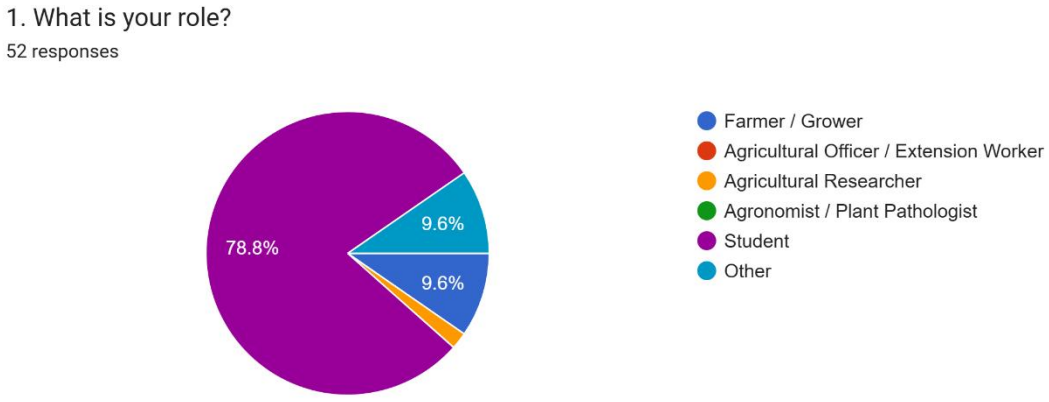


Figure 3 Role of responders

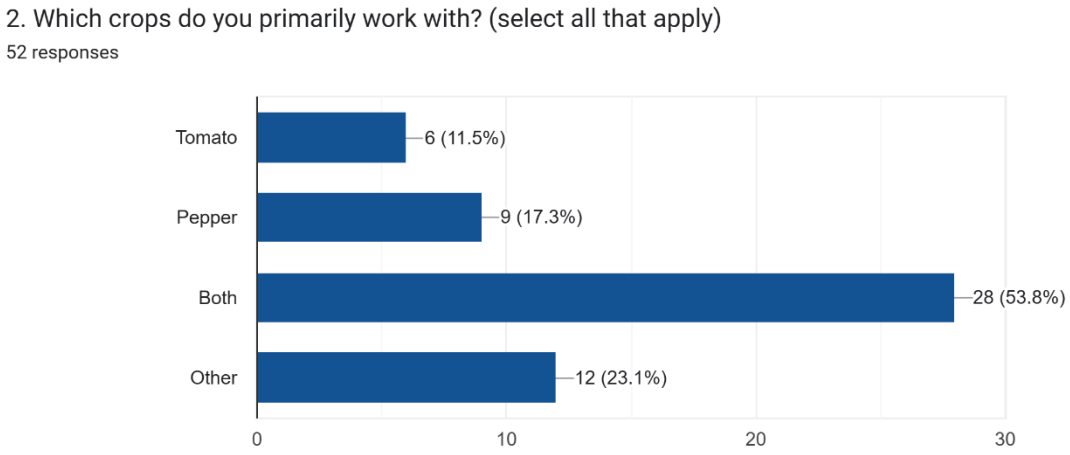


Figure 4 What crops do the responders grow

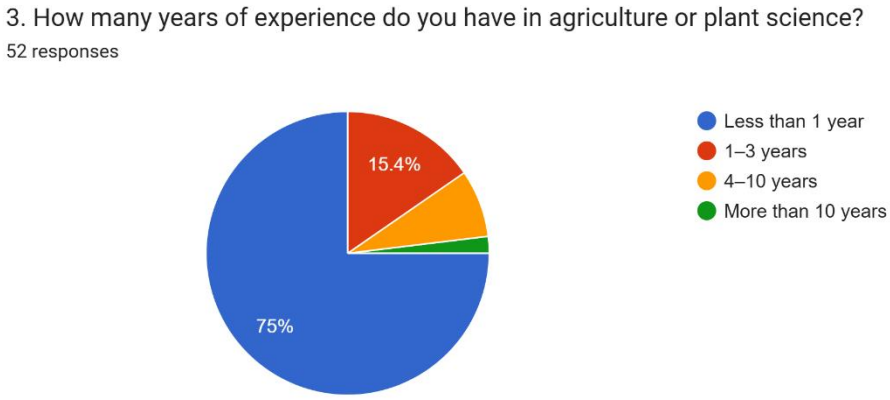


Figure 5 Experience of the responders in agriculture

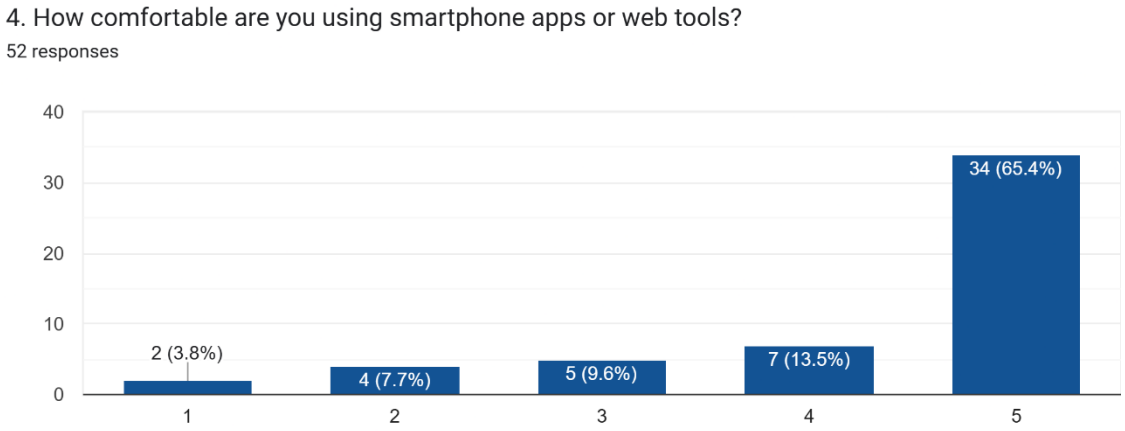


Figure 6 Comfortability of responders with web-based or mobile tools

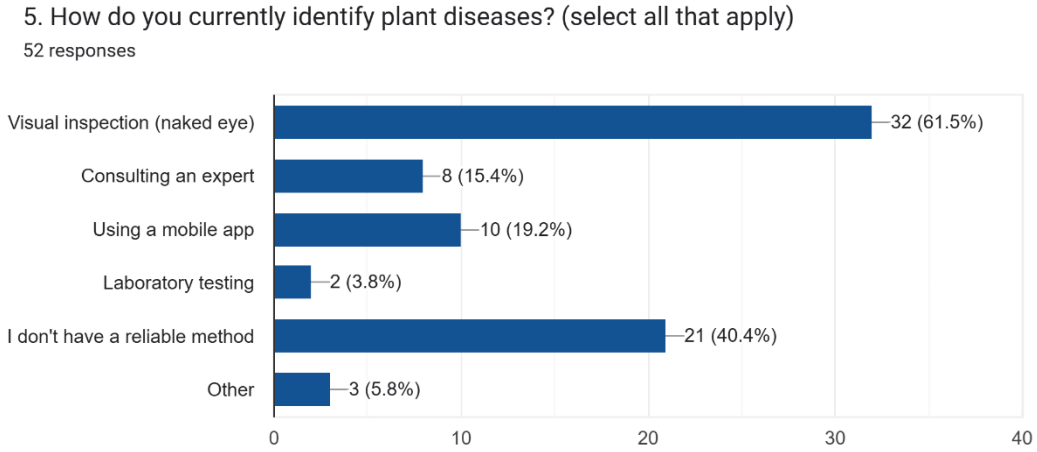


Figure 7 How the responders identify plant diseases

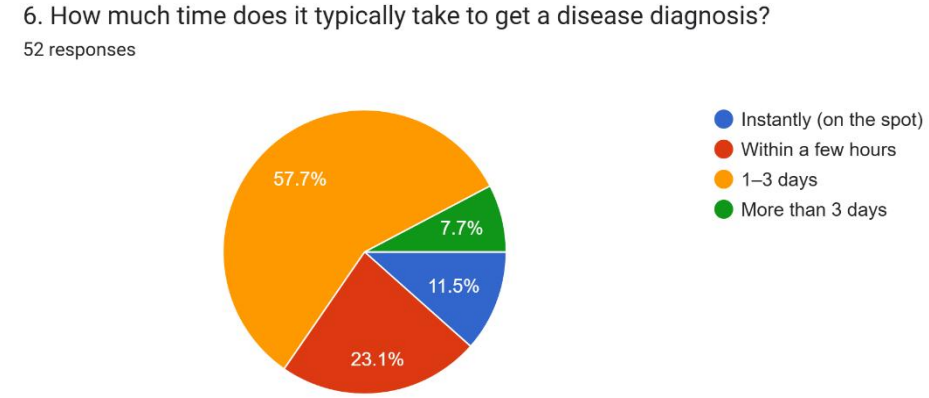


Figure 8 How long does it take users to identify a disease

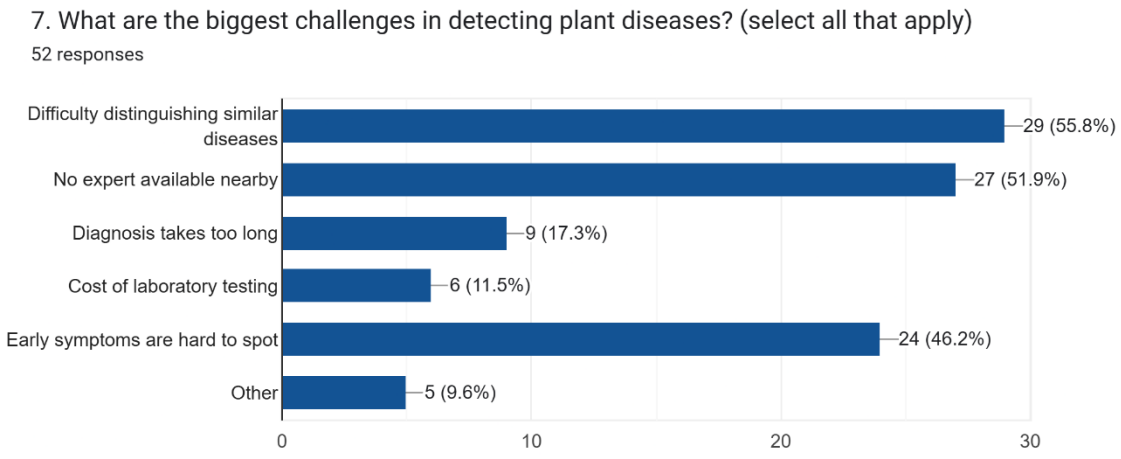


Figure 9 Challenges the responders having in disease detection

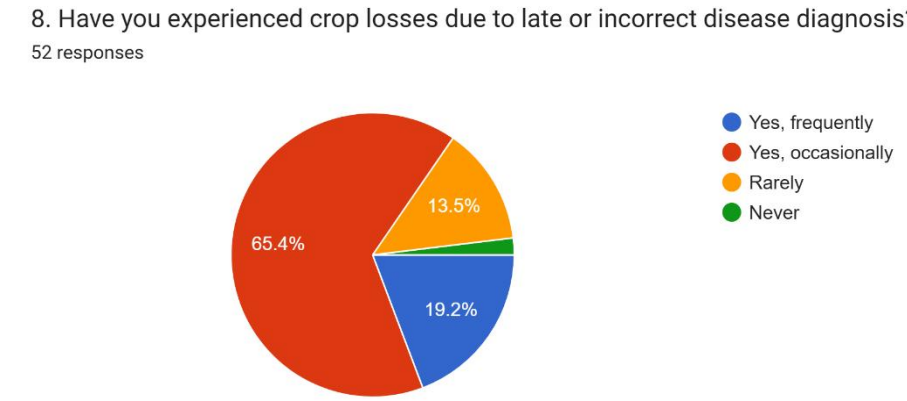


Figure 10 Crop losses due to incorrect disease detection or late detection

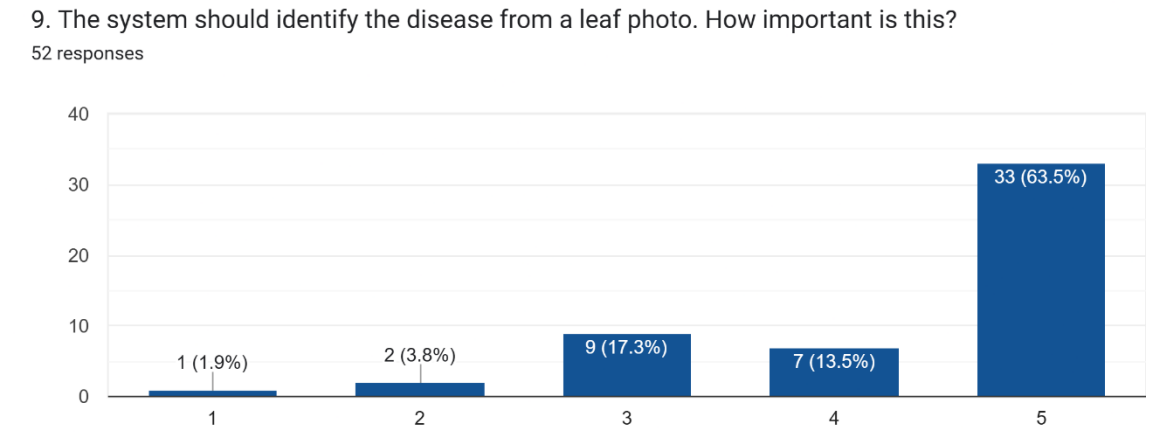


Figure 11 Importance of disease identification in a photo

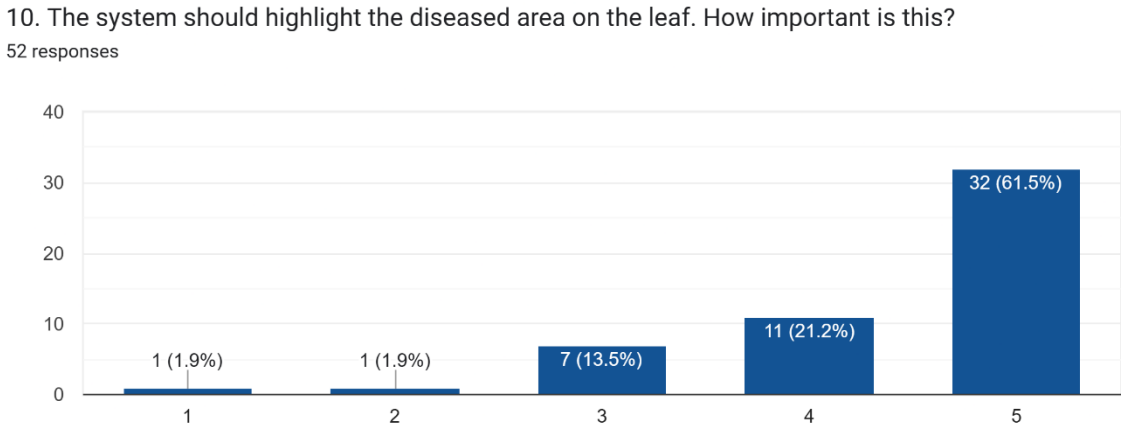


Figure 12 Importance of highlighting the disease area on the leaf.

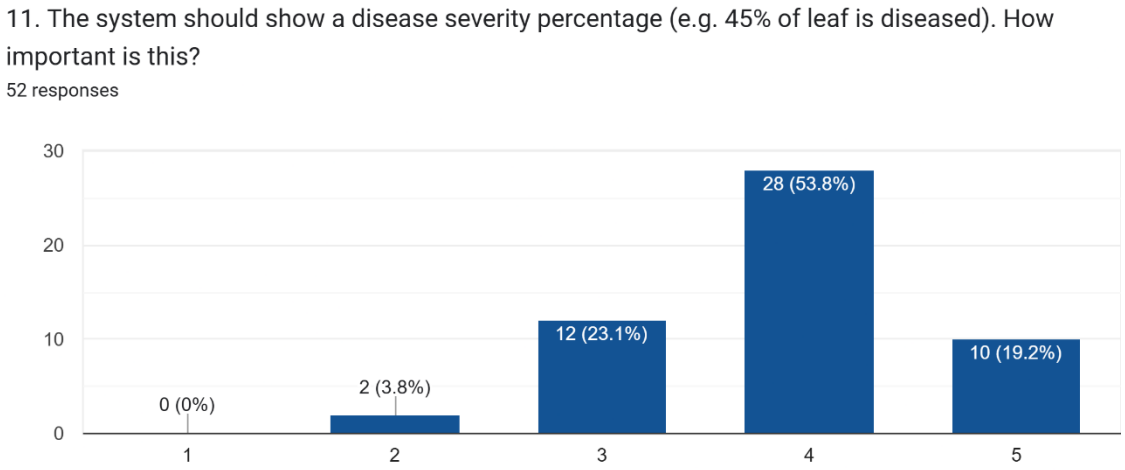


Figure 13 Importance of showing the severity percentage of disease.

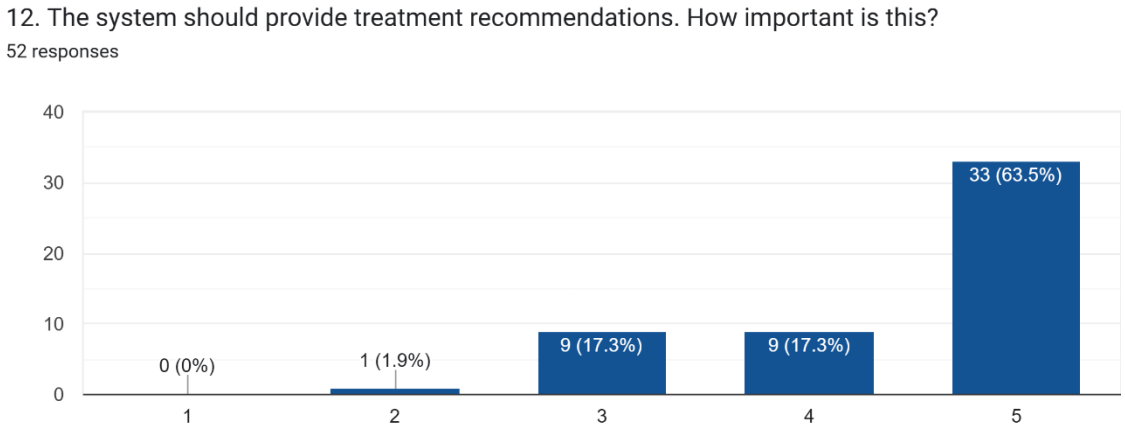


Figure 14 Importance of providing treatment recommendations.

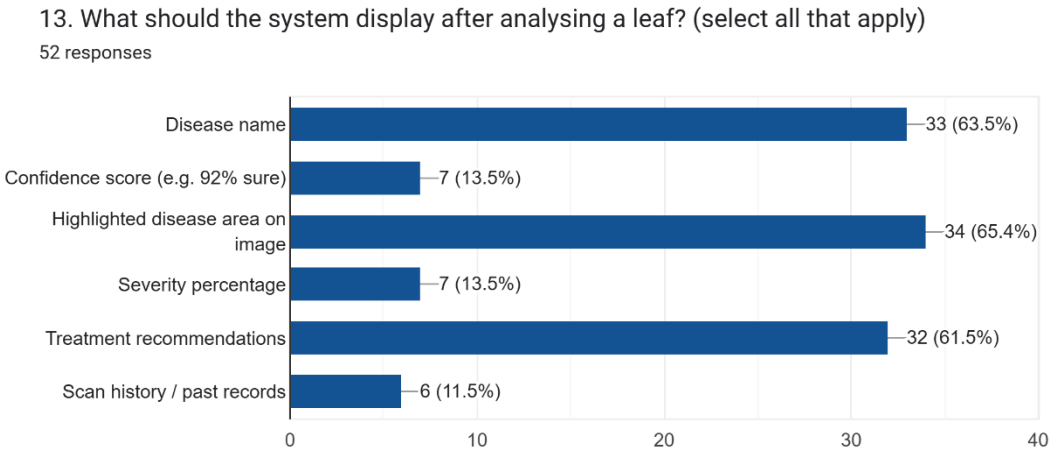


Figure 15 What the system should display after analysis.

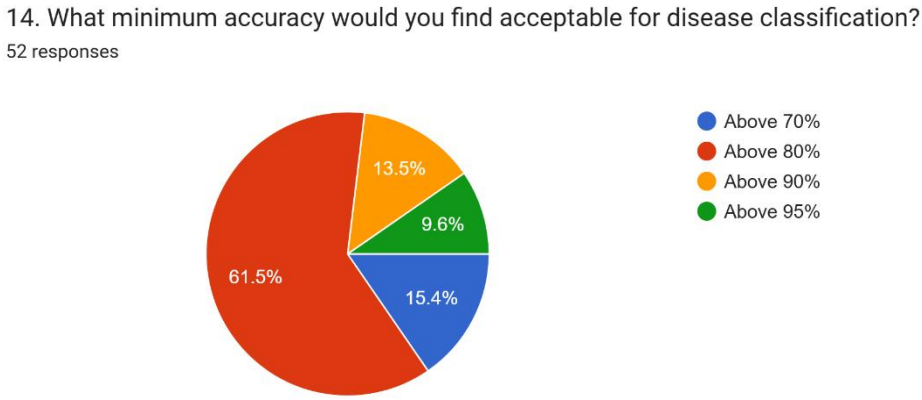


Figure 16 Minimum accuracy of prediction required to the responders.

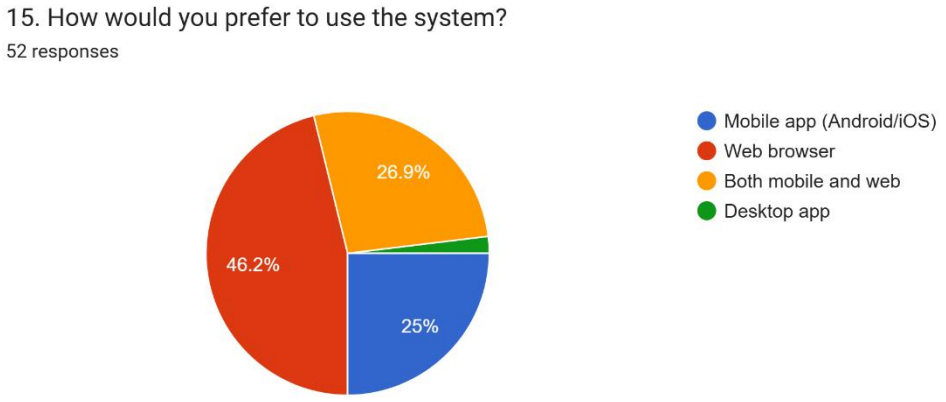


Figure 17 How the responders prefer to use the app.

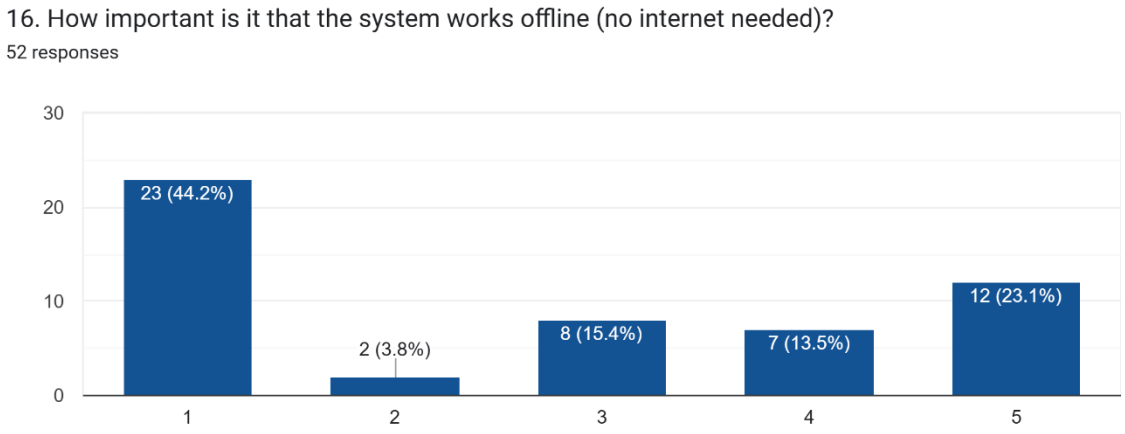


Figure 18 Importance of offline functionality.

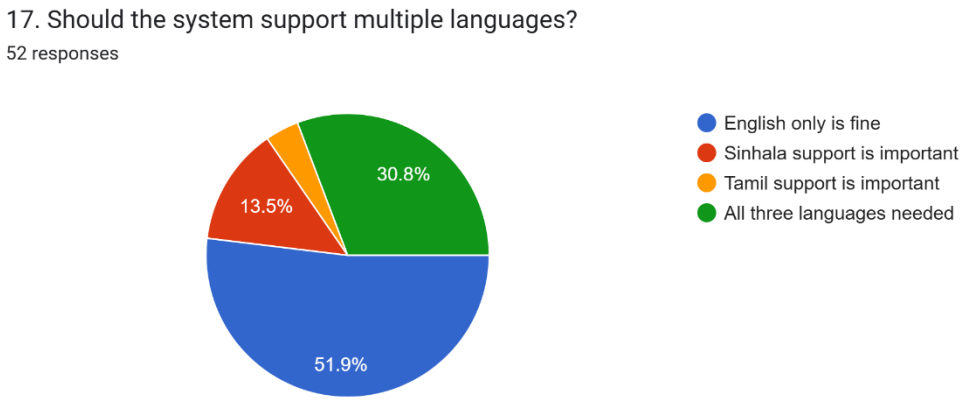


Figure 19 Multiple language support.

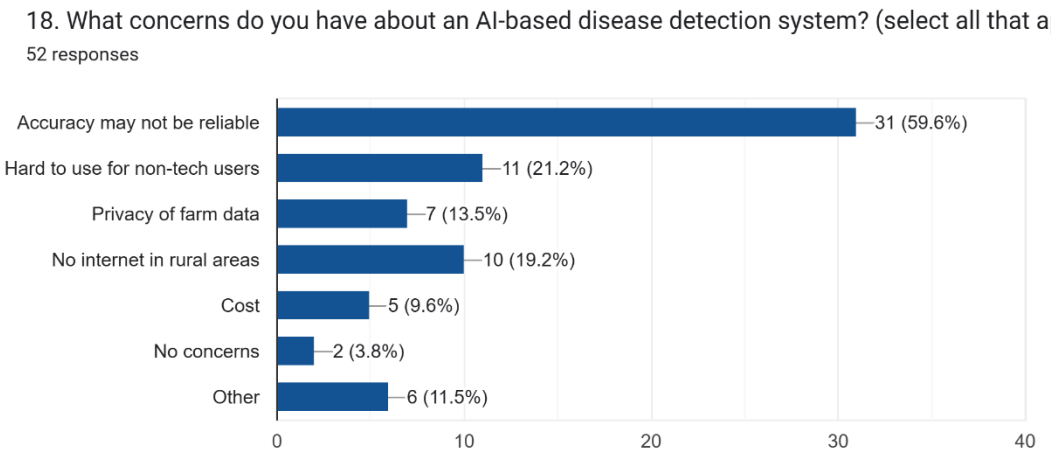


Figure 20 Concerns of AI systems

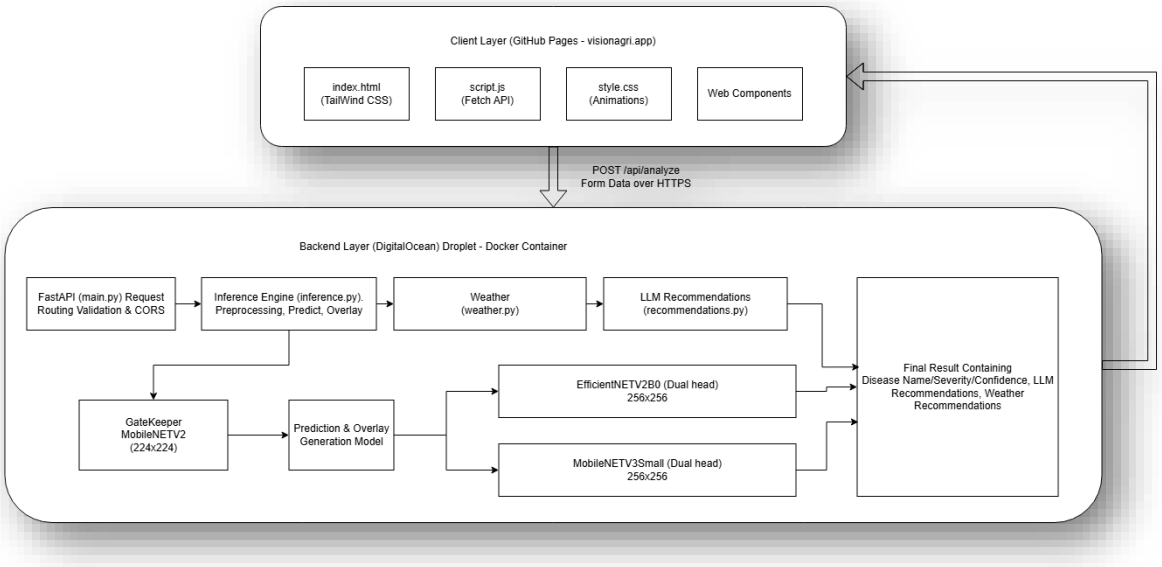


Figure 21 System architecture

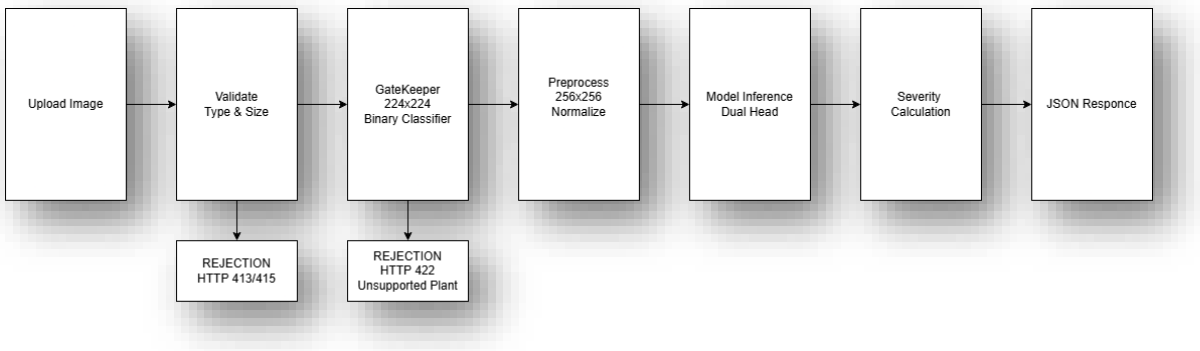


Figure 22 Inference pipeline

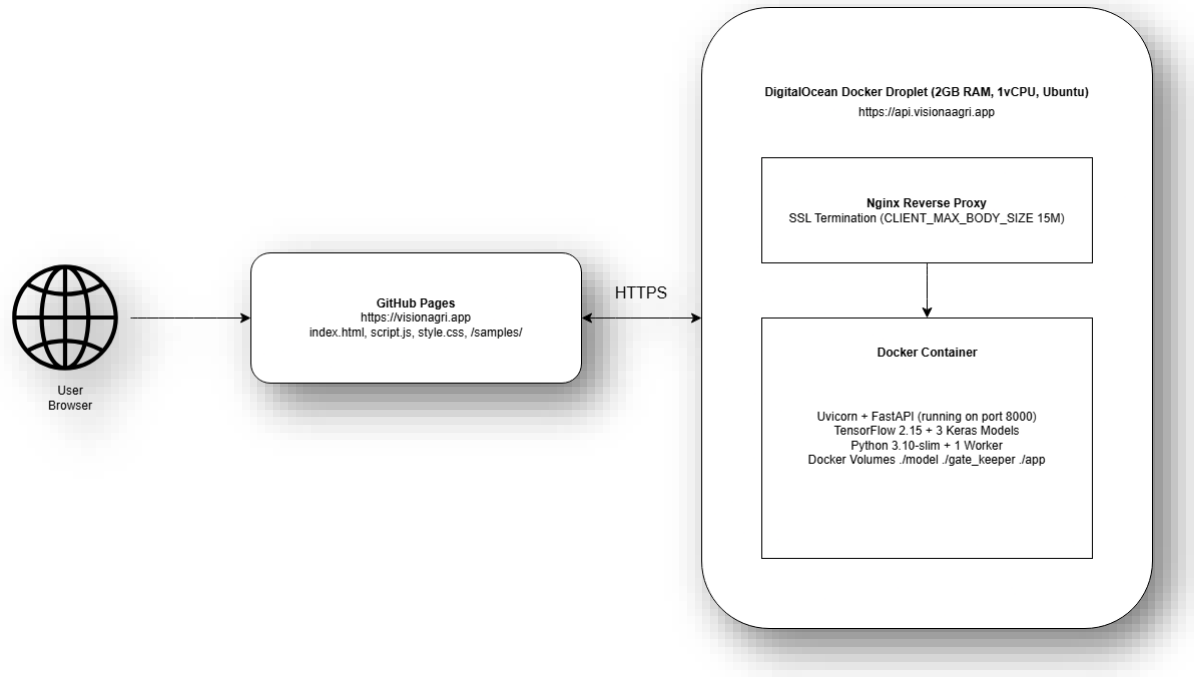


Figure 23 Deployment diagram

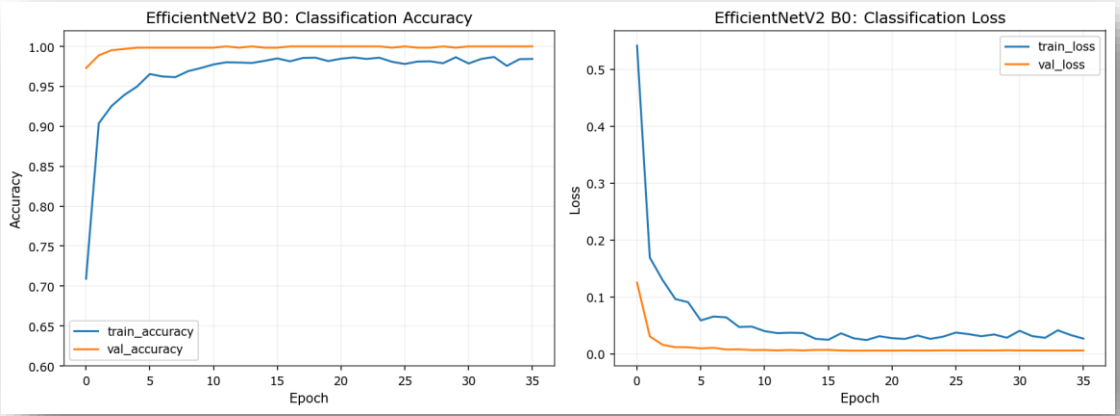


Figure 24 EfficintNetV2 B0 Classification accuracy/loss curves

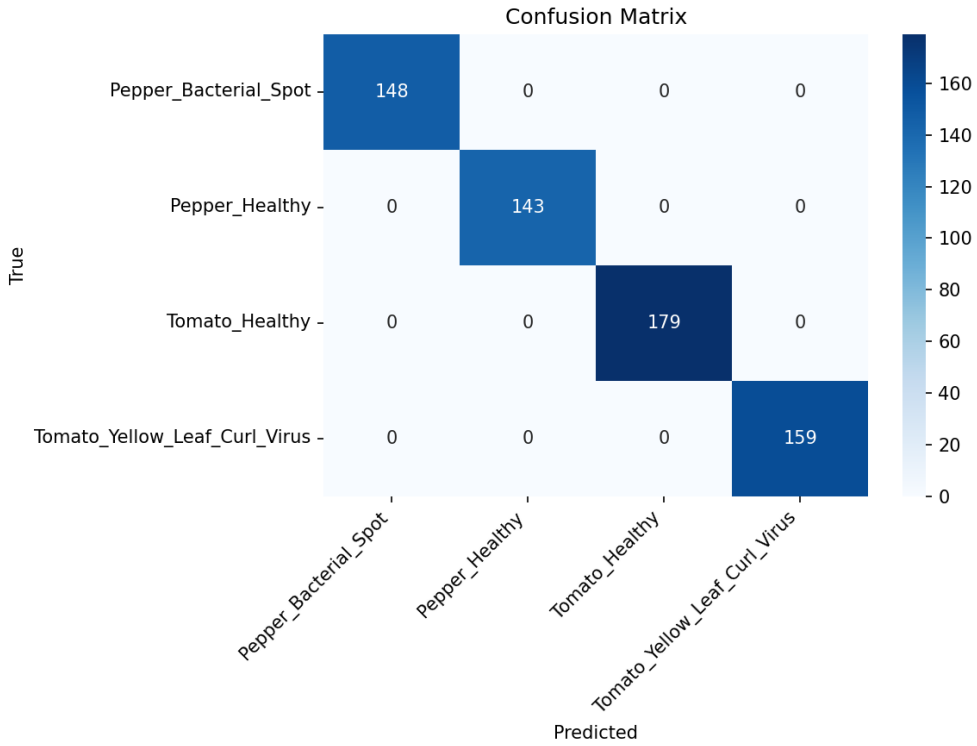


Figure 25 EfficientNetV2 B0 confusion matrix on test set

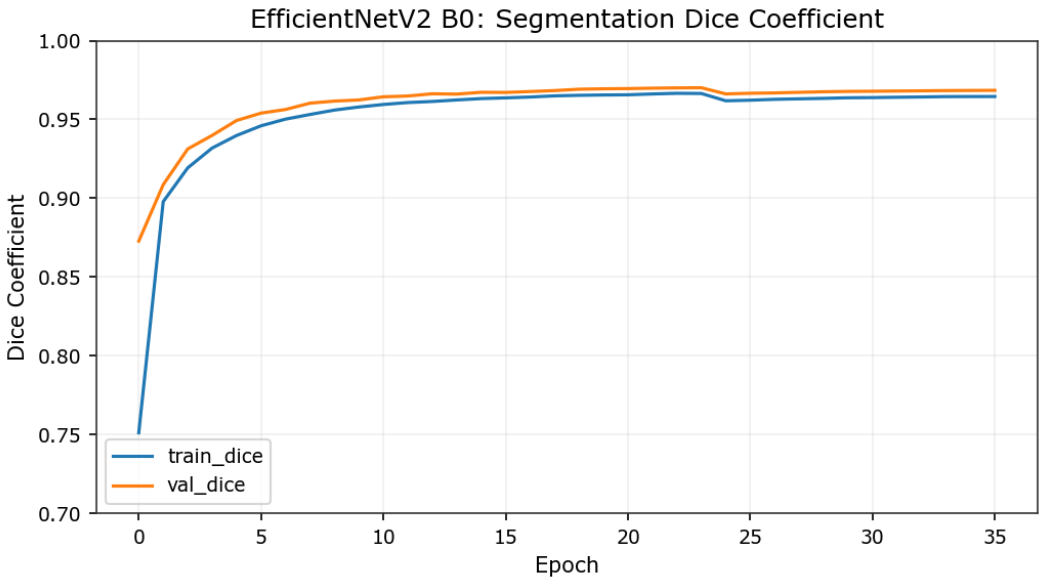


Figure 26 EfficientNetV2 B0 Dice coefficient

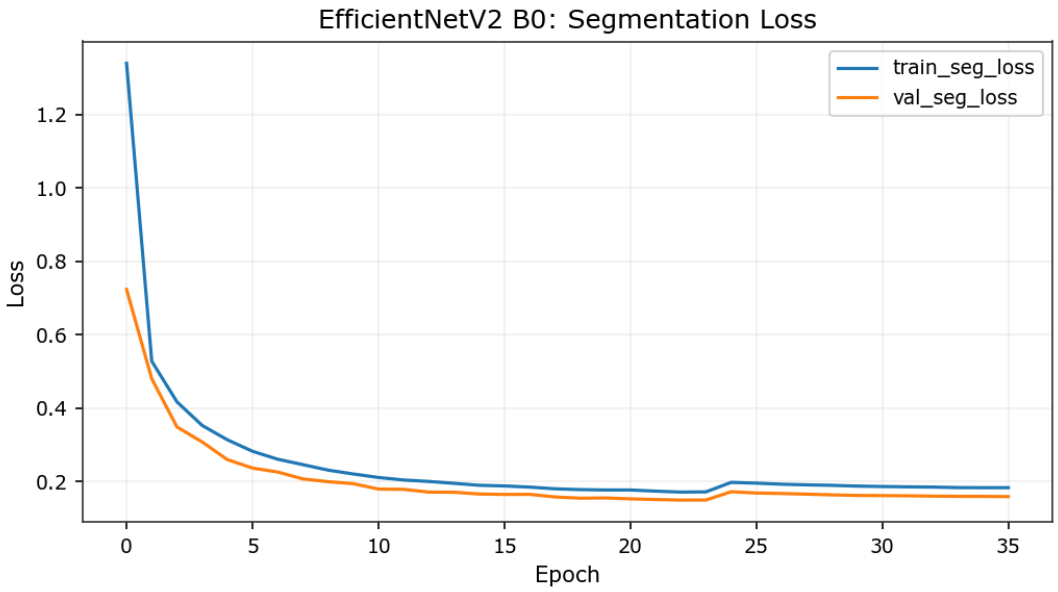


Figure 27 EfficientNetV2 B0 Segmentation Loss curve

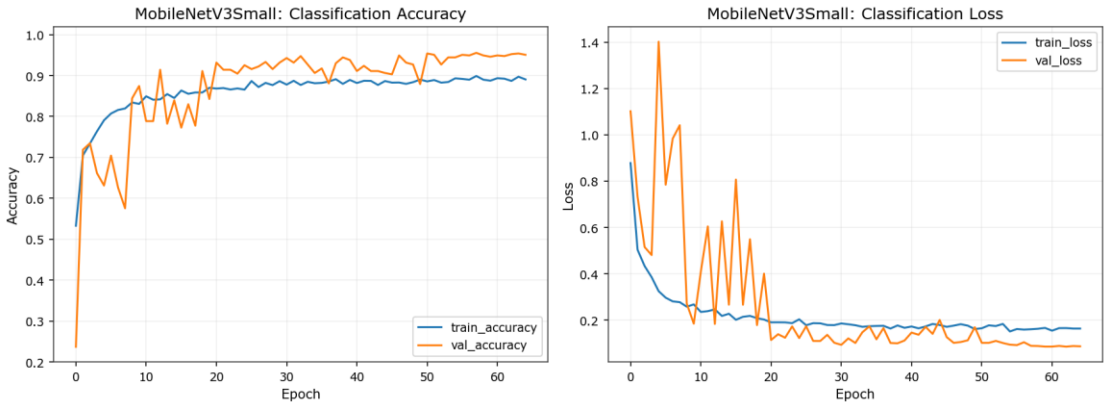


Figure 28 Mobilenetv3small classification accuracy/loss curve

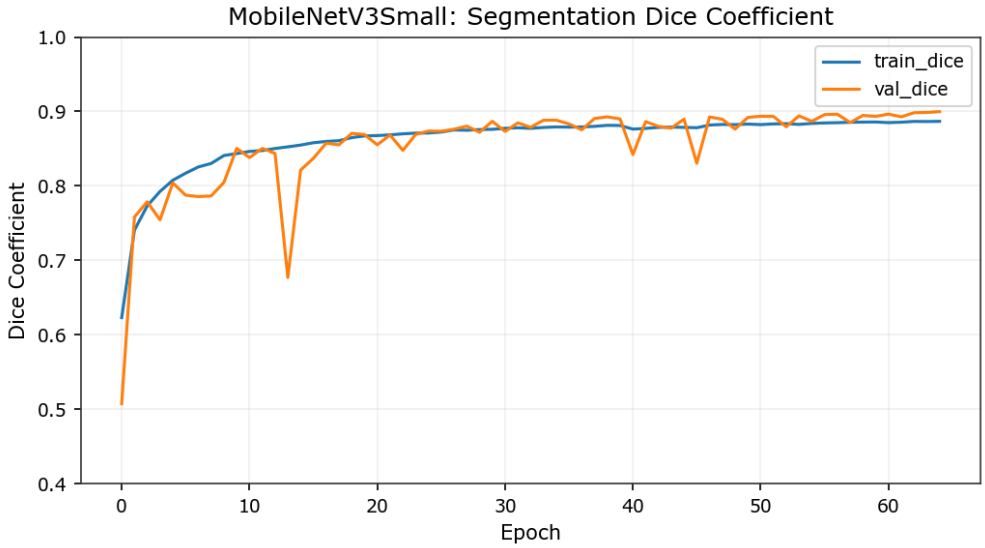


Figure 29 Mobilenetv3small segmentation dice coefficient

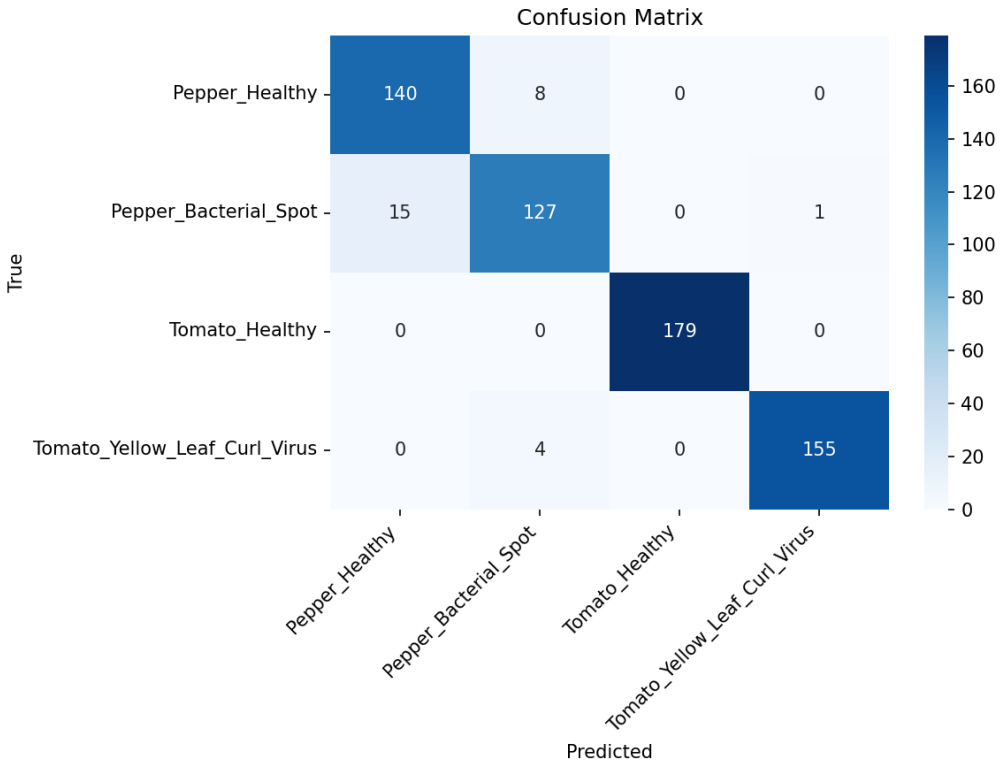


Figure 30 MobileNetV3Small Segmentation Dice Coefficient

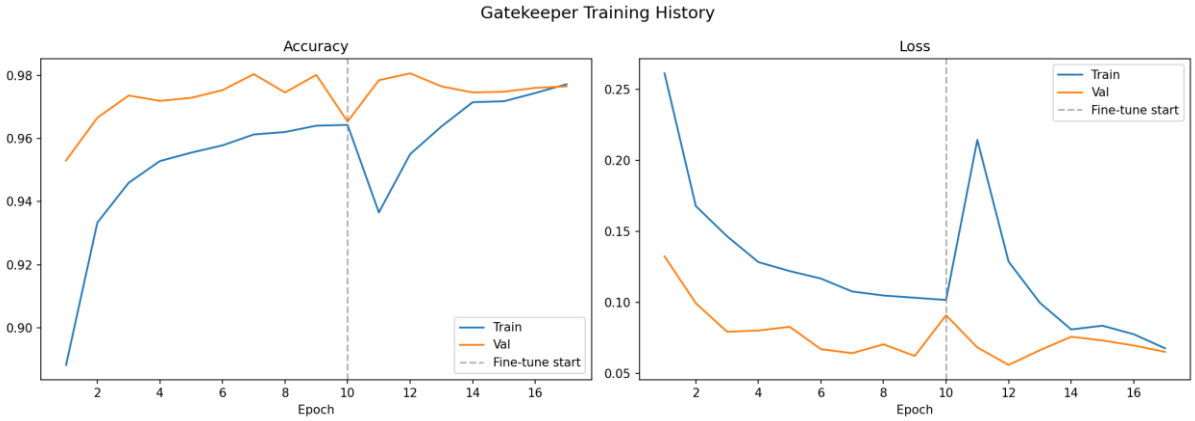


Figure 31 MobileNetV2 gatekeeper training accuracy and loss curve

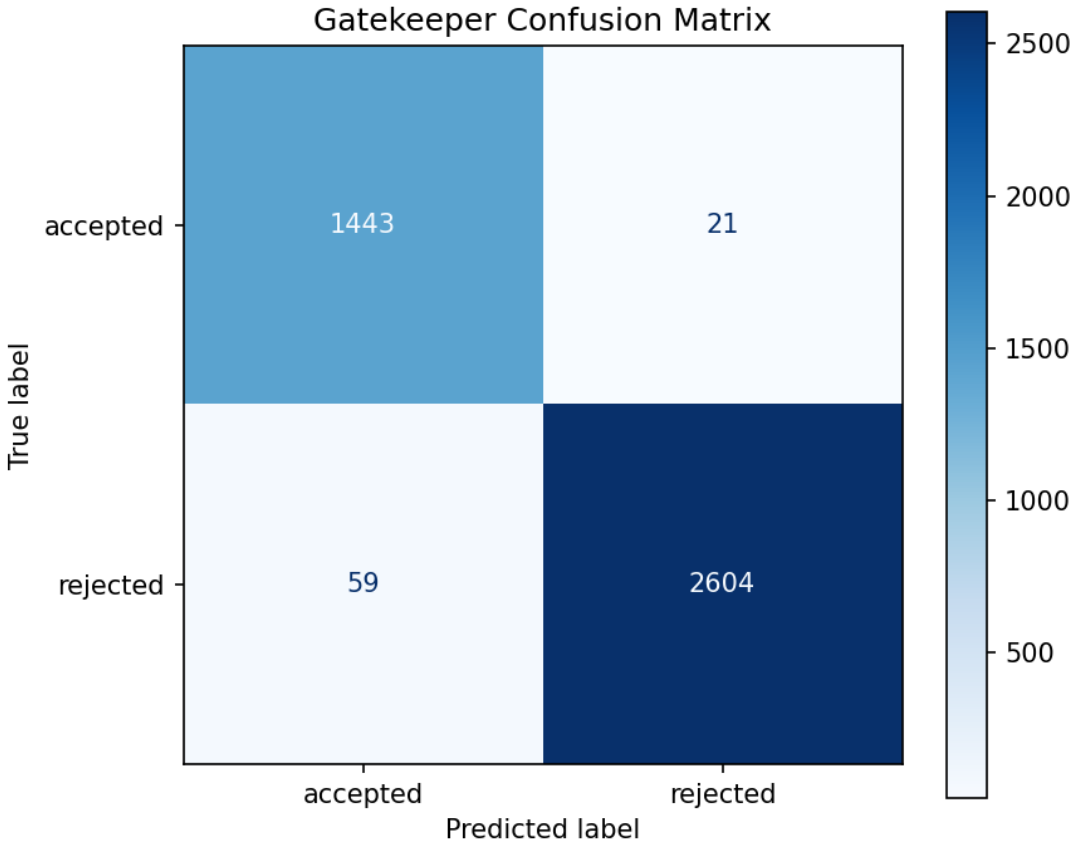


Figure 32 Gatekeeper binary classifier confusion matrix

Figure 33 Side by side validation comparison of Mobilenetv3small and efficientnetv2 b0

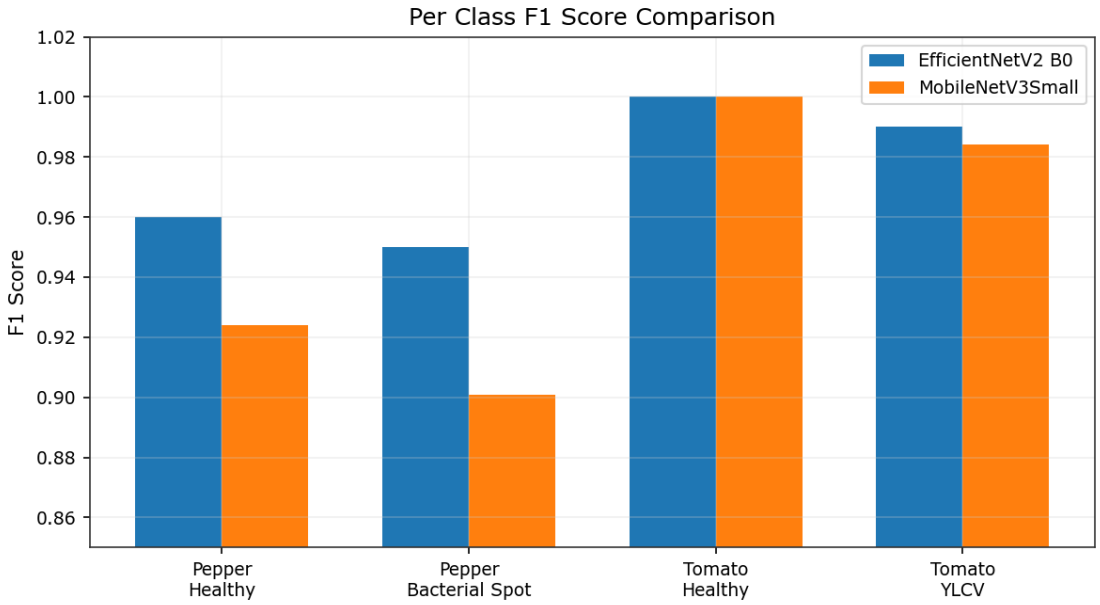


Figure 34 F1 Score comparison of two models

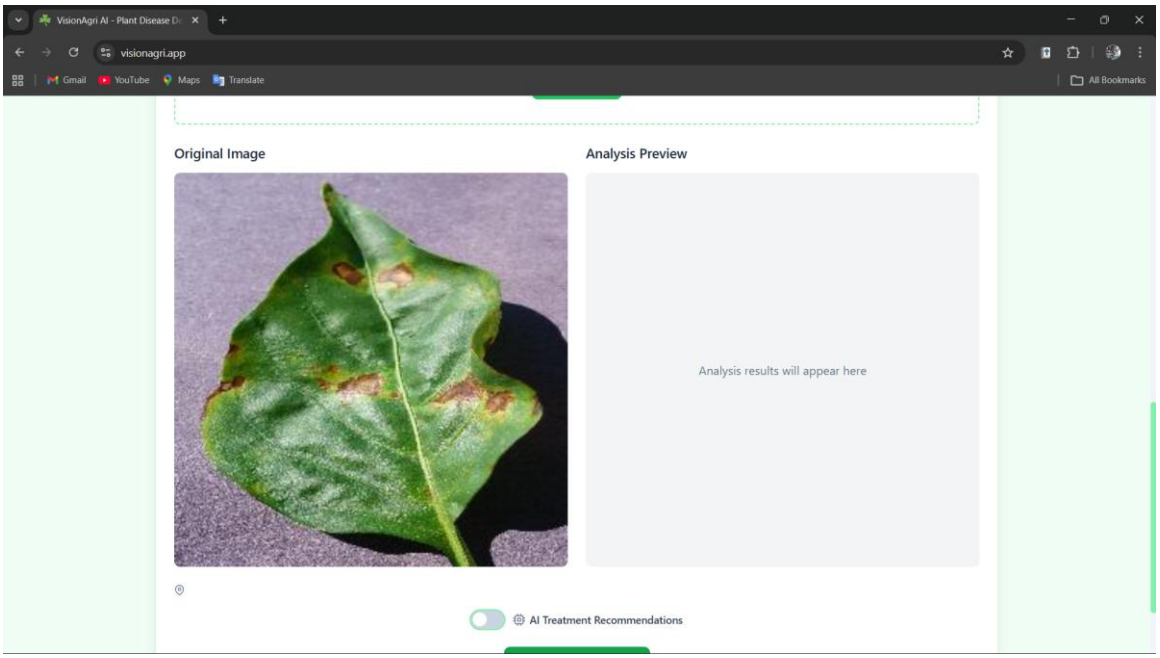


Figure 35 Image upload via file picker.

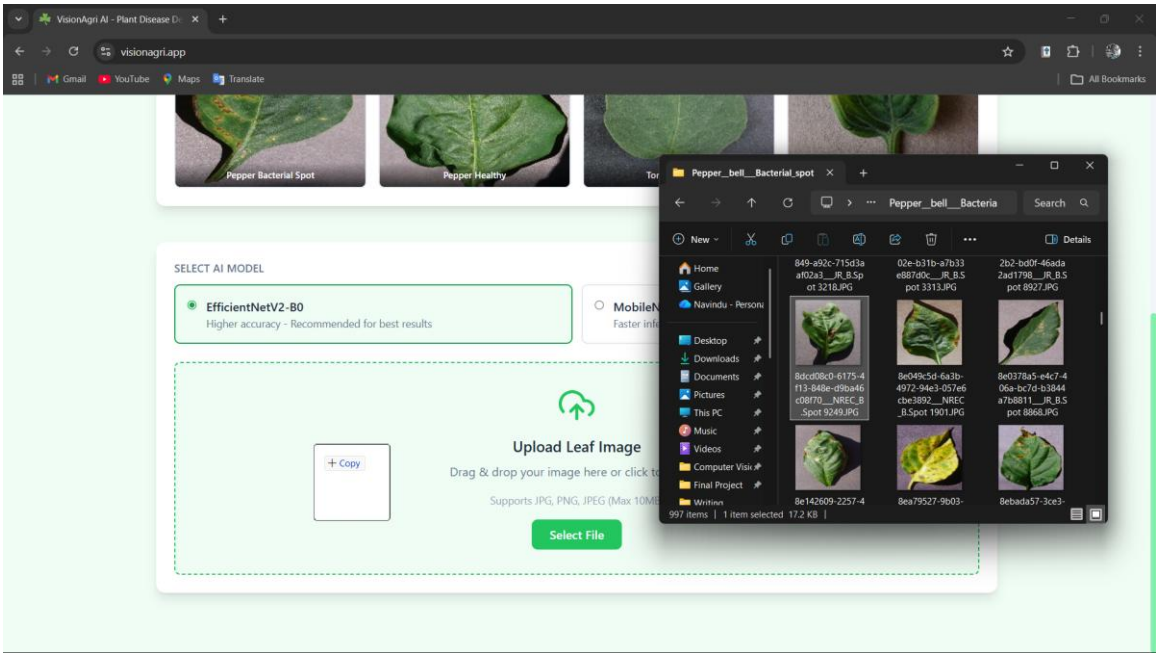


Figure 36 Image upload via drag and drop

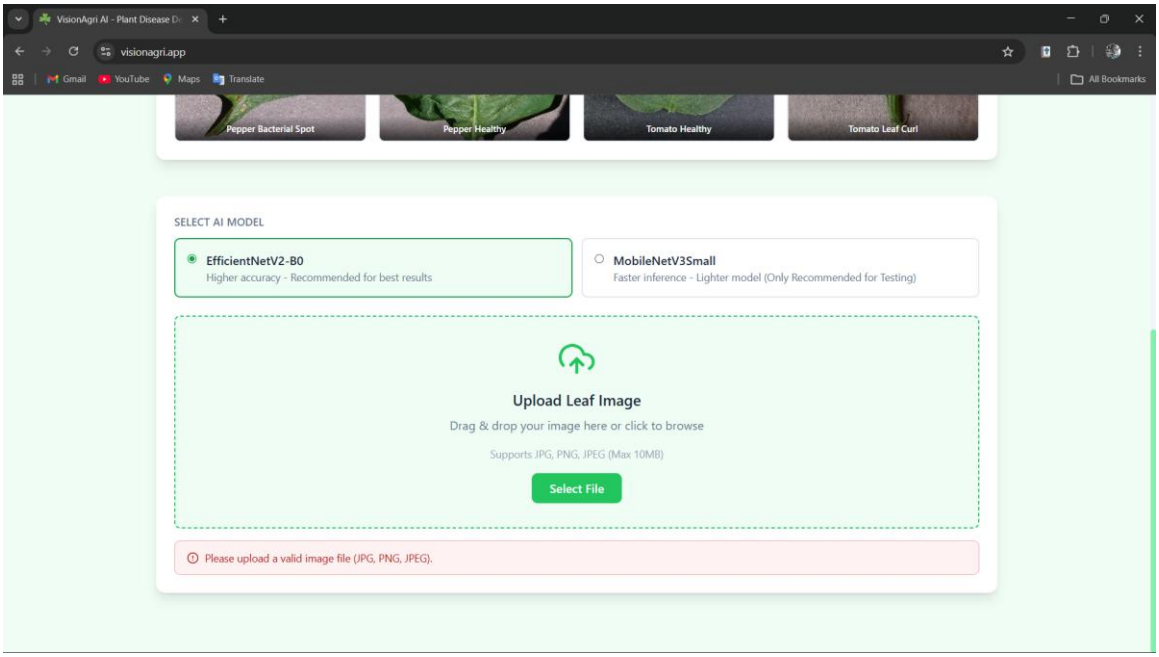


Figure 37 Invalid file type rejection

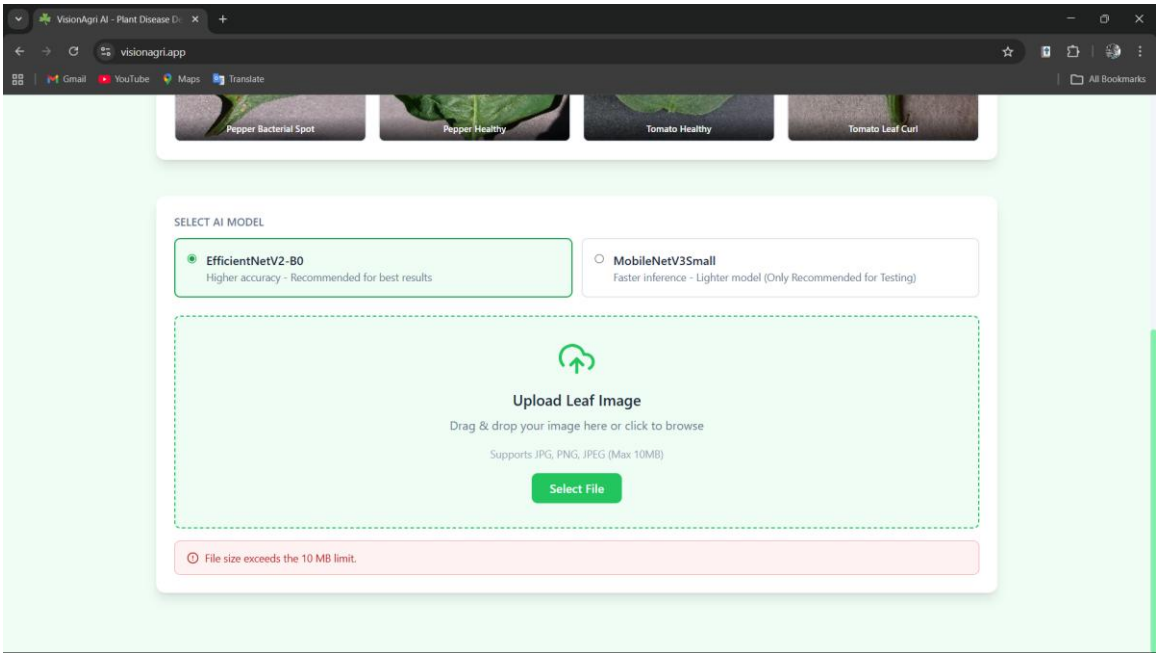


Figure 38 Oversized file rejection

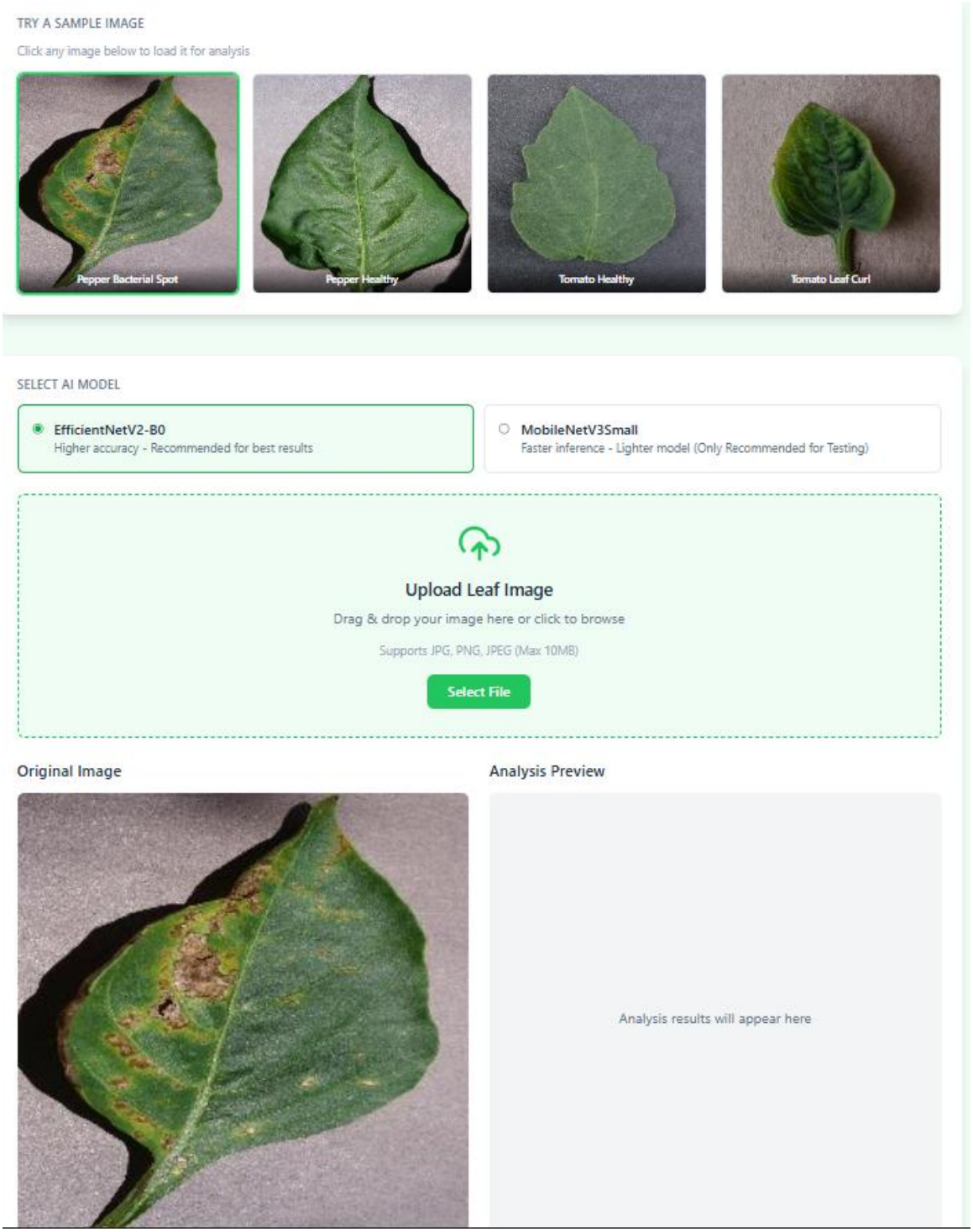


Figure 39 Sample image selection

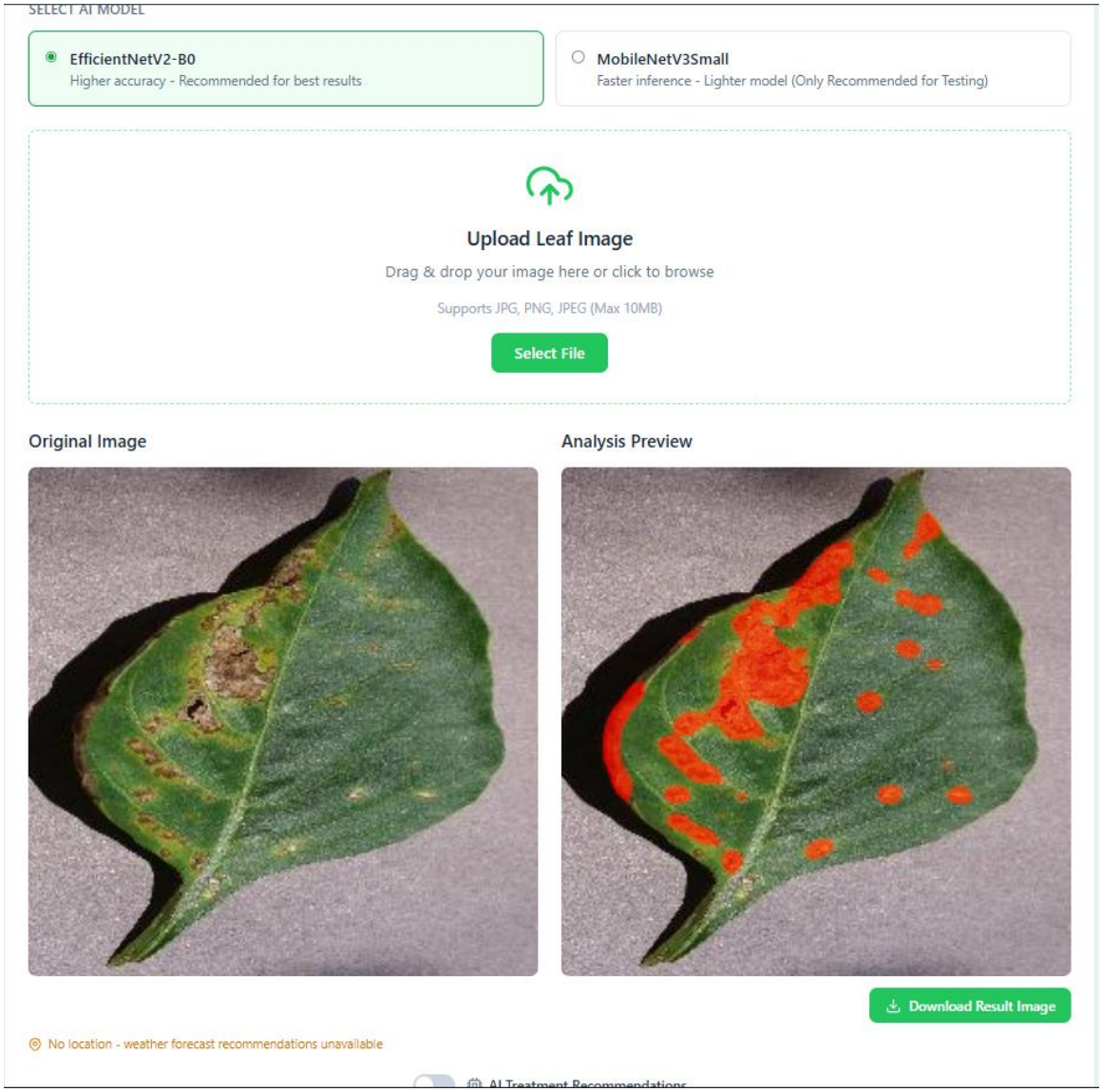


Figure 40 EfficientNetV2 B0 model selection and prediction 1

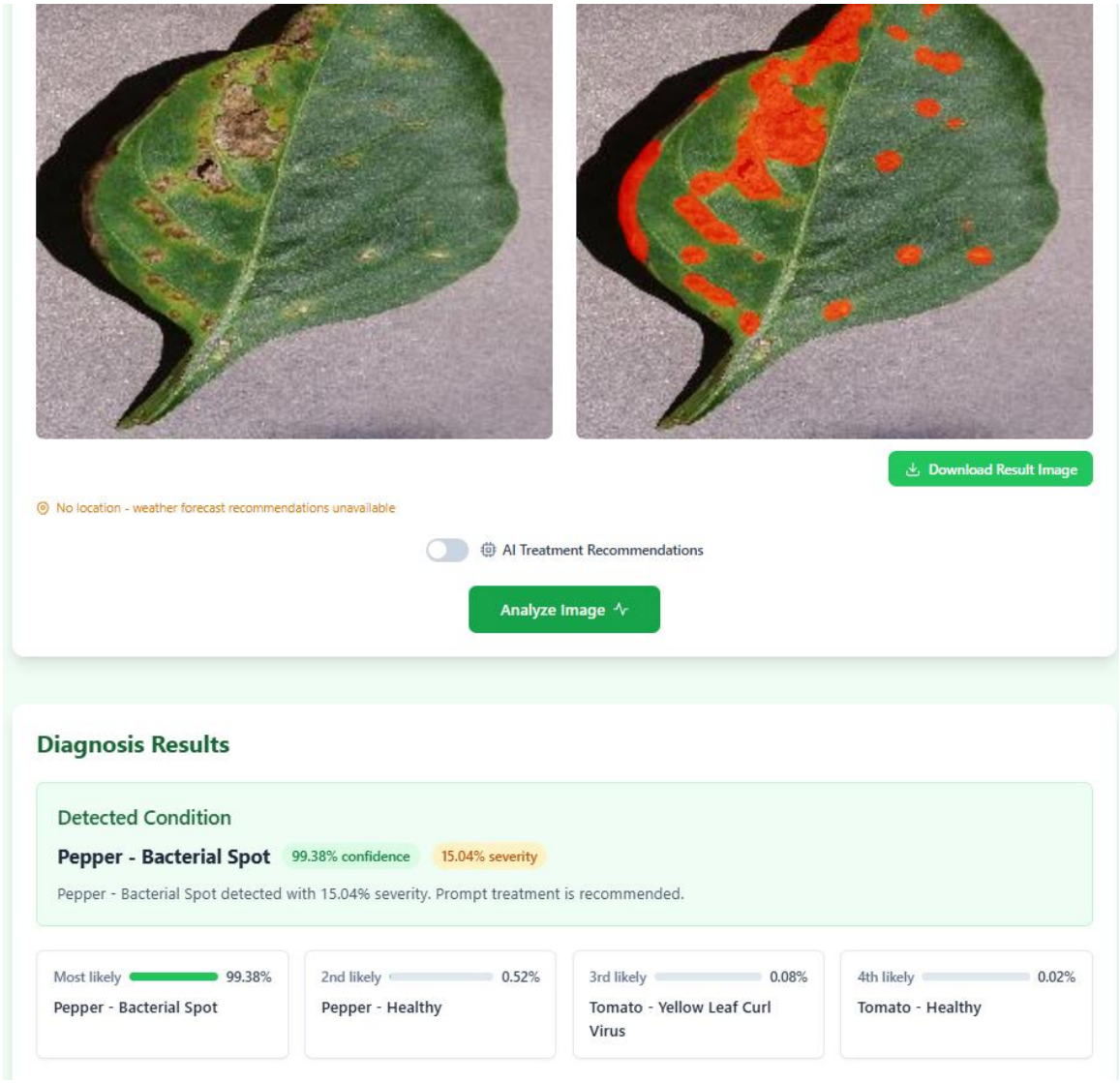


Figure 41 EfficientNetV2 B0 model selection and prediction 2

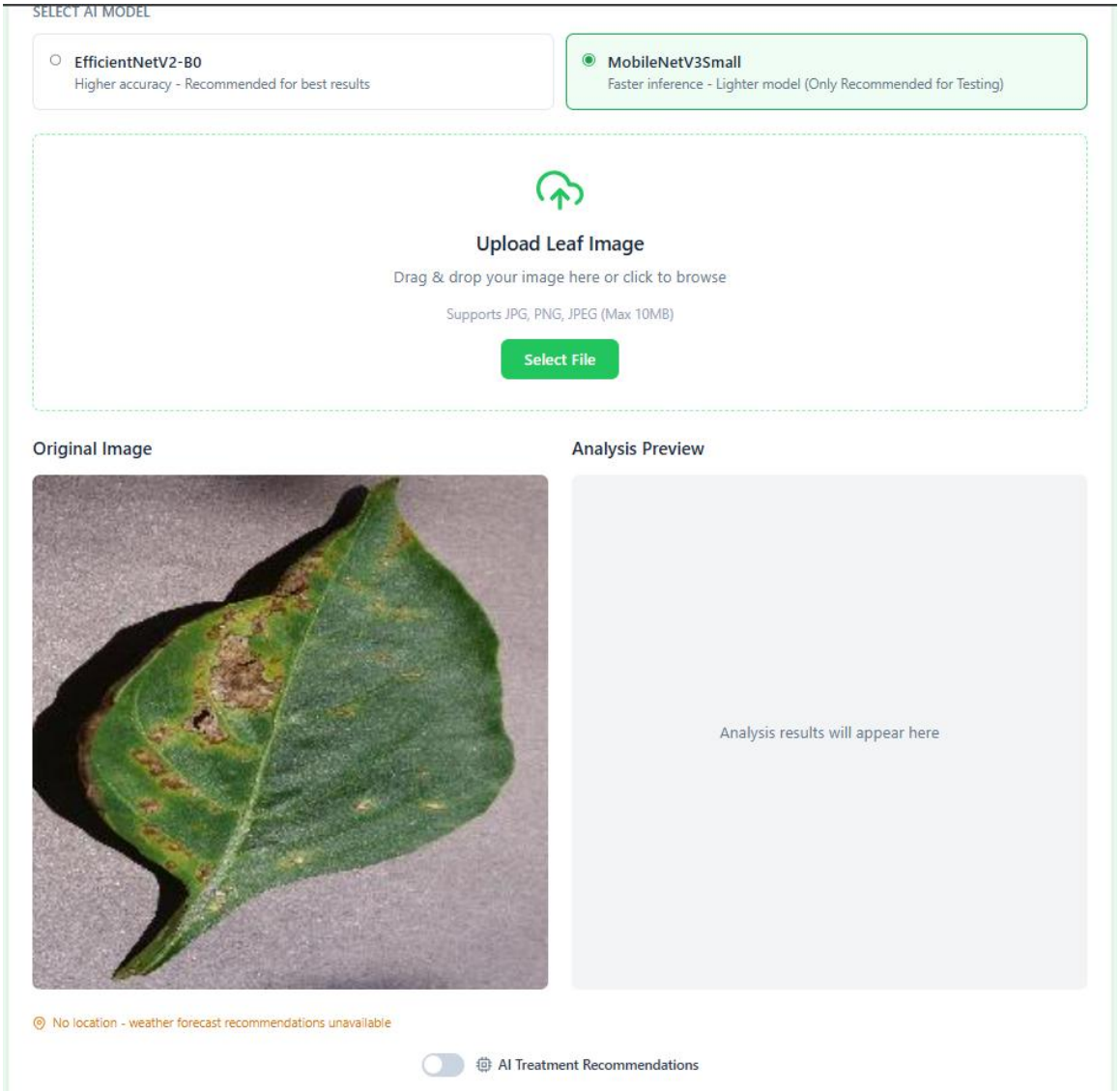


Figure 42 MobileNetV3Small model selection and making predictions 1

Figure 43 Result of MobileNetV3Small, result is more different from the EfficientNetV2 B0

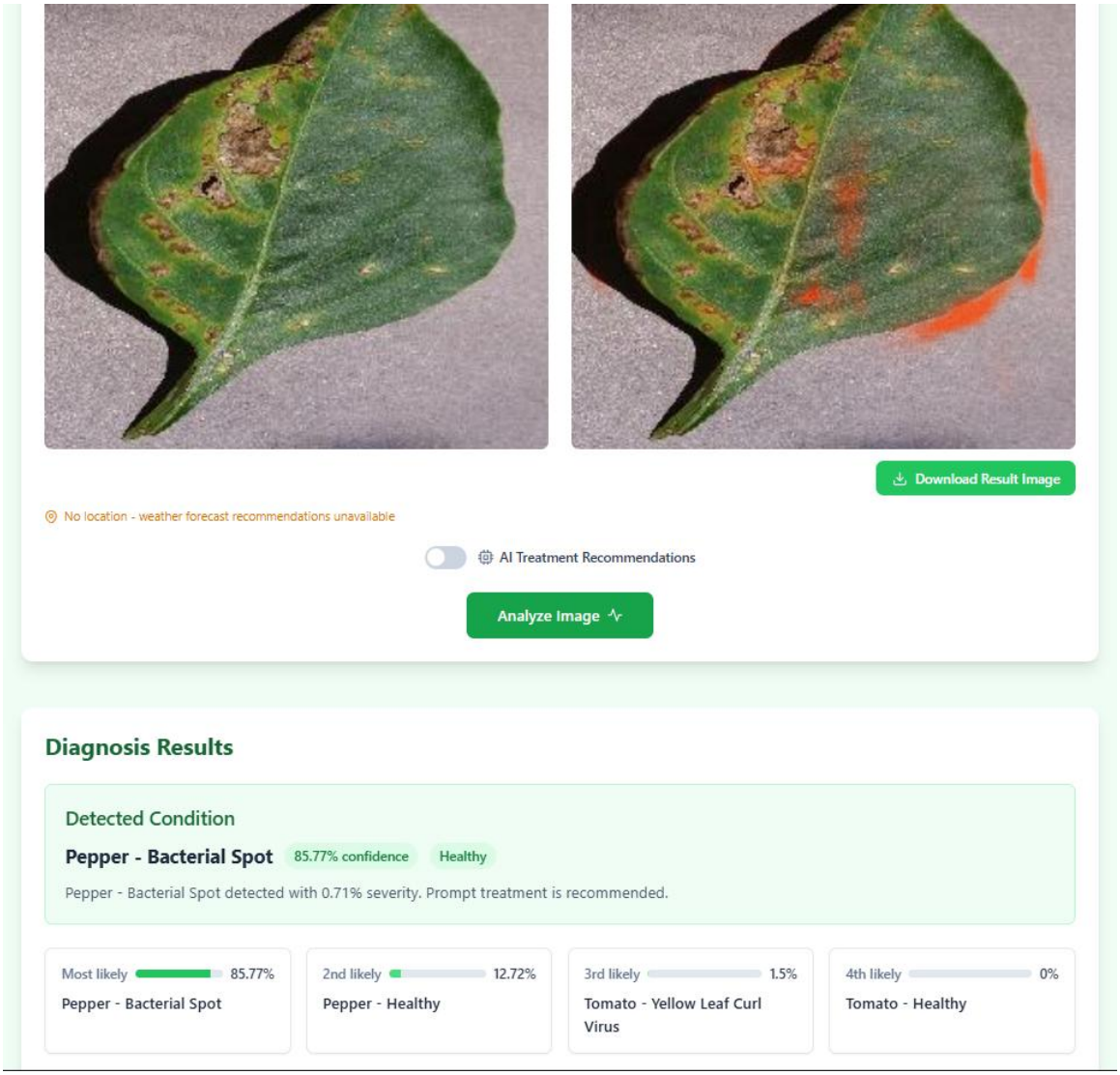


Figure 44 Though, the MobileNetV3Small has classified the disease correctly, the accuracy is lower than EfficientNetV2 B0

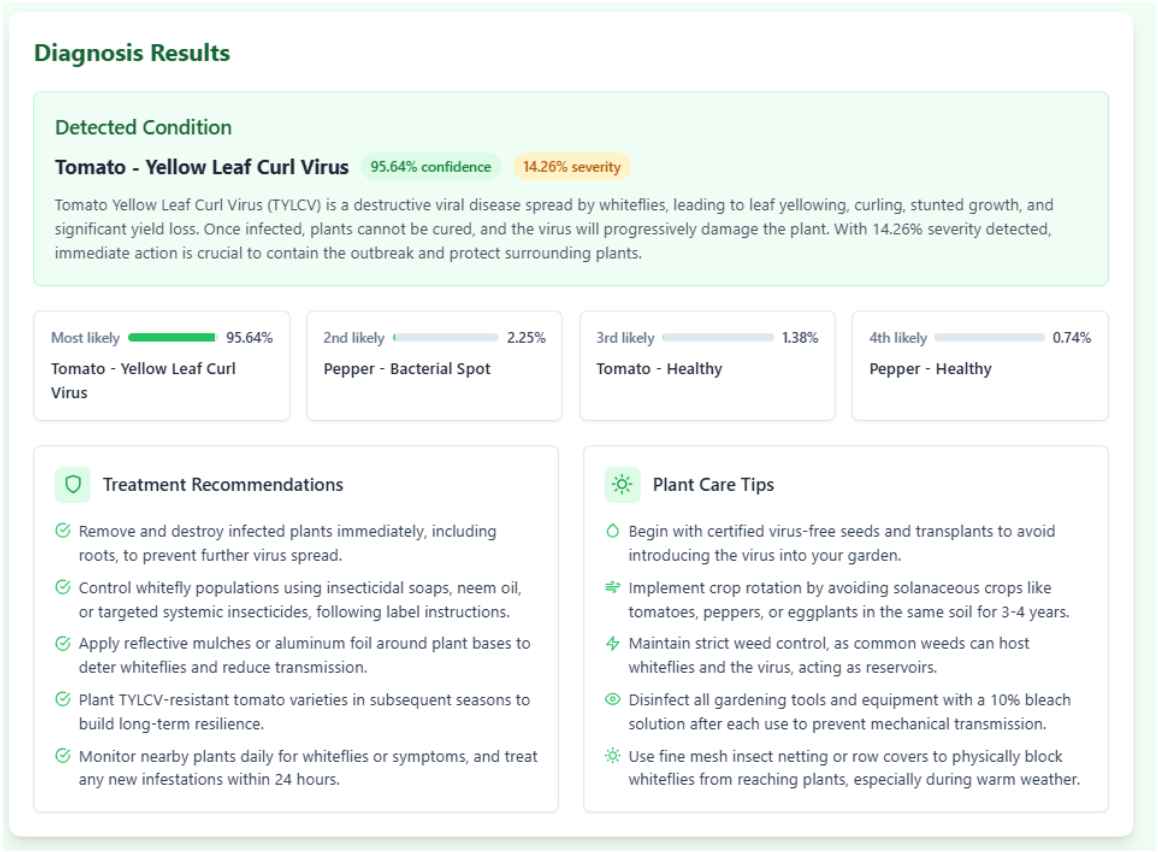


Figure 45 AI recommendations toggle ON

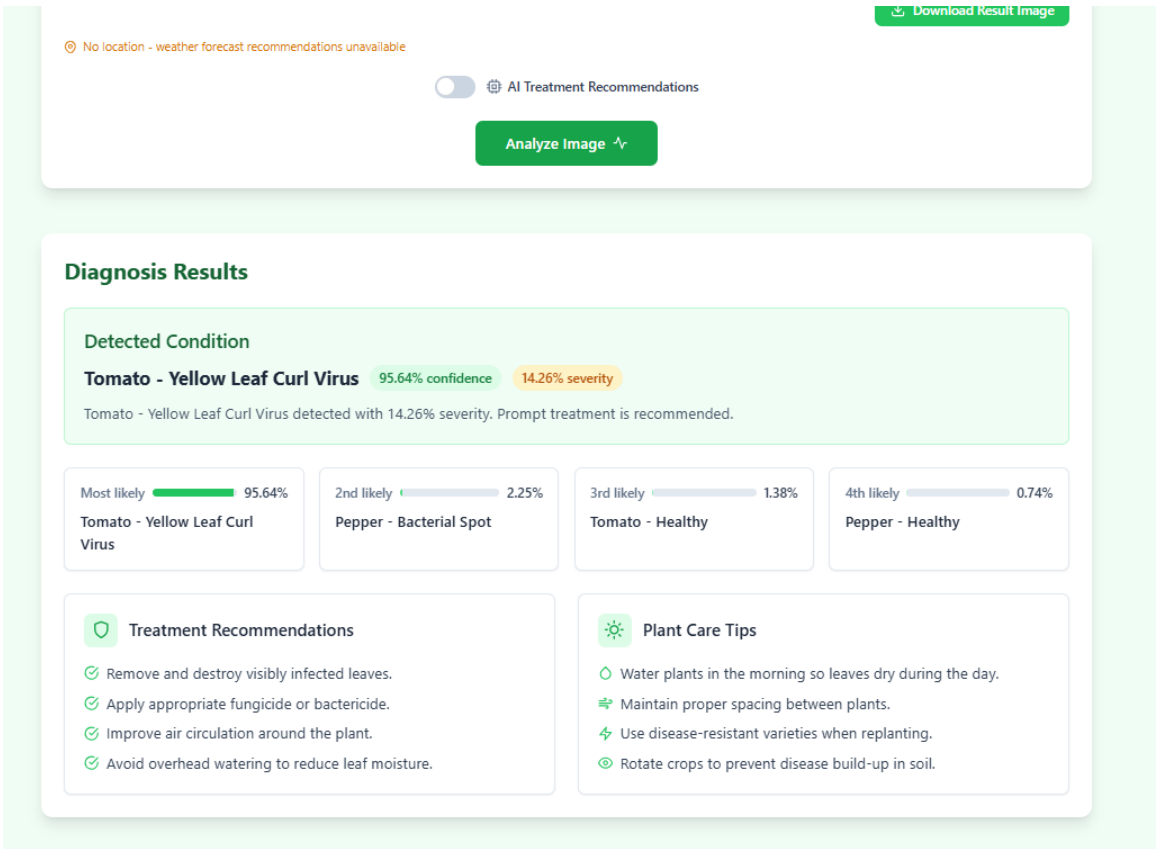


Figure 46 AI recommendations toggle off.

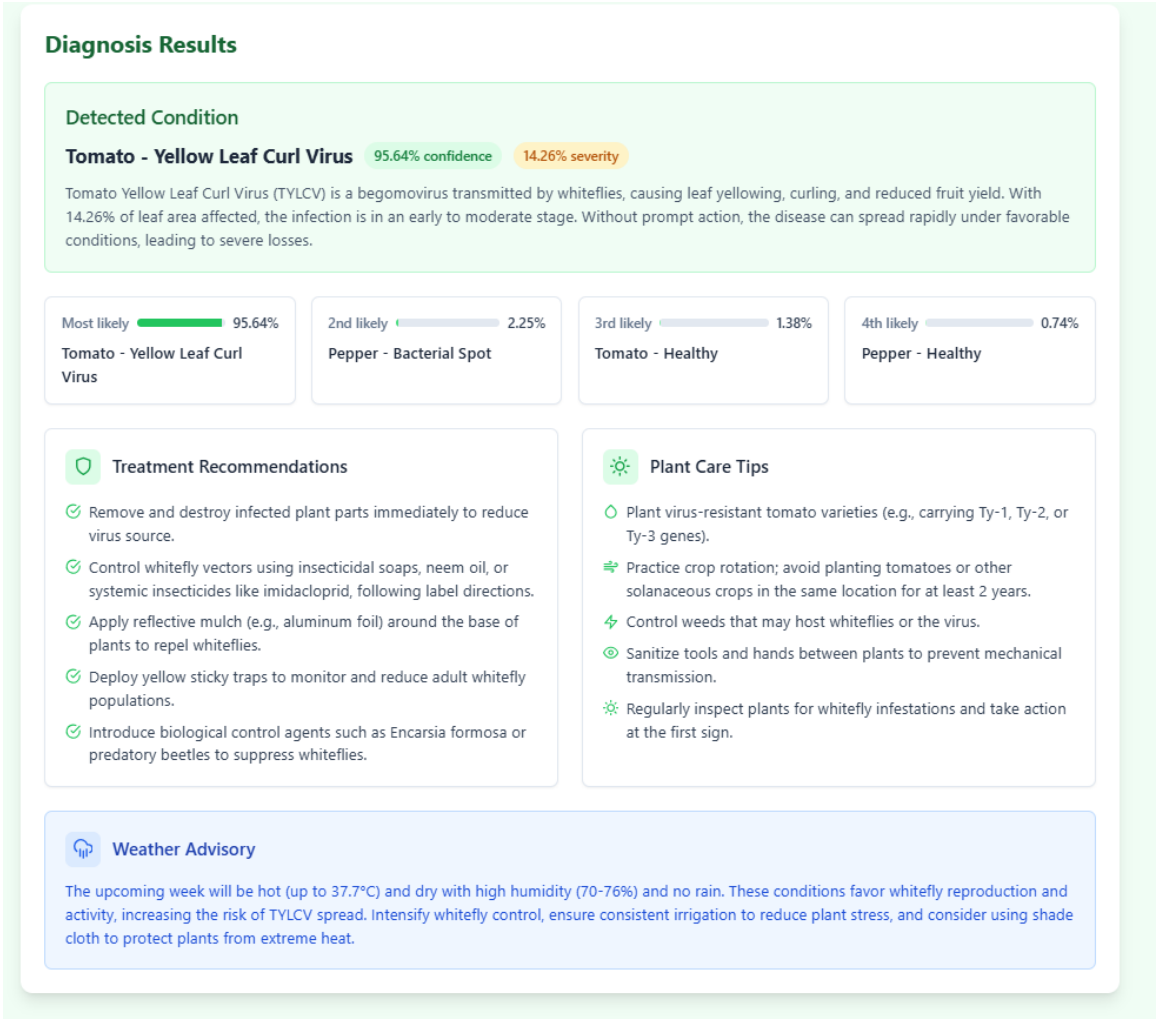


Figure 47 Geolocation based weather AI recommendations

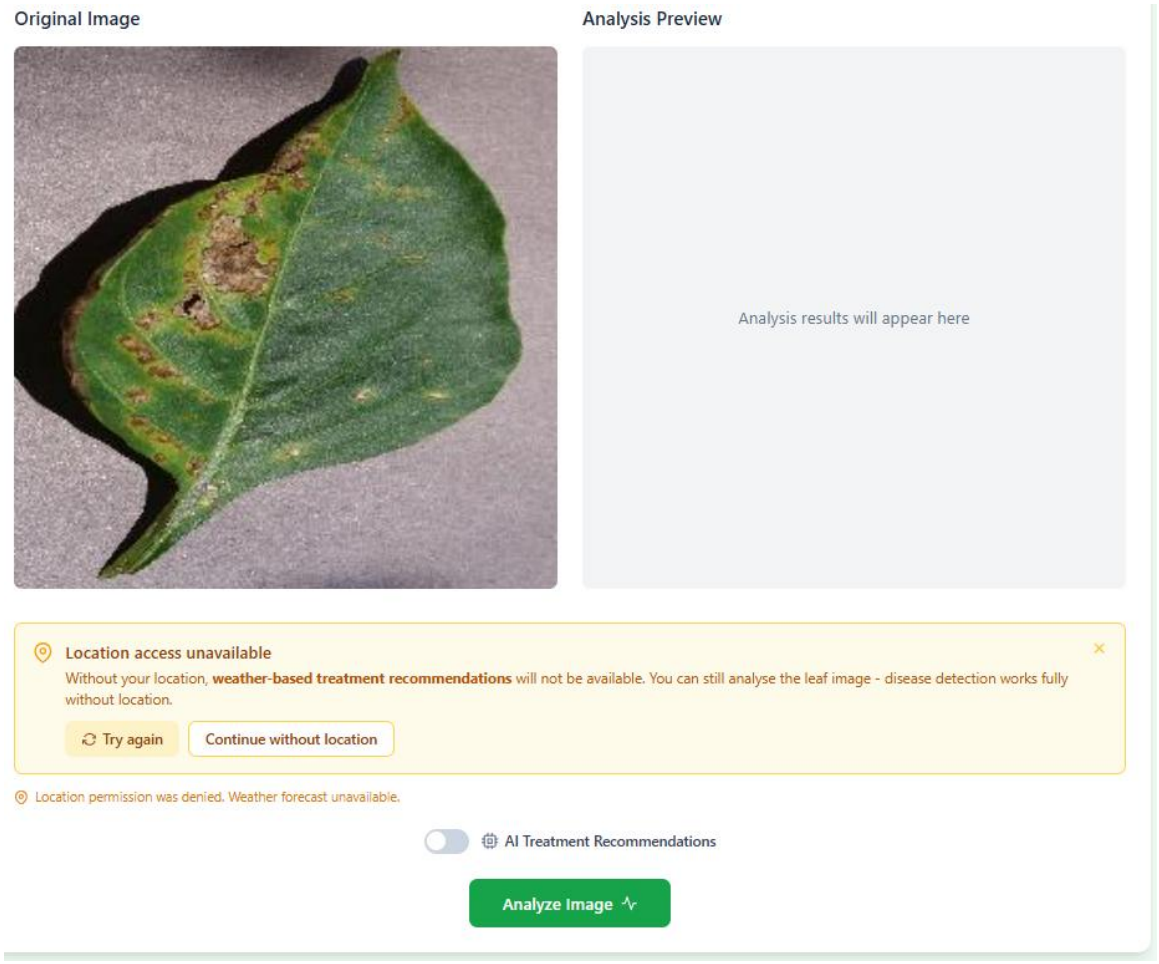


Figure 48 Location service denied.

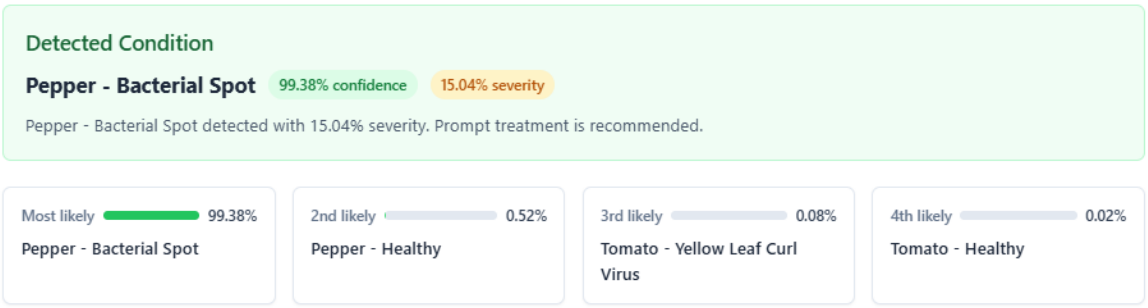


Figure 49 Score distribution bars

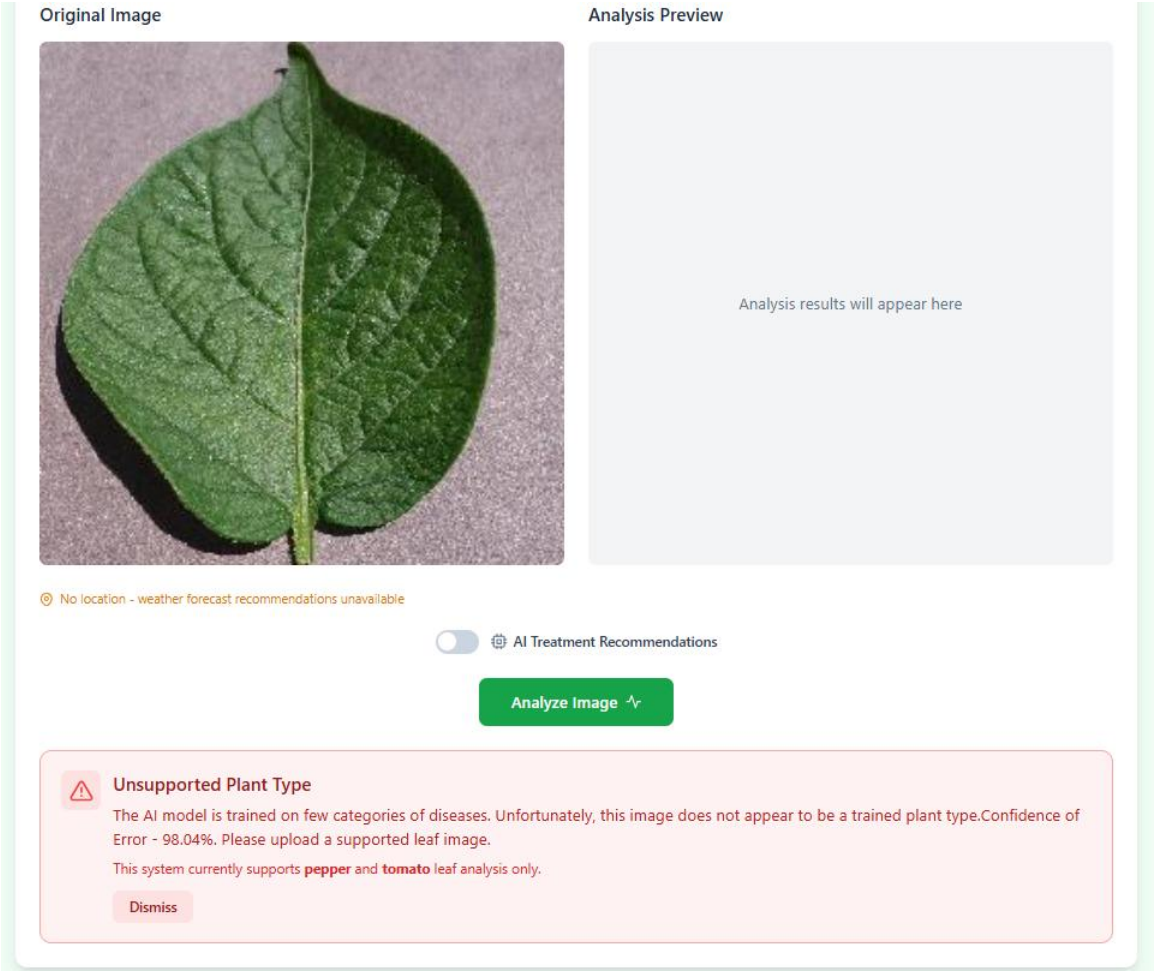


Figure 50 Gatekeeper has successfully rejected a "Potato Healthy" image with 98.04% confidence.

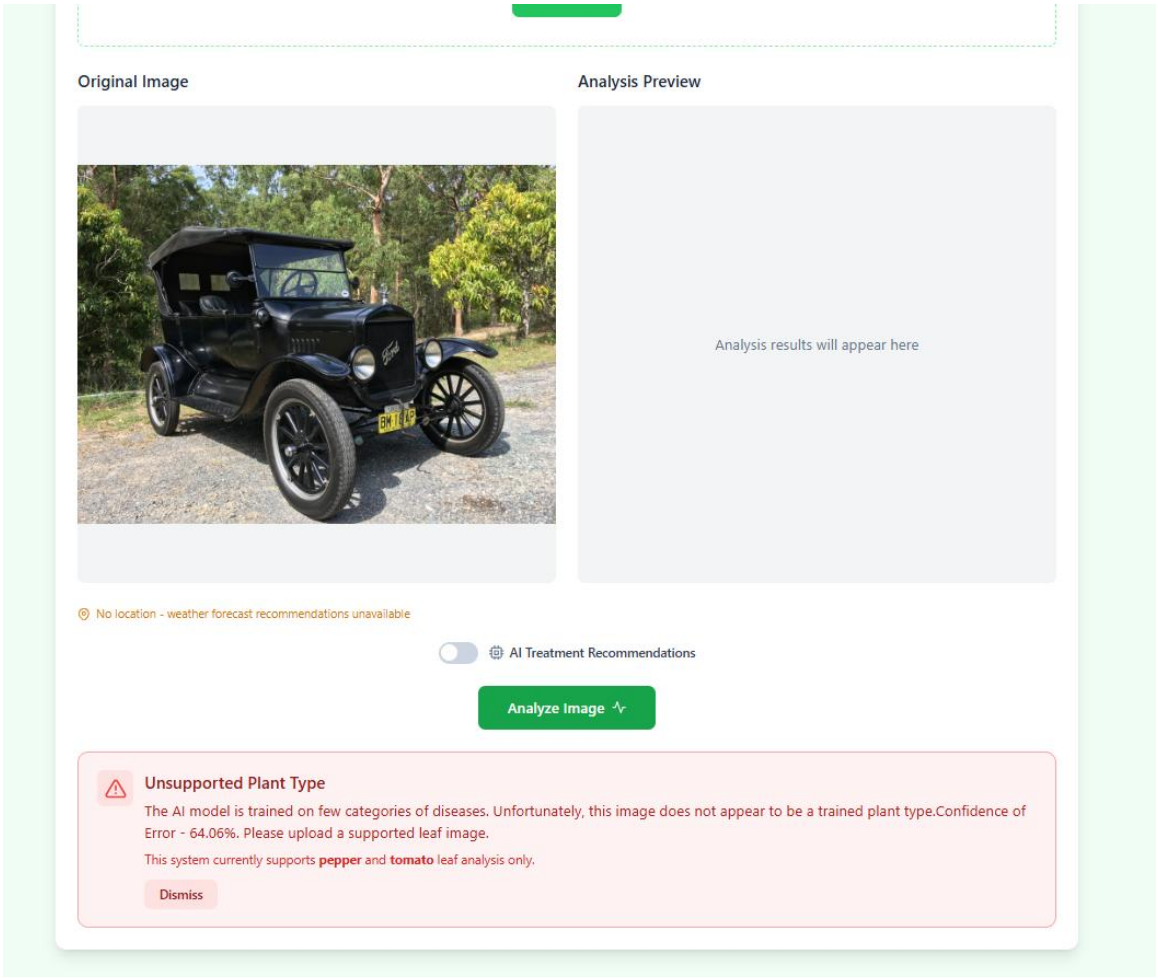


Figure 51 Low confidence image handling.

Figure 52 Disease segmentation overlay display

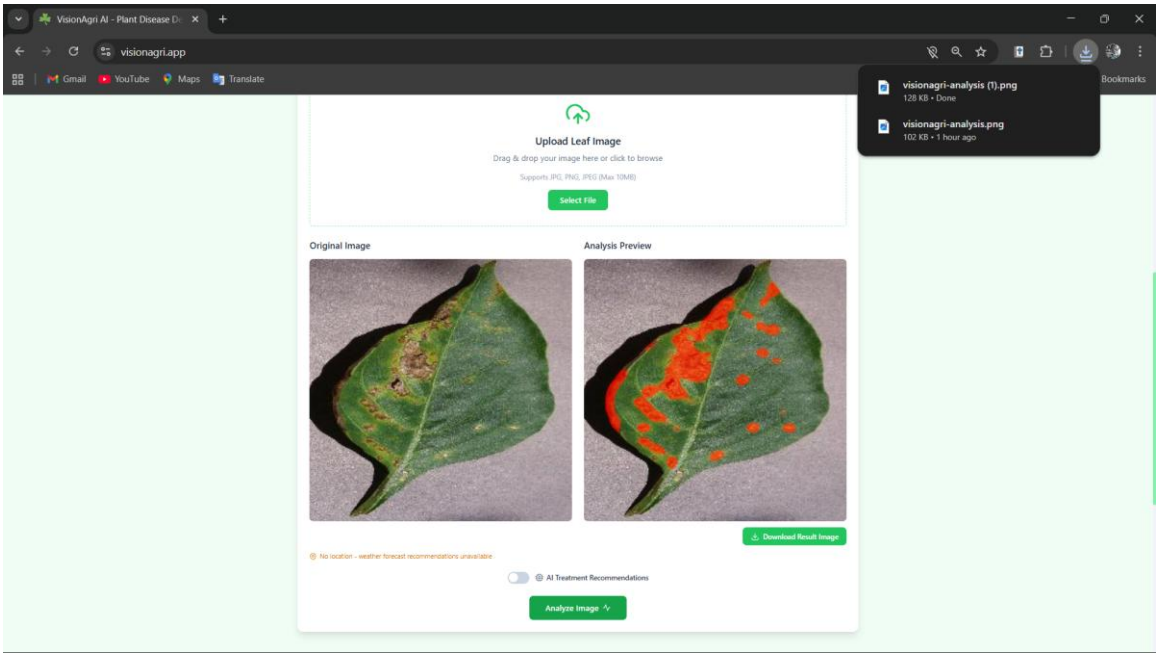


Figure 53 Analyzed image downloaded

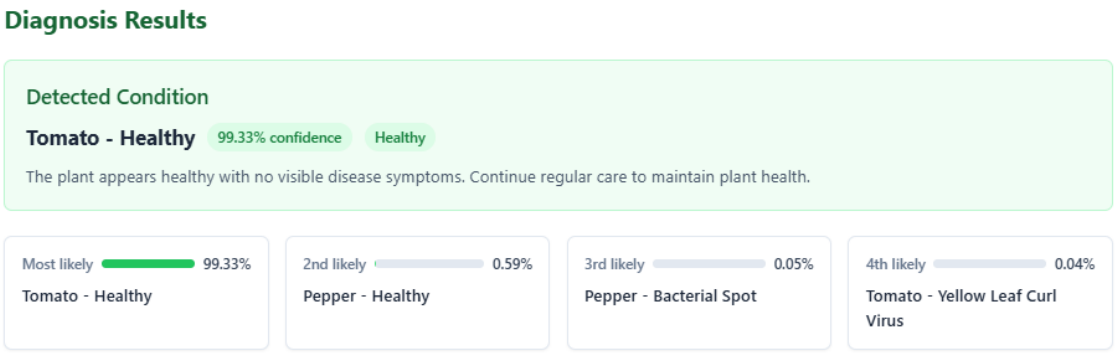


Figure 54 Healthy plant detection

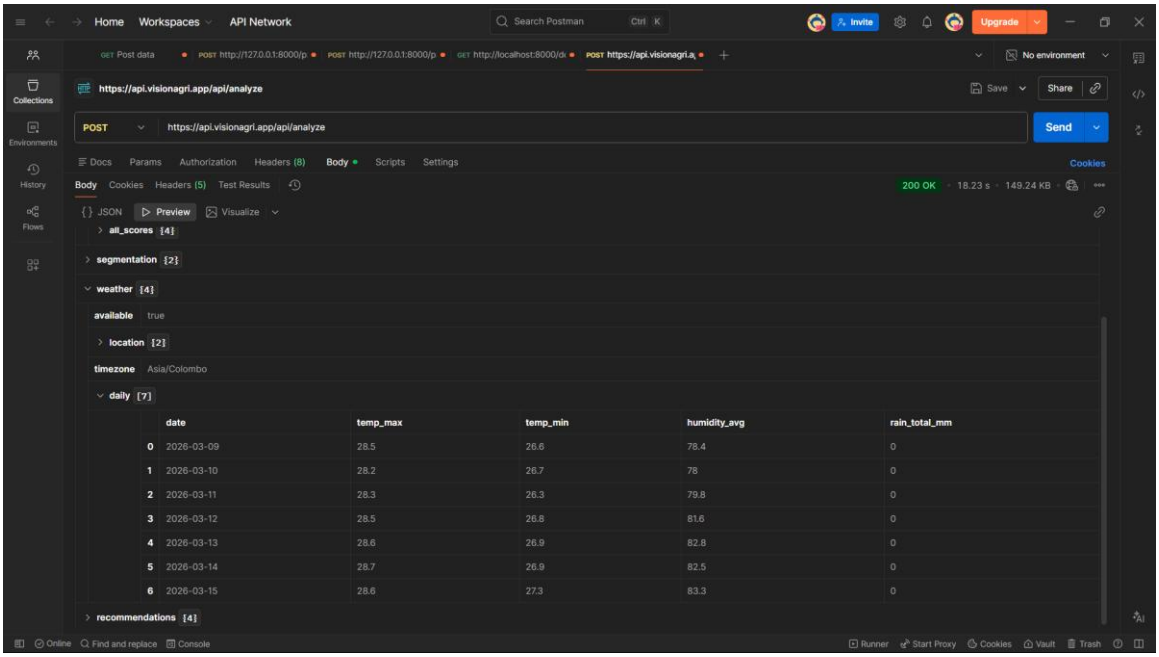


Figure 55 7-day weather data returned.

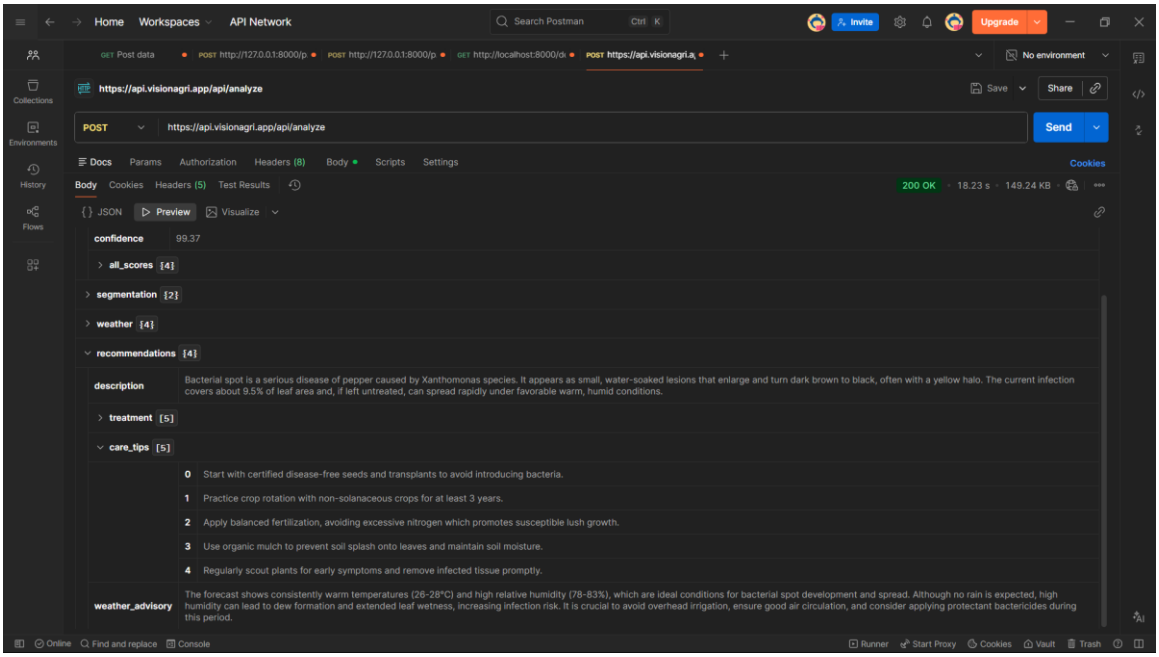


Figure 56 care tips JSON

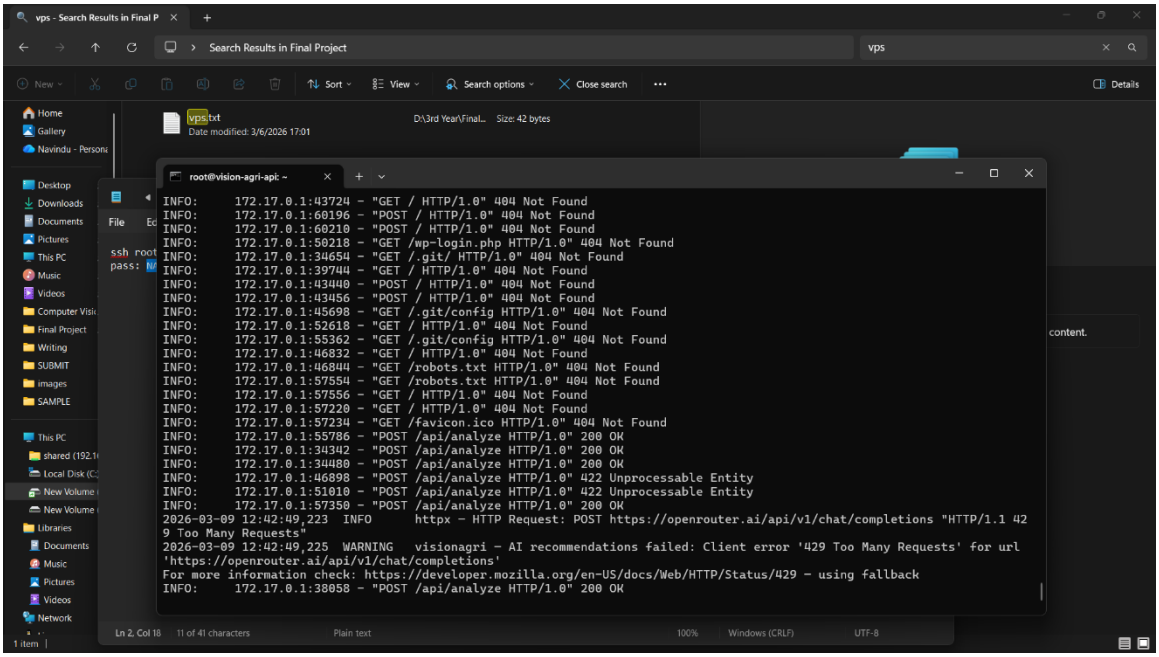


Figure 57 API key failed, gracefully returned an error.

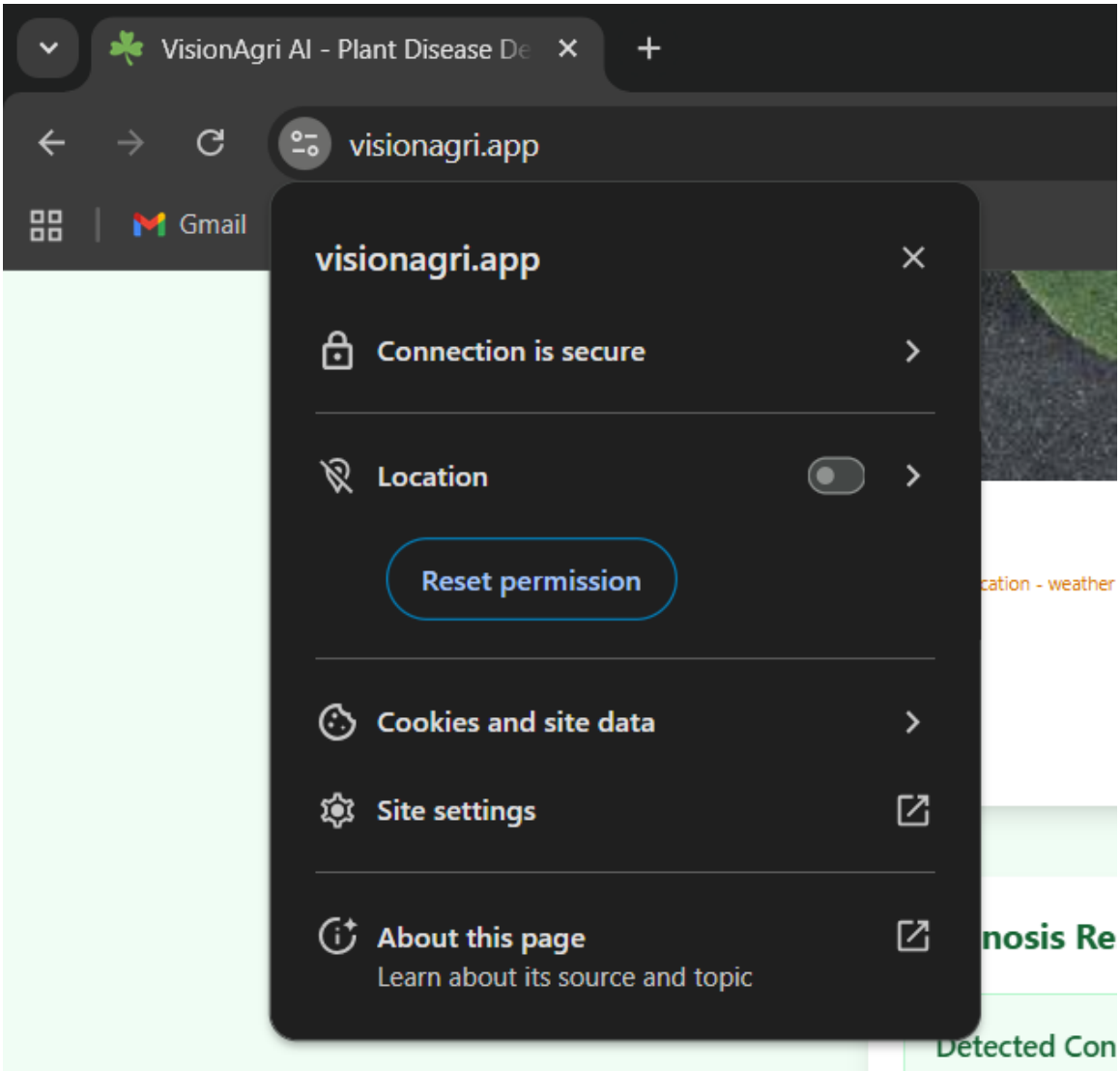


Figure 58 Site is SSL secured

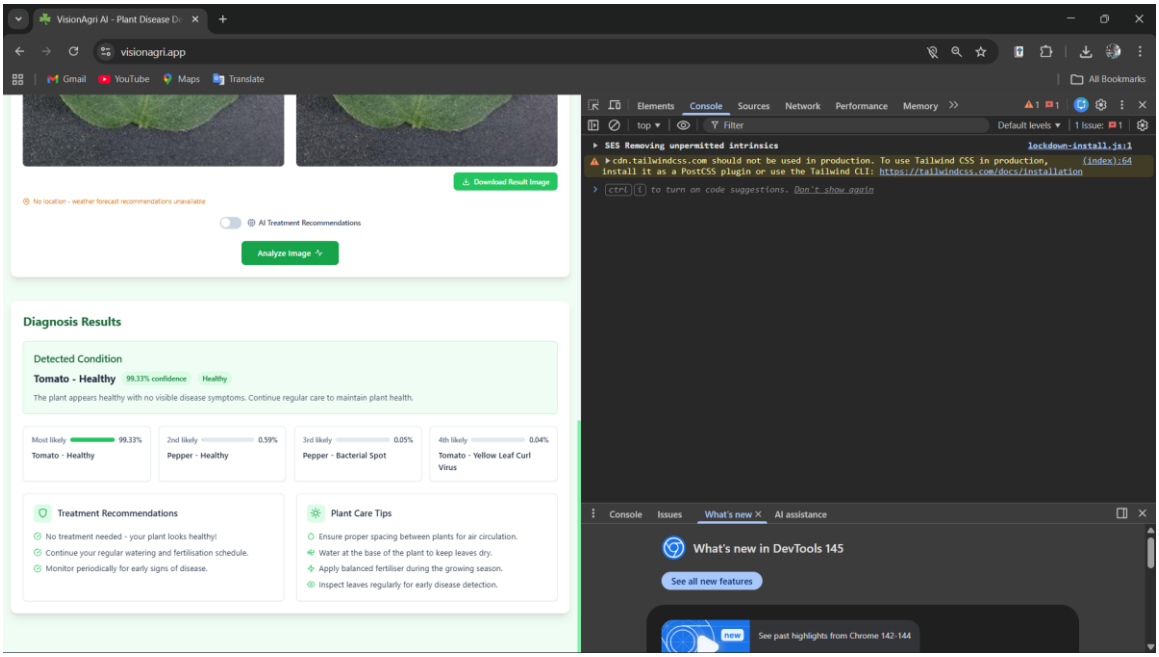


Figure 59 No CORS errors

```
INFO:     172.17.0.1:41634 - "GET /api/analyze HTTP/1.0" 405 Method Not Allowed
2026-03-09 13:56:01,466  INFO      httpx - HTTP Request: GET https://api.open-meteo.com/v1/forecast?latitude=6.93&longit
ude=79.85&hourly=temperature_2m%2Crelative_humidity_2m%2Crain&models=best_match&timezone=auto "HTTP/1.1 200 OK"
2026-03-09 13:56:04,138  INFO      httpx - HTTP Request: POST https://openrouter.ai/api/v1/chat/completions "HTTP/1.1 20
0 OK"
```

Figure 60 405 error in VPS after sending wrong geolocations

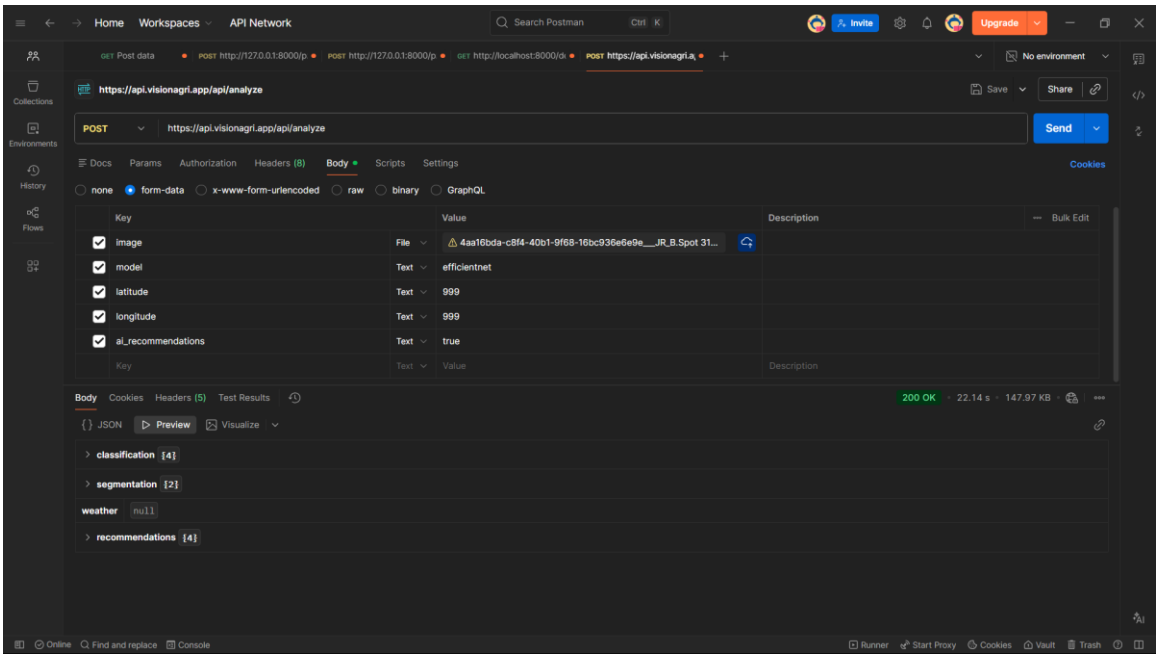


Figure 61 Null weather data is received without breaking the app

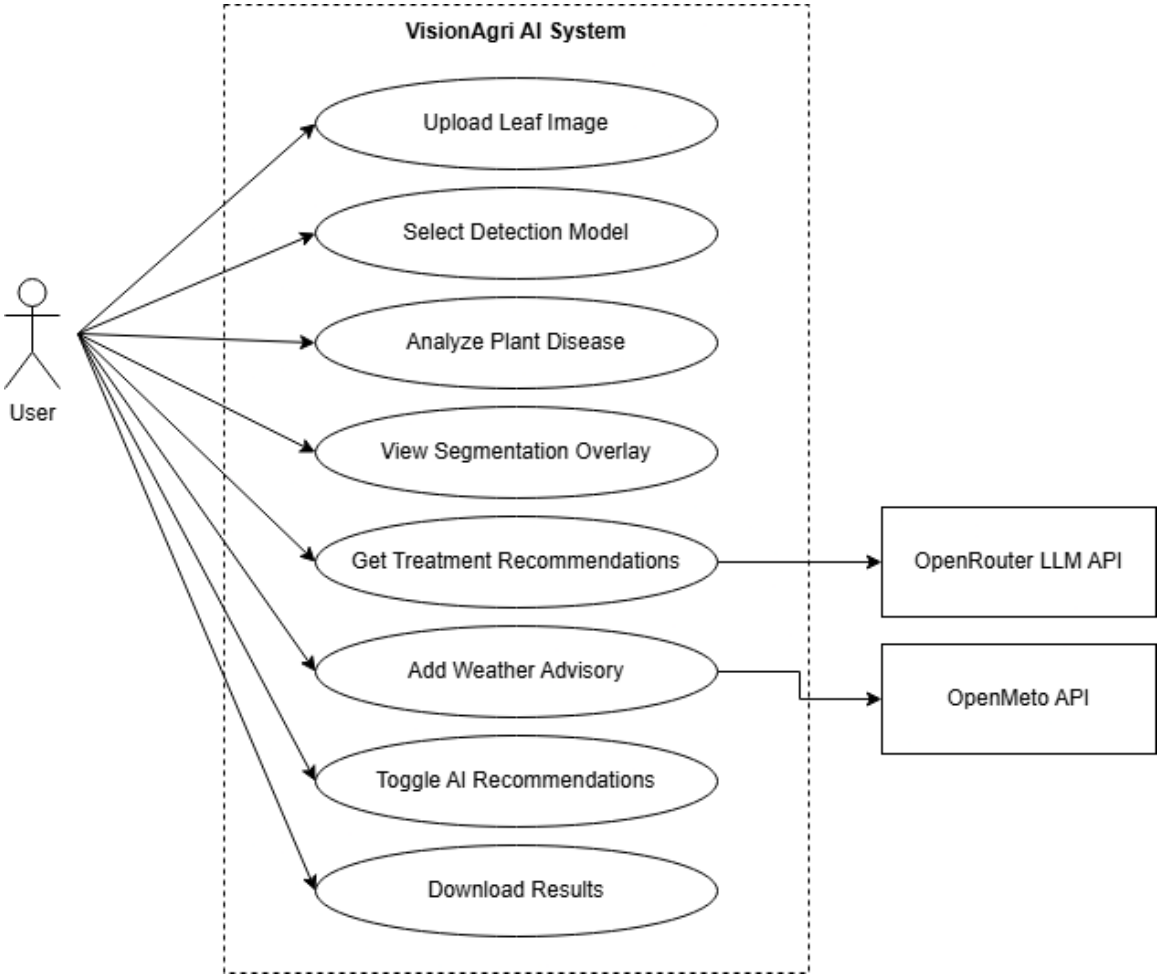


Figure 62 Use case diagram

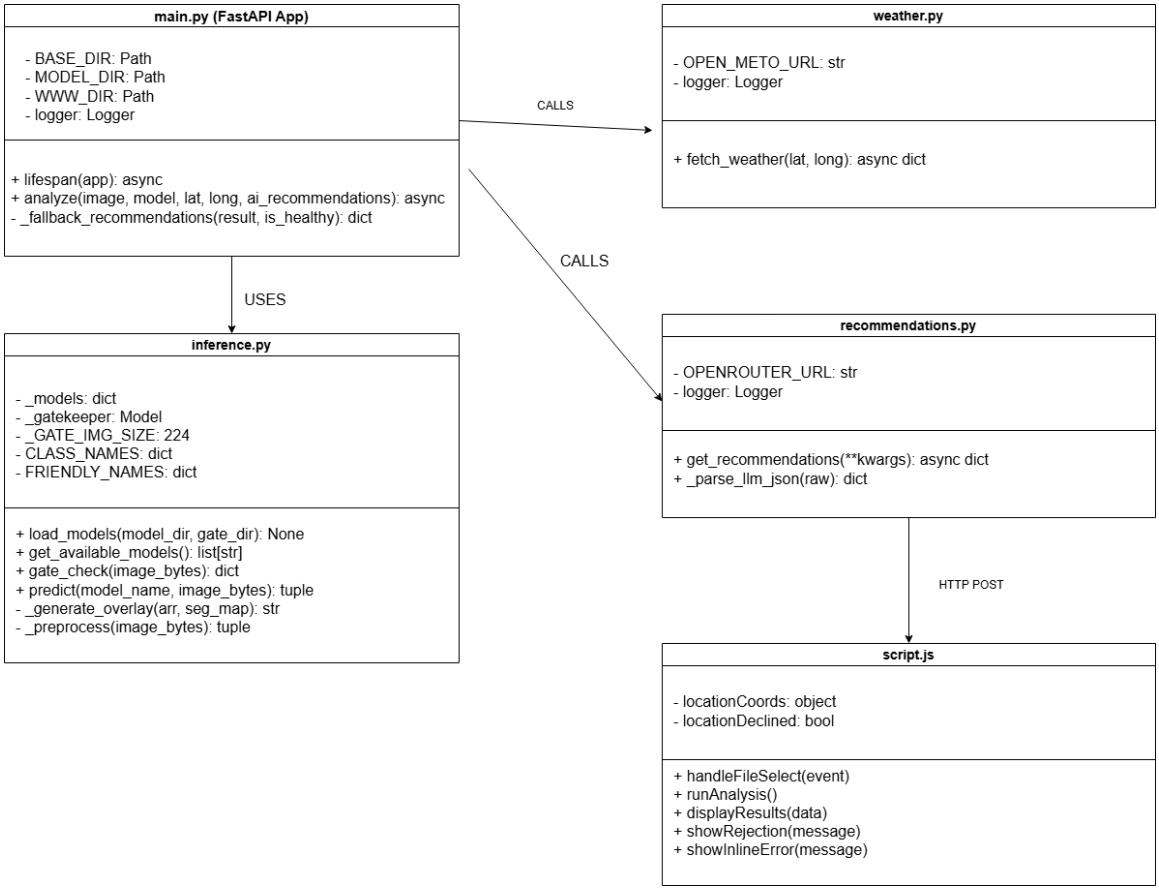
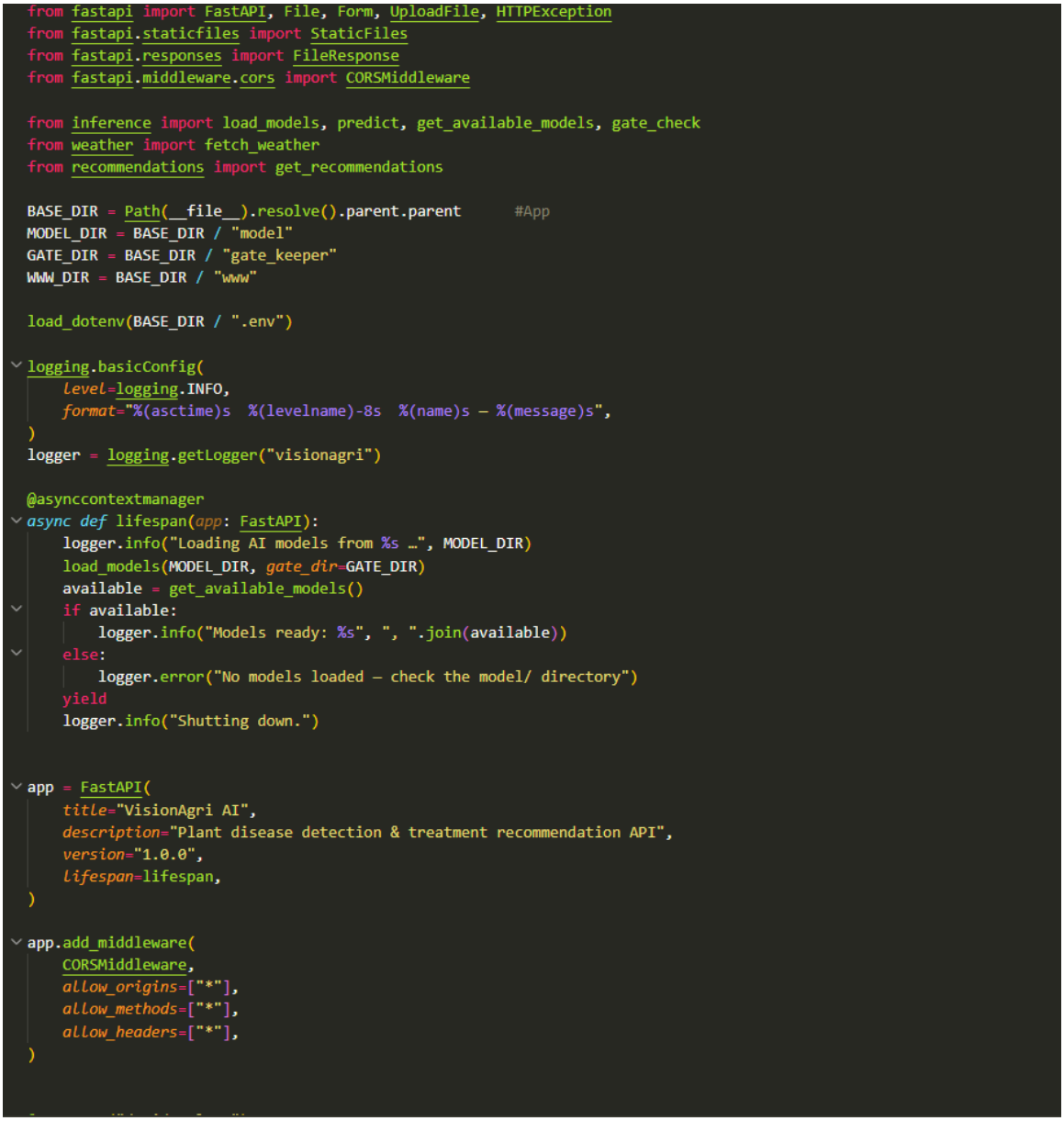


Figure 63 Class diagram



```python
from fastapi import FastAPI, File, Form, UploadFile, HTTPException
from fastapi.staticfiles import StaticFiles
from fastapi.responses import FileResponse
from fastapi.middleware.cors import CORSMiddleware

from inference import load_models, predict, get_available_models, gate_check
from weather import fetch_weather
from recommendations import get_recommendations

BASE_DIR = Path(__file__).resolve().parent.parent      #App
MODEL_DIR = BASE_DIR / "model"
GATE_DIR = BASE_DIR / "gate_keeper"
WWW_DIR = BASE_DIR / "www"

load_dotenv(BASE_DIR / ".env")

logging.basicConfig(
    level=logging.INFO,
    format="%(asctime)s  %(levelname)-8s  %(name)s — %(message)s",
)
logger = logging.getLogger("visionagri")

@asynccontextmanager
async def lifespan(app: FastAPI):
    logger.info("Loading AI models from %s …", MODEL_DIR)
    load_models(MODEL_DIR, gate_dir=GATE_DIR)
    available = get_available_models()
    if available:
        logger.info("Models ready: %s", ", ".join(available))
    else:
        logger.error("No models loaded — check the model/ directory")
    yield
    logger.info("Shutting down.")


app = FastAPI(
    title="VisionAgri AI",
    description="Plant disease detection & treatment recommendation API",
    version="1.0.0",
    lifespan=lifespan,
)

app.add_middleware(
    CORSMiddleware,
    allow_origins=["*"],
    allow_methods=["*"],
    allow_headers=["*"],
)
```

Figure 64 Main.py 1

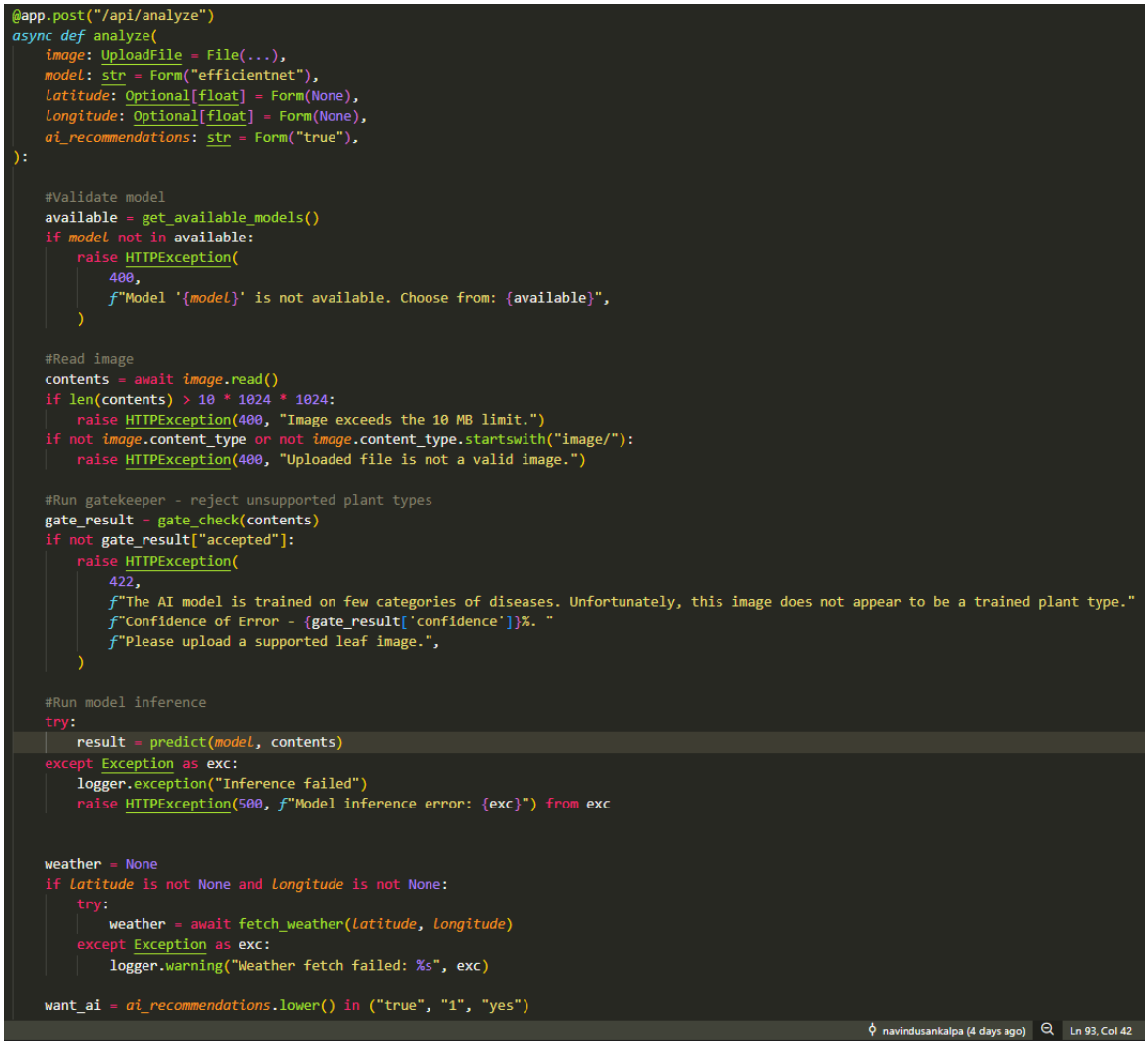


Figure 65 Main.py 2

```python
def predict(model_name: str, image_bytes: bytes) -> dict:
    if model_name not in _models:
        raise ValueError(
            f"Model '{model_name}' is not loaded. "
            f"Available: {get_available_models()}"
        )

    model = _models[model_name]
    img_arr, original_pil = _preprocess(image_bytes)
    input_batch = np.expand_dims(img_arr, axis=0)

    outputs = model.predict(input_batch, verbose=0)

    # Both models define outputs=[seg, cls] so index 0=seg, 1=cls.
    # Double-check with output_names in case order differs.
    out_names = model.output_names
    seg_idx = out_names.index("segmentation") if "segmentation" in out_names else 0
    cls_idx = out_names.index("classification") if "classification" in out_names else 1

    seg_map = outputs[seg_idx][0]   #(256, 256, 3)
    cls_probs = outputs[cls_idx][0]  #(4,)

    #Classification
    predicted_idx = int(np.argmax(cls_probs))
    predicted_class = CLASS_NAMES[predicted_idx]
    confidence = float(cls_probs[predicted_idx]) * 100

    all_scores = {
        FRIENDLY_NAMES[CLASS_NAMES[i]]: round(float(cls_probs[i]) * 100, 2)
        for i in range(len(CLASS_NAMES))
    }

    #Segmentation and severe detection
    disease_pixels = float(np.sum(seg_map[:, :, SEG_DISEASE] > 0.5))
    healthy_pixels = float(np.sum(seg_map[:, :, SEG_HEALTHY] > 0.5))
    leaf_pixels = disease_pixels + healthy_pixels
    severity = (disease_pixels / leaf_pixels * 100) if leaf_pixels > 0 else 0.0

    overlay_b64 = _generate_overlay(img_arr, seg_map)

    return {
        "predicted_class": predicted_class,
        "friendly_name": FRIENDLY_NAMES[predicted_class],
        "confidence": round(confidence, 2)
```

Figure 66 prediction in inference.py

```python
def _generate_overlay(original_arr: np.ndarray, seg_map: np.ndarray) -> str:
    disease_prob = seg_map[:, :, SEG_DISEASE]
    overlay = original_arr.copy()

    # Only colour pixels above visibility threshold
    alpha = np.clip(disease_prob * 1.5, 0, 0.75)

    # Red channel boost, slight green/blue suppression
    overlay[:, :, 0] = overlay[:, :, 0] * (1 - alpha) + alpha          #red
    overlay[:, :, 1] = overlay[:, :, 1] * (1 - alpha * 0.6)            #green
    overlay[:, :, 2] = overlay[:, :, 2] * (1 - alpha)                  #blue

    overlay = np.clip(overlay * 255, 0, 255).astype(np.uint8)
    img = Image.fromarray(overlay)

    buf = io.BytesIO()
    img.save(buf, format="PNG", optimize=True)
    return base64.b64encode(buf.getvalue()).decode("utf-8")
```

Figure 67 Overlay generation

```python
import json
import logging
import os

import httpx

logger = logging.getLogger(__name__)

OPENROUTER_URL = "https://openrouter.ai/api/v1/chat/completions"


async def get_recommendations(
    *,
    disease_name: str,
    confidence: float,
    severity: float,
    is_healthy: bool,
    weather: dict | None = None,
) -> dict:
    api_key = os.getenv("OPENROUTER_API_KEY")
    model = os.getenv("OPENROUTER_MODEL", "openai/gpt-4o-mini")

    if not api_key:
        raise ValueError("OPENROUTER_API_KEY environment variable is not set")

    prompt = _build_prompt(disease_name, confidence, severity, is_healthy, weather)

    headers = {
        "Authorization": f"Bearer {api_key}",
        "Content-Type": "application/json",
        "HTTP-Referer": "https://visionagri-ai.local",
        "X-Title": "VisionAgri AI",
    }

    payload = {
        "model": model,
        "messages": [
            {
                "role": "system",
                "content": (
                    "You are a plant pathology expert. "
                    "Always respond with valid JSON only — no markdown, no code fences."
                ),
            },
            {"role": "user", "content": prompt},
        ],
        "temperature": 0.3,
        "max_tokens": 5000,
```

Figure 68 Recommendations.py

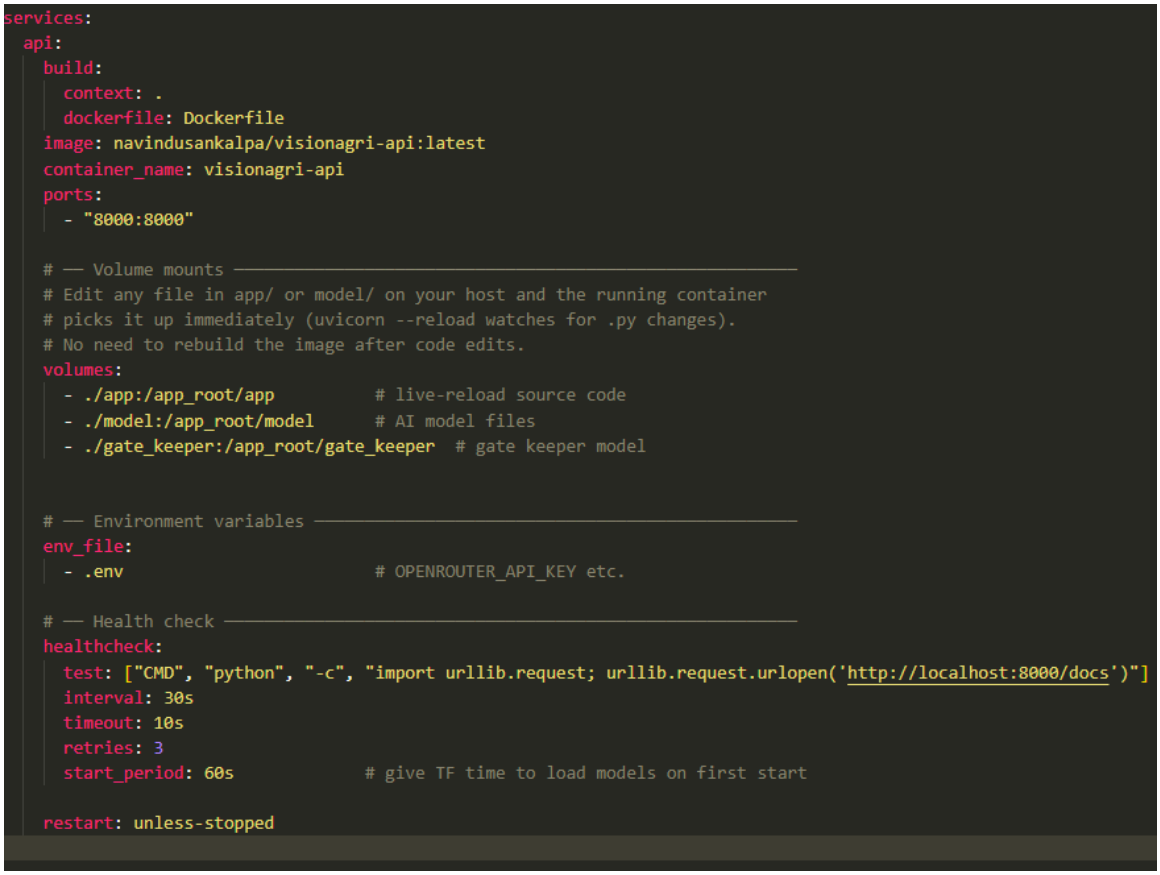
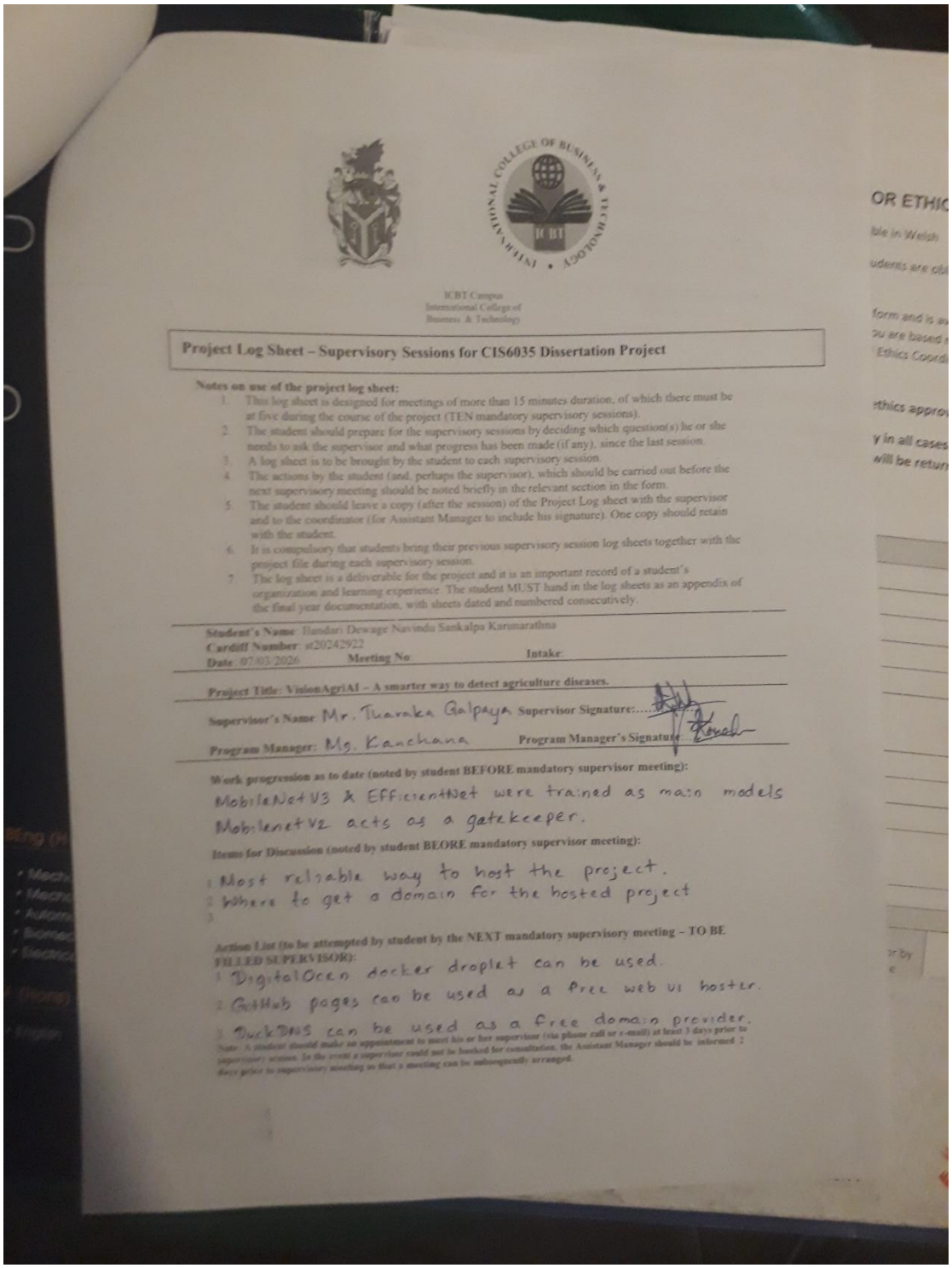


Figure 69 docker-compose file

```dockerfile
FROM python:3.10-slim

# System dependencies (needed by TensorFlow / Pillow)
RUN apt-get update && apt-get install -y --no-install-recommends \
        libgl1 \
        libglib2.0-0 \
    && rm -rf /var/lib/apt/lists/*

# ── Python dependencies ──────────────────────────────────────────
# Copy only requirements first so Docker can cache this layer.
# Rebuild this layer only when requirements.txt changes.
COPY app/requirements.txt /tmp/requirements.txt
RUN pip install --no-cache-dir -r /tmp/requirements.txt

# ── Directory layout ─────────────────────────────────────────────
# main.py resolves BASE_DIR = Path(__file__).parent.parent
#   => BASE_DIR = /app_root
#   => MODEL_DIR = /app_root/model
#   => WWW_DIR   = /app_root/www  (made optional – see main.py)
WORKDIR /app_root/app

# Create the model and www directories so the container starts cleanly
# even before volumes are attached for the first time.
RUN mkdir -p /app_root/model /app_root/gate_keeper /app_root/www

# Copy application source (overridden by the volume mount in development)
COPY app/ /app_root/app/

# Copy model files into the image (required for cloud deployments without volume mounts)
COPY model/ /app_root/model/
COPY gate_keeper/ /app_root/gate_keeper/

# ── Runtime ──────────────────────────────────────────────────────
EXPOSE 8000

# Reduce TensorFlow memory footprint for low-RAM deployments
ENV TF_CPP_MIN_LOG_LEVEL=2
ENV TF_ENABLE_ONEDNN_OPTS=0
ENV MALLOC_TRIM_THRESHOLD_=65536

# Production: no --reload, single worker, longer timeout for slow inference
CMD ["uvicorn", "main:app", "--host", "0.0.0.0", "--port", "8000", "--workers", "1", "--timeout-keep-alive", "120"]
```

Figure 70 Dockerfile

APPENDIX 2 – Supervisor Meeting Log Sheet



ICBT Campus
International College of
Business & Technology

Project Log Sheet – Supervisory Sessions for CIS6035 Dissertation Project

Notes on use of the project log sheet:

1. This log sheet is designed for meetings of more than 15 minutes duration, of which there must be at five during the course of the project (TEN mandatory supervisory sessions).
2. The student should prepare for the supervisory sessions by deciding which question(s) he or she needs to ask the supervisor and what progress has been made (if any), since the last session.
3. A log sheet is to be brought by the student to each supervisory session.
4. The actions by the student (and, perhaps the supervisor), which should be carried out before the next supervisory meeting should be noted briefly in the relevant section in the form.
5. The student should leave a copy (after the session) of the Project Log sheet with the supervisor and to the coordinator (for Assistant Manager to include his signature). One copy should retain with the student.
6. It is compulsory that students bring their previous supervisory session log sheets together with the project file during each supervisory session.
7. The log sheet is a deliverable for the project and it is an important record of a student's organization and learning experience. The student MUST hand in the log sheets as an appendix of the final year documentation, with sheets dated and numbered consecutively.

Student's Name: Ilandari Dewage Navindu Sankalpa Karunarathna
Cardiff Number: st20242922
Date: 07/03/2026 **Meeting No** **Intake**:

Project Title: VisionAgriAI – A smarter way to detect agriculture diseases.

Supervisor's Name: Mr. Tharaka Galpaya **Supervisor Signature**:

Program Manager: Ms. Kanchana **Program Manager's Signature**:

Work progression as to date (noted by student BEFORE mandatory supervisor meeting):

MobileNetV3 & EfficientNet were trained as main models
Mobilenetv2 acts as a gatekeeper.

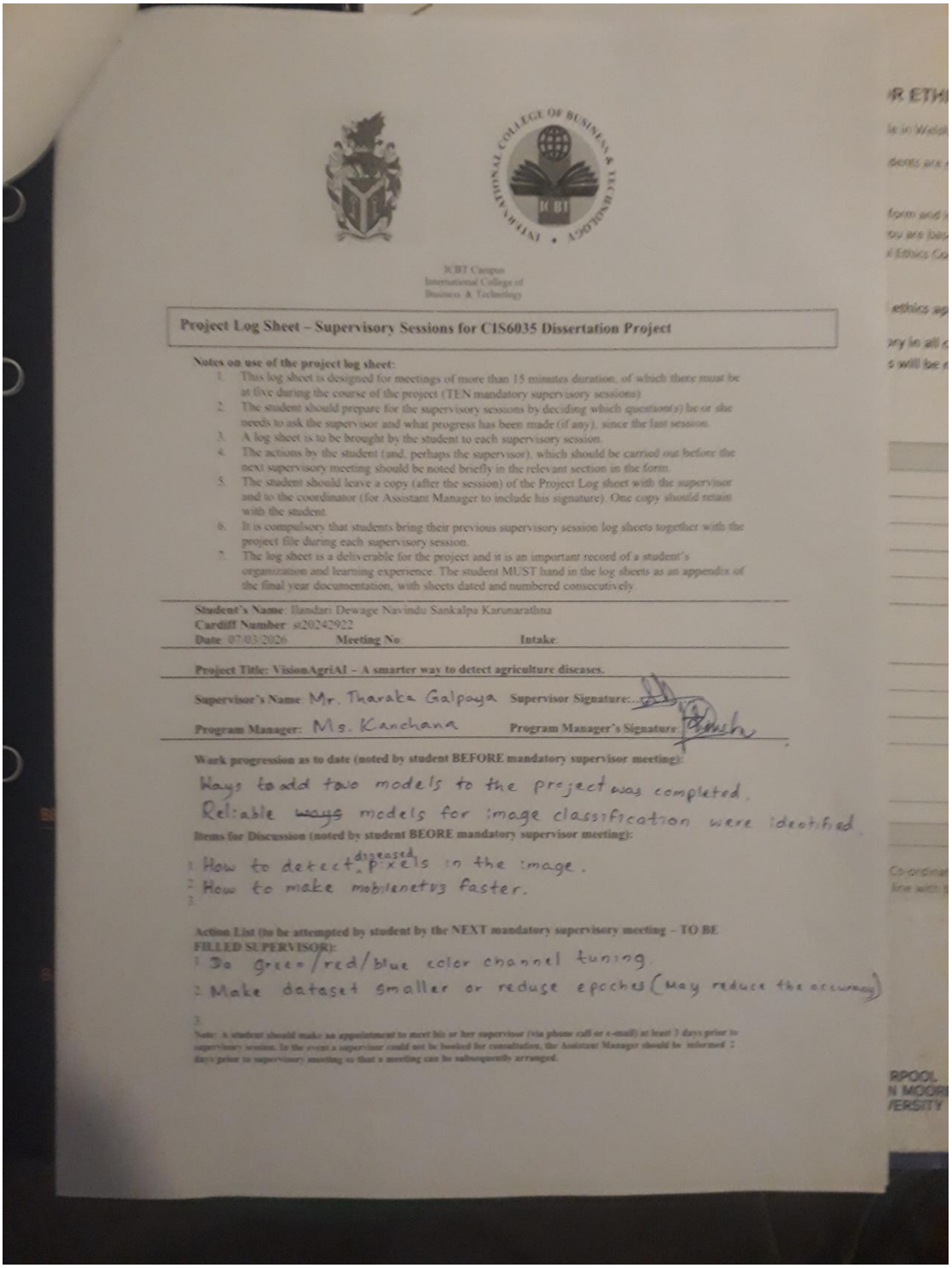
Items for Discussion (noted by student BEORE mandatory supervisor meeting):

1. Most relyable way to host the project.
2. Where to get a domain for the hosted project
3.

Action List (to be attempted by student by the NEXT mandatory supervisory meeting – TO BE FILLED SUPERVISOR):

1. DigitalOcen docker droplet can be used.
2. GitHub pages can be used as a free web ui hoster.
3. DuckDNS can be used as a free domain provider.

Note: A student should make an appointment to meet his or her supervisor (via phone call or e-mail) at least 3 days prior to supervisory session. In the event a supervisor could not be booked for consultation, the Assistant Manager should be informed 2 days prior to supervisory meeting so that a meeting can be subsequently arranged.



ICBT Campus
International College of
Business & Technology

Project Log Sheet – Supervisory Sessions for CIS6035 Dissertation Project

Notes on use of the project log sheet:

1. This log sheet is designed for meetings of more than 15 minutes duration, of which there must be at five during the course of the project (TEN mandatory supervisory sessions).
2. The student should prepare for the supervisory sessions by deciding which question(s) he or she needs to ask the supervisor and what progress has been made (if any), since the last session.
3. A log sheet is to be brought by the student to each supervisory session.
4. The actions by the student (and, perhaps the supervisor), which should be carried out before the next supervisory meeting should be noted briefly in the relevant section in the form.
5. The student should leave a copy (after the session) of the Project Log sheet with the supervisor and to the coordinator (for Assistant Manager to include his signature). One copy should retain with the student.
6. It is compulsory that students bring their previous supervisory session log sheets together with the project file during each supervisory session.
7. The log sheet is a deliverable for the project and it is an important record of a student's organization and learning experience. The student MUST hand in the log sheets as an appendix of the final year documentation, with sheets dated and numbered consecutively.

Student's Name: Ilandari Dewage Navindu Sankalpa Karunarathna
Cardiff Number: st20242922
Date: 07/03/2026 **Meeting No**: **Intake**:

Project Title: VisionAgriAI – A smarter way to detect agriculture diseases.

Supervisor's Name: Mr. Tharaka Galpaya **Supervisor Signature**:

Program Manager: Ms. Kanchana **Program Manager's Signature**:

Work progression as to date (noted by student BEFORE mandatory supervisor meeting):

Ways to add two models to the project was completed.
Reliable ways models for image classification were identified.

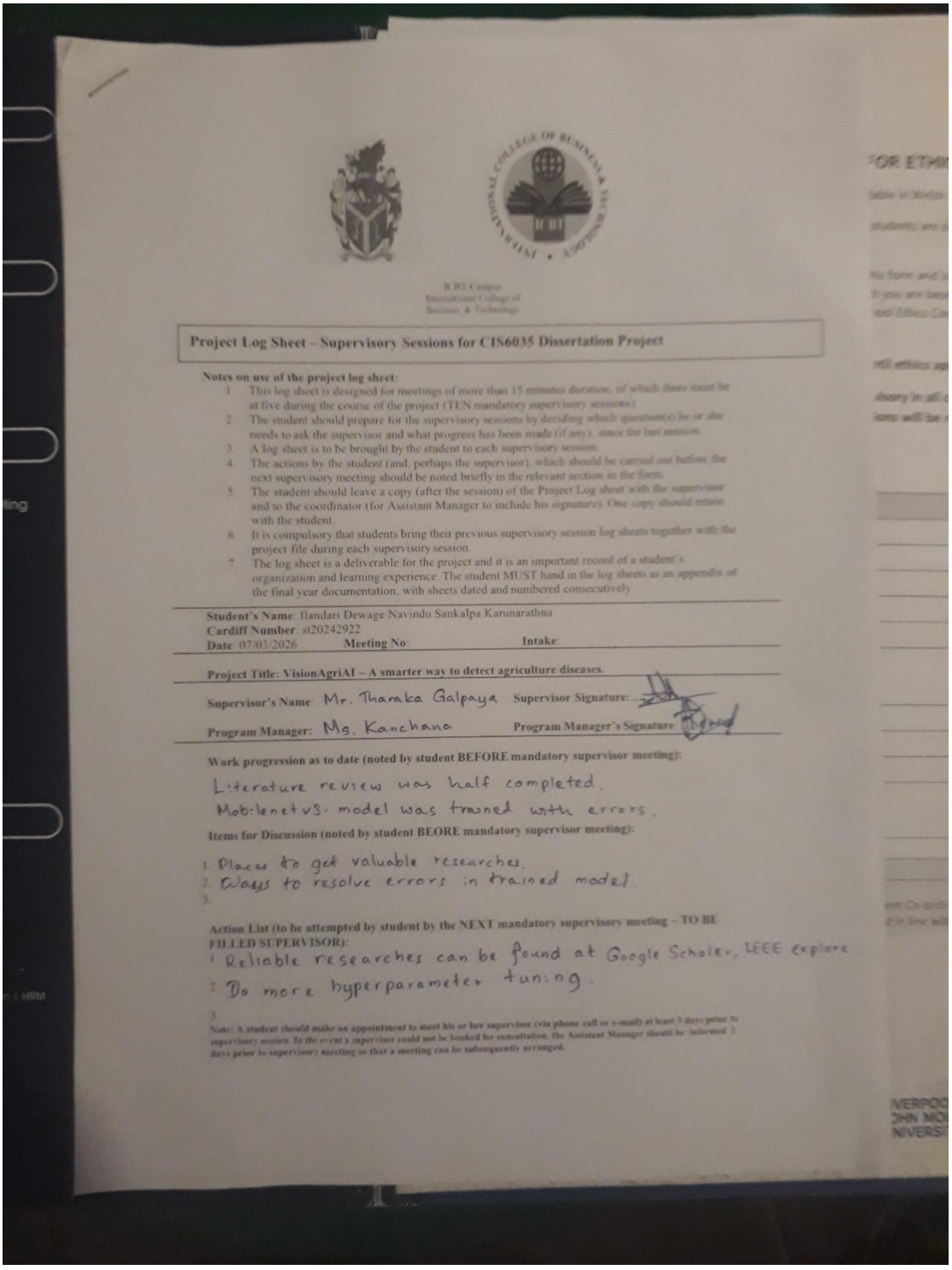
Items for Discussion (noted by student BEORE mandatory supervisor meeting):

1. How to detect diseased pixels in the image.
2. How to make mobilenetv3 faster.
3.

Action List (to be attempted by student by the NEXT mandatory supervisory meeting – TO BE FILLED SUPERVISOR):

1. Do green/red/blue color channel tuning.
2. Make dataset smaller or reduse epoches (May reduce the accuracy)
3.

Note: A student should make an appointment to meet his or her supervisor (via phone call or e-mail) at least 3 days prior to supervisory session. In the event a supervisor could not be booked for consultation, the Assistant Manager should be informed 2 days prior to supervisory meeting so that a meeting can be subsequently arranged.



IC BT Campus
International College of
Business & Technology

Project Log Sheet – Supervisory Sessions for CIS6035 Dissertation Project

Notes on use of the project log sheet:

1. This log sheet is designed for meetings of more than 15 minutes duration, of which there must be at five during the course of the project (TEN mandatory supervisory sessions).
2. The student should prepare for the supervisory sessions by deciding which question(s) he or she needs to ask the supervisor and what progress has been made (if any), since the last session.
3. A log sheet is to be brought by the student to each supervisory session.
4. The actions by the student (and, perhaps the supervisor), which should be carried out before the next supervisory meeting should be noted briefly in the relevant section in the form.
5. The student should leave a copy (after the session) of the Project Log sheet with the supervisor and to the coordinator (for Assistant Manager to include his signature). One copy should retain with the student.
6. It is compulsory that students bring their previous supervisory session log sheets together with the project file during each supervisory session.
7. The log sheet is a deliverable for the project and it is an important record of a student's organization and learning experience. The student MUST hand in the log sheets as an appendix of the final year documentation, with sheets dated and numbered consecutively.

Student's Name: Ilandari Dewage Navindu Sankalpa Karunarathna
Cardiff Number: st20242922
Date: 07/03/2026 **Meeting No**: **Intake**:

Project Title: VisionAgriAI – A smarter way to detect agriculture diseases.

Supervisor's Name: Mr. Tharaka Galpaya **Supervisor Signature**:

Program Manager: Ms. Kanchana **Program Manager's Signature**:

Work progression as to date (noted by student BEFORE mandatory supervisor meeting):

Literature review was half completed.
Mobilenet v3 model was trained with errors.

Items for Discussion (noted by student BEORE mandatory supervisor meeting):

1. Places to get valuable researches.
2. Ways to resolve errors in trained model.
3.

Action List (to be attempted by student by the NEXT mandatory supervisory meeting – TO BE FILLED SUPERVISOR):

1. Reliable researches can be found at Google Scholer, IEEE explore.
2. Do more hyperparameter tuning.
3.

Note: A student should make an appointment to meet his or her supervisor (via phone call or e-mail) at least 3 days prior to supervisory session. In the event a supervisor could not be booked for consultation, the Assistant Manager should be informed 2 days prior to supervisory meeting so that a meeting can be subsequently arranged.